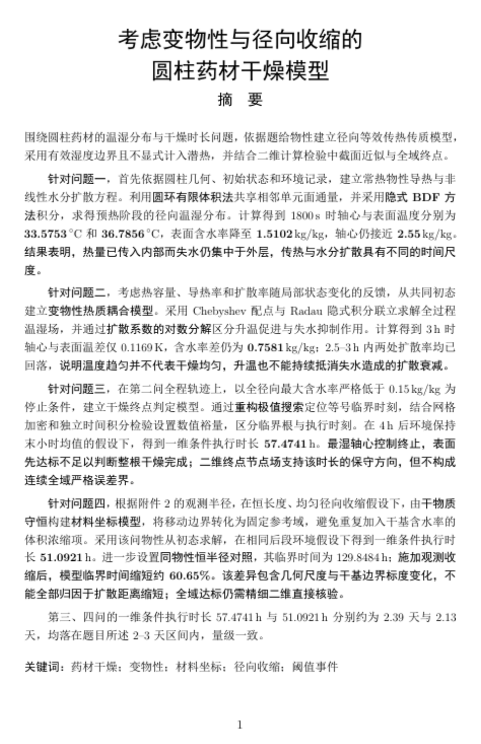
1



考虑变物性与径向收缩的圆柱药材干燥模型

摘 要

围绕圆柱药材的温湿分布与干燥时长问题，依据题给物性建立径向等效传热传质模型，采用有效湿度边界且不显式计入潜热，并结合二维计算检验中截面近似与全域终点。

针对问题一，首先依据圆柱几何、初始状态和环境记录，建立常热物性导热与非线性水分扩散方程。利用**圆环有限体积法**共享相邻单元面通量，并采用**隐式 BDF 方法**积分，求得预热阶段的径向温湿分布。计算得到 1800 s 时轴心与表面温度分别为 **33.5753** °C 和 **36.7856** °C，表面含水率降至 **1.5102** kg/kg，轴心仍接近 **2.55** kg/kg。**结果表明，热量已传入内部而失水仍集中于外层，传热与水分扩散具有不同的时间尺度。**

针对问题二，考虑热容量、导热率和扩散率随局部状态变化的反馈，从共同初态建立**变物性热质耦合模型**。采用 Chebyshev 配点与 Radau 隐式积分联立求解全过程温湿场，并通过**扩散系数的对数分解**区分升温促进与失水抑制作用。计算得到 3 h 时轴心与表面温差仅 0.1169 K，含水率差仍为 **0.7581** kg/kg；2.5–3 h 内两处扩散率均已回落，**说明温度趋匀并不代表干燥均匀，升温也不能持续抵消失水造成的扩散衰减。**

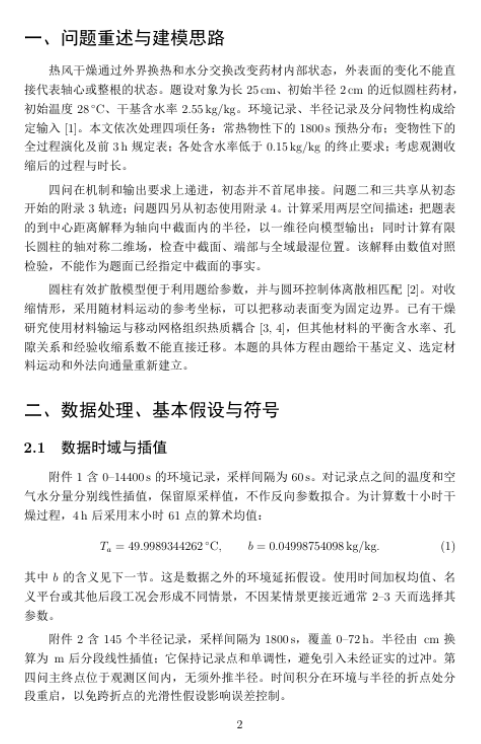
针对问题三，在第二问全程轨迹上，以全径向最大含水率严格低于 0.15 kg/kg 为停止条件，建立干燥终点判定模型。通过**重构极值搜索**定位等号临界时刻，结合网格加密和独立时间积分检验设置数值裕量，区分临界根与执行时刻。在 4 h 后环境保持末小时均值的假设下，得到一维条件执行时长 **57.4741** h。**最湿轴心控制终止，表面先达标不足以判断整根干燥完成；二维终点节点场支持该时长的保守方向，但不构成连续全域严格误差界。**

针对问题四，根据附件 2 的观测半径，在恒长度、均匀径向收缩假设下，由**干物质守恒**构建**材料坐标模型**，将移动边界转化为固定参考域，避免重复加入干基含水率的体积浓缩项。采用该问物性从初态求解，在相同后段环境假设下得到一维条件执行时长 **51.0921** h。进一步设置**同物性恒半径对照**，其临界时间为 129.8484 h；**施加观测收缩后，模型临界时间缩短约 60.65%。该差异包含几何尺度与干基边界标度变化，不能全部归因于扩散距离缩短；全域达标仍需精细二维直接核验。**

第三、四问的一维条件执行时长 57.4741 h 与 51.0921 h 分别约为 2.39 天与 2.13 天，均落在题目所述 2–3 天区间内，量级一致。

关键词：药材干燥；变物性；材料坐标；径向收缩；阈值事件

2



一、问题重述与建模思路

热风干燥通过外界换热和水分交换改变药材内部状态，外表面的变化不能直接代表轴心或整根的状态。题设对象为长 25 cm、初始半径 2 cm 的近似圆柱药材，初始温度 28 °C、干基含水率 2.55 kg/kg。环境记录、半径记录及分问物性构成给定输入 [1]。本文依次处理四项任务：常热物性下的 1800 s 预热分布；变物性下的全过程演化及前 3 h 规定表；各处含水率低于 0.15 kg/kg 的终止要求；考虑观测收缩后的过程与时长。

四问在机制和输出要求上递进，初态并不首尾串接。问题二和三共享从初态开始的附录 3 轨迹；问题四另从初态使用附录 4。计算采用两层空间描述：把题表的到中心距离解释为轴向中截面内的半径，以一维径向模型输出；同时计算有限长圆柱的轴对称二维场，检查中截面、端部与全域最湿位置。该解释由数值对照检验，不能作为题面已经指定中截面的事实。

圆柱有效扩散模型便于利用题给参数，并与圆环控制体离散相匹配 [2]。对收缩情形，采用随材料运动的参考坐标，可以把移动表面变为固定边界。已有干燥研究使用材料输运与移动网格组织热质耦合 [3, 4]，但其他材料的平衡含水率、孔隙关系和经验收缩系数不能直接迁移。本题的具体方程由题给干基定义、选定材料运动和外法向通量重新建立。

二、数据处理、基本假设与符号

2.1 数据时域与插值

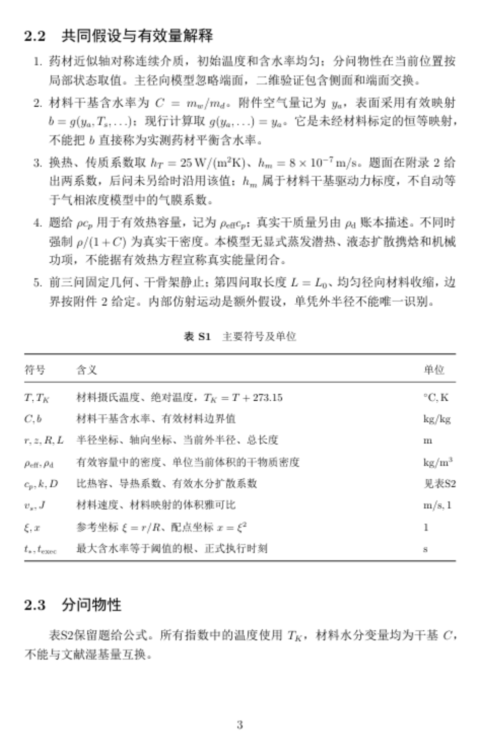
附件 1 含 0–14400 s 的环境记录，采样间隔为 60 s。对记录点之间的温度和空气水分量分别线性插值，保留原采样值，不作反向参数拟合。为计算数十小时干燥过程，4 h 后采用末小时 61 点的算术均值：

$$T_a = 49.9989344262\,^\circ\mathrm{C}, \qquad b = 0.04998754098\,\mathrm{kg/kg}. \tag{1}$$

其中 b 的含义见下一节。这是数据之外的环境延拓假设。使用时间加权均值、名义平台或其他后段工况会形成不同情景，不因某情景更接近通常 2–3 天而选择其参数。

附件 2 含 145 个半径记录，采样间隔为 1800 s，覆盖 0–72 h。半径由 cm 换算为 m 后分段线性插值；它保持记录点和单调性，避免引入未经证实的过冲。第四问主终点位于观测区间内，无须外推半径。时间积分在环境与半径的折点处分段重启，以免跨折点的光滑性假设影响误差控制。

3



2.2 共同假设与有效量解释

1. 药材近似轴对称连续介质，初始温度和含水率均匀；分问物性在当前位置按局部状态取值。主径向模型忽略端面，二维验证包含侧面和端面交换。
2. 材料干基含水率为 $C = m_w/m_d$。附件空气量记为 y_a，表面采用有效映射 $b = g(y_a, T_s, \ldots)$；现行计算取 $g(y_a, \ldots) = y_a$。它是未经材料标定的恒等映射，不能把 b 直接称为实测药材平衡含水率。
3. 换热、传质系数取 $h_T = 25\,\mathrm{W/(m^2K)}$、$h_m = 8 \times 10^{-7}\,\mathrm{m/s}$。题面在附录 2 给出两系数，后问未另给时沿用该值；h_m 属于材料干基驱动力标度，不自动等于气相浓度模型中的气膜系数。
4. 题给 ρc_p 用于有效热容量，记为 $\rho_{\mathrm{eff}} c_p$；真实干质量另由 ρ_{d} 账本描述。不同时强制 $\rho/(1+C)$ 为真实干密度。本模型无显式蒸发潜热、液态扩散携焓和机械功项，不能据有效热方程宣称真实能量闭合。
5. 前三问固定几何、干骨架静止；第四问取长度 $L = L_0$、均匀径向材料收缩，边界按附件 2 给定。内部仿射运动是额外假设，单凭外半径不能唯一识别。

表 S1 主要符号及单位

符号	含义	单位
T, T_K	材料摄氏温度、绝对温度，$T_K = T + 273.15$	°C, K
C, b	材料干基含水率、有效材料边界值	kg/kg
r, z, R, L	半径坐标、轴向坐标、当前外半径、总长度	m
$\rho_{\mathrm{eff}}, \rho_{\mathrm{d}}$	有效容量中的密度、单位当前体积的干物质密度	$\mathrm{kg/m^3}$
c_p, k, D	比热容、导热系数、有效水分扩散系数	见表S2
v_s, J	材料速度、材料映射的体积雅可比	m/s, 1
ξ, x	参考坐标 $\xi = r/R$、配点坐标 $x = \xi^2$	1
t_*, t_{exec}	最大含水率等于阈值的根、正式执行时刻	s

2.3 分问物性

表S2保留题给公式。所有指数中的温度使用 T_K，材料水分变量均为干基 C，不能与文献湿基量互换。

4

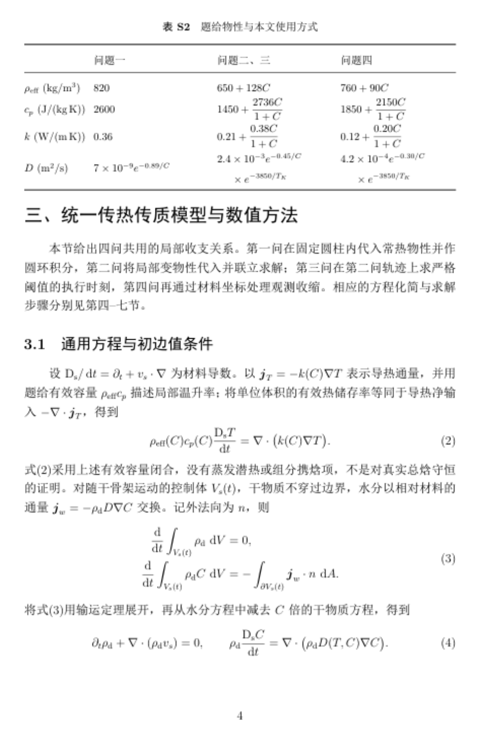


表 S2 题给物性与本文使用方式

	问题一	问题二、三	问题四
ρ_{eff} $(\mathrm{kg/m^3})$	820	$650 + 128C$	$760 + 90C$
c_p $(\mathrm{J/(kg\,K)})$	2600	$1450 + \dfrac{2736C}{1+C}$	$1850 + \dfrac{2150C}{1+C}$
k $(\mathrm{W/(m\,K)})$	0.36	$0.21 + \dfrac{0.38C}{1+C}$	$0.12 + \dfrac{0.20C}{1+C}$
D $(\mathrm{m^2/s})$	$7 \times 10^{-9} e^{-0.89/C}$	$2.4 \times 10^{-3} e^{-0.45/C}$ $\times e^{-3850/T_K}$	$4.2 \times 10^{-4} e^{-0.30/C}$ $\times e^{-3850/T_K}$

三、统一传热传质模型与数值方法

本节给出四问共用的局部收支关系。第一问在固定圆柱内代入常热物性并作圆环积分，第二问将局部变物性代入并联立求解；第三问在第二问轨迹上求严格阈值的执行时刻，第四问再通过材料坐标处理观测收缩。相应的方程化简与求解步骤分别见第四–七节。

3.1 通用方程与初边值条件

设 $\mathrm{D}_s/\mathrm{d}t = \partial_t + v_s \cdot \nabla$ 为材料导数。以 $\boldsymbol{j}_T = -k(C)\nabla T$ 表示导热通量，并用题给有效容量 $\rho_{\mathrm{eff}} c_p$ 描述局部温升率；将单位体积的有效热储存率等同于导热净输入 $-\nabla \cdot \boldsymbol{j}_T$，得到

$$\rho_{\mathrm{eff}}(C) c_p(C) \frac{\mathrm{D}_s T}{\mathrm{d}t} = \nabla \cdot \big(k(C)\nabla T\big). \tag{2}$$

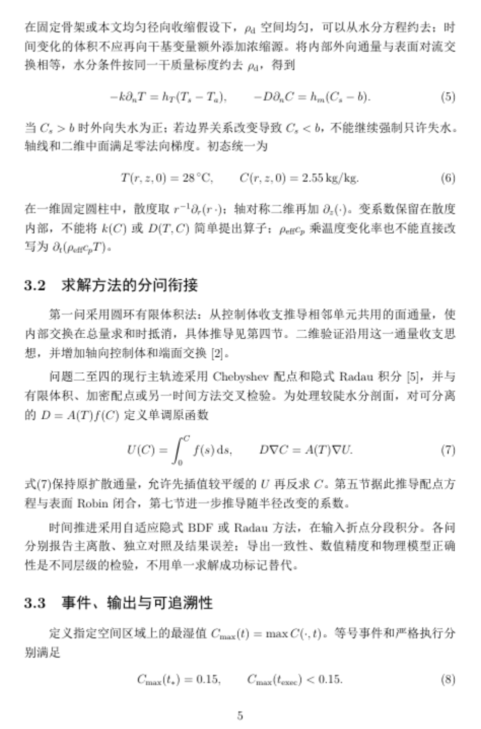
式(2)采用上述有效容量闭合，没有蒸发潜热或组分携焓项，不是对真实总焓守恒的证明。对随干骨架运动的控制体 $V_s(t)$，干物质不穿过边界，水分以相对材料的通量 $\boldsymbol{j}_w = -\rho_{\mathrm{d}} D \nabla C$ 交换。记外法向为 n，则

$$\begin{aligned} \frac{\mathrm{d}}{\mathrm{d}t}\int_{V_s(t)} \rho_{\mathrm{d}}\,\mathrm{d}V &= 0, \\ \frac{\mathrm{d}}{\mathrm{d}t}\int_{V_s(t)} \rho_{\mathrm{d}} C\,\mathrm{d}V &= -\int_{\partial V_s(t)} \boldsymbol{j}_w \cdot n\,\mathrm{d}A. \end{aligned} \tag{3}$$

将式(3)用输运定理展开，再从水分方程中减去 C 倍的干物质方程，得到

$$\partial_t \rho_{\mathrm{d}} + \nabla \cdot (\rho_{\mathrm{d}} v_s) = 0, \qquad \rho_{\mathrm{d}} \frac{\mathrm{D}_s C}{\mathrm{d}t} = \nabla \cdot \big(\rho_{\mathrm{d}} D(T, C) \nabla C\big). \tag{4}$$

5



在固定骨架或本文均匀径向收缩假设下，ρ_{d} 空间均匀，可以从水分方程约去；时间变化的体积不应再向干基变量额外添加浓缩源。将内部外向通量与表面对流交换相等，水分条件按同一干质量标度约去 ρ_{d}，得到

$$-k\partial_n T = h_T(T_s - T_a), \qquad -D\partial_n C = h_m(C_s - b). \tag{5}$$

当 $C_s > b$ 时外向失水为正；若边界关系改变导致 $C_s < b$，不能继续强制只许失水。轴线和二维中面满足零法向梯度。初态统一为

$$T(r, z, 0) = 28\,^\circ\mathrm{C}, \qquad C(r, z, 0) = 2.55\,\mathrm{kg/kg}. \tag{6}$$

在一维固定圆柱中，散度取 $r^{-1}\partial_r(r\,\cdot)$；轴对称二维再加 $\partial_z(\cdot)$。变系数保留在散度内部，不能将 $k(C)$ 或 $D(T, C)$ 简单提出算子；$\rho_{\mathrm{eff}} c_p$ 乘温度变化率也不能直接改写为 $\partial_t(\rho_{\mathrm{eff}} c_p T)$。

3.2 求解方法的分问衔接

第一问采用圆环有限体积法：从控制体收支推导相邻单元共用的面通量，使内部交换在总量求和时抵消，具体推导见第四节。二维验证沿用这一通量收支思想，并增加轴向控制体和端面交换 [2]。

问题二至四的现行主轨迹采用 Chebyshev 配点和隐式 Radau 积分 [5]，并与有限体积、加密配点或另一时间方法交叉检验。为处理较陡水分剖面，对可分离的 $D = A(T)f(C)$ 定义单调原函数

$$U(C) = \int_0^C f(s)\,\mathrm{d}s, \qquad D\nabla C = A(T)\nabla U. \tag{7}$$

式(7)保持原扩散通量，允许先插值较平缓的 U 再反求 C。第五节据此推导配点方程与表面 Robin 闭合，第七节进一步推导随半径改变的系数。

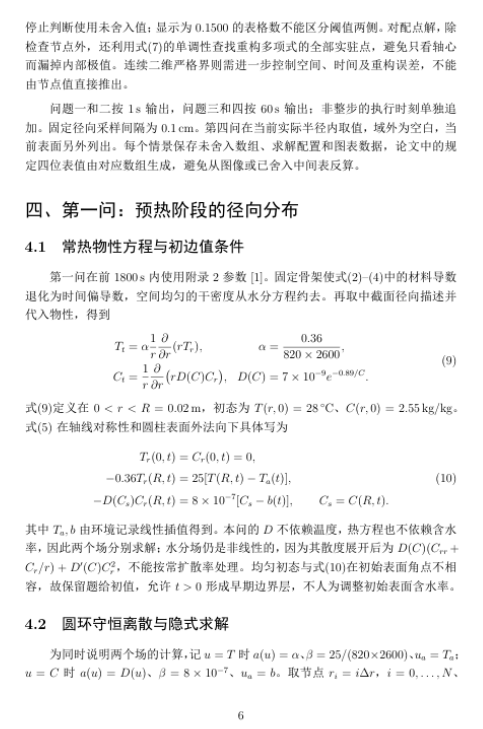
时间推进采用自适应隐式 BDF 或 Radau 方法，在输入折点分段积分。各问分别报告主离散、独立对照及结果误差：导出一致性、数值精度和物理模型正确性是不同层级的检验，不用单一求解成功标记替代。

3.3 事件、输出与可追溯性

定义指定空间区域上的最湿值 $C_{\max}(t) = \max C(\cdot, t)$。等号事件和严格执行分别满足

$$C_{\max}(t_*) = 0.15, \qquad C_{\max}(t_{\mathrm{exec}}) < 0.15. \tag{8}$$

6



停止判断使用未舍入值；显示为 0.1500 的表格数不能区分阈值两侧。对配点解，除检查节点外，还利用式(7)的单调性查找重构多项式的全部实驻点，避免只看轴心而漏掉内部极值。连续二维严格界则需进一步控制空间、时间及重构误差，不能由节点值直接推出。

问题一和二按 1 s 输出，问题三和四按 60 s 输出；非整步的执行时刻单独追加。固定径向采样间隔为 0.1 cm。第四问在当前实际半径内取值，域外为空白，当前表面另外列出。每个情景保存未舍入数组、求解配置和图表数据，论文中的规定四位表值由对应数组生成，避免从图像或已舍入中间表反算。

四、第一问：预热阶段的径向分布

4.1 常热物性方程与初边值条件

第一问在前 1800 s 内使用附录 2 参数 [1]。固定骨架使式(2)–(4)中的材料导数退化为时间偏导数，空间均匀的干密度从水分方程约去。再取中截面径向描述并代入物性，得到

$$\begin{aligned} T_t &= \alpha \frac{1}{r}\frac{\partial}{\partial r}(rT_r), & \alpha &= \frac{0.36}{820 \times 2600}, \\ C_t &= \frac{1}{r}\frac{\partial}{\partial r}\big(rD(C)C_r\big), & D(C) &= 7 \times 10^{-9} e^{-0.89/C}. \end{aligned} \tag{9}$$

式(9)定义在 $0 < r < R = 0.02\,\mathrm{m}$，初态为 $T(r, 0) = 28\,^\circ\mathrm{C}$、$C(r, 0) = 2.55\,\mathrm{kg/kg}$。式(5) 在轴线对称性和圆柱表面外法向下具体写为

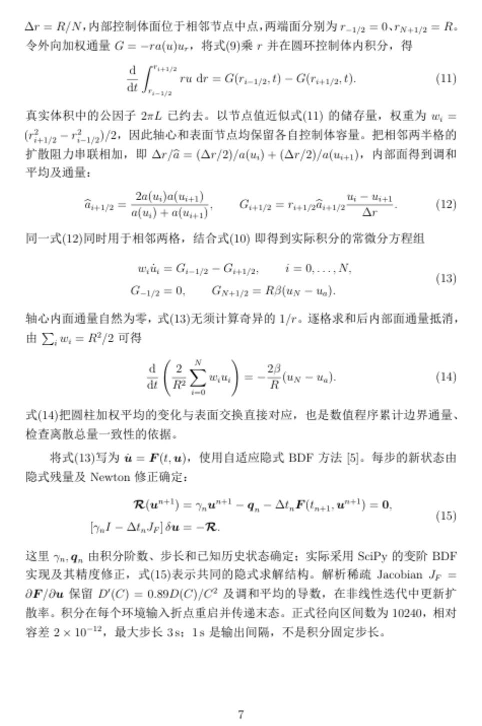
$$\begin{aligned} T_r(0, t) = C_r(0, t) &= 0, \\ -0.36T_r(R, t) &= 25[T(R, t) - T_a(t)], \\ -D(C_s)C_r(R, t) &= 8 \times 10^{-7}[C_s - b(t)], \qquad C_s = C(R, t). \end{aligned} \tag{10}$$

其中 T_a, b 由环境记录线性插值得到。本问的 D 不依赖温度，热方程也不依赖含水率，因此两个场分别求解；水分场仍是非线性的，因为其散度展开后为 $D(C)(C_{rr} + C_r/r) + D'(C)C_r^2$，不能按常扩散率处理。均匀初态与式(10)在初始表面角点不相容，故保留题给初值，允许 $t > 0$ 形成早期边界层，不人为调整初始表面含水率。

4.2 圆环守恒离散与隐式求解

为同时说明两个场的计算，记 $u = T$ 时 $a(u) = \alpha$、$\beta = 25/(820 \times 2600)$、$u_a = T_a$；$u = C$ 时 $a(u) = D(u)$、$\beta = 8 \times 10^{-7}$、$u_a = b$。取节点 $r_i = i\Delta r$，$i = 0, \ldots, N$、

7



$\Delta r = R/N$，内部控制体面位于相邻节点中点，两端面分别为 $r_{-1/2} = 0$、$r_{N+1/2} = R$。令外向加权通量 $G = -ra(u)u_r$，将式(9)乘 r 并在圆环控制体内积分，得

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{r_{i-1/2}}^{r_{i+1/2}} ru\,\mathrm{d}r = G(r_{i-1/2}, t) - G(r_{i+1/2}, t). \tag{11}$$

真实体积中的公因子 $2\pi L$ 已约去。以节点值近似式(11) 的储存量，权重为 $w_i = (r_{i+1/2}^2 - r_{i-1/2}^2)/2$，因此轴心和表面节点均保留各自控制体容量。把相邻两半格的扩散阻力串联相加，即 $\Delta r/\widehat{a} = (\Delta r/2)/a(u_i) + (\Delta r/2)/a(u_{i+1})$，内部面得到调和平均及通量：

$$\widehat{a}_{i+1/2} = \frac{2a(u_i)a(u_{i+1})}{a(u_i) + a(u_{i+1})}, \qquad G_{i+1/2} = r_{i+1/2}\widehat{a}_{i+1/2}\frac{u_i - u_{i+1}}{\Delta r}. \tag{12}$$

同一式(12)同时用于相邻两格，结合式(10) 即得到实际积分的常微分方程组

$$\begin{aligned} w_i \dot{u}_i &= G_{i-1/2} - G_{i+1/2}, \qquad i = 0, \ldots, N, \\ G_{-1/2} &= 0, \qquad G_{N+1/2} = R\beta(u_N - u_a). \end{aligned} \tag{13}$$

轴心内面通量自然为零，式(13)无须计算奇异的 $1/r$。逐格求和后内部面通量抵消，由 $\sum_i w_i = R^2/2$ 可得

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(\frac{2}{R^2}\sum_{i=0}^{N} w_i u_i\right) = -\frac{2\beta}{R}(u_N - u_a). \tag{14}$$

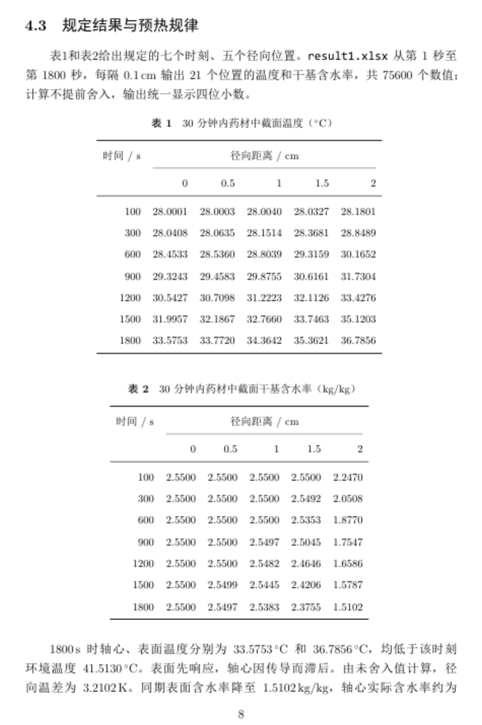
式(14)把圆柱加权平均的变化与表面交换直接对应，也是数值程序累计边界通量、检查离散总量一致性的依据。

将式(13)写为 $\dot{\boldsymbol{u}} = \boldsymbol{F}(t, \boldsymbol{u})$，使用自适应隐式 BDF 方法 [5]。每步的新状态由隐式残量及 Newton 修正确定：

$$\begin{aligned} \boldsymbol{\mathcal{R}}(\boldsymbol{u}^{n+1}) &= \gamma_n \boldsymbol{u}^{n+1} - \boldsymbol{q}_n - \Delta t_n \boldsymbol{F}(t_{n+1}, \boldsymbol{u}^{n+1}) = \boldsymbol{0}, \\ [\gamma_n I - \Delta t_n J_F]\,\delta\boldsymbol{u} &= -\boldsymbol{\mathcal{R}}. \end{aligned} \tag{15}$$

这里 $\gamma_n, \boldsymbol{q}_n$ 由积分阶数、步长和已知历史状态确定；实际采用 SciPy 的变阶 BDF 实现及其精度修正，式(15)表示共同的隐式求解结构。解析稀疏 Jacobian $J_F = \partial\boldsymbol{F}/\partial\boldsymbol{u}$ 保留 $D'(C) = 0.89D(C)/C^2$ 及调和平均的导数，在非线性迭代中更新扩散率。积分在每个环境输入折点重启并传递末态。正式径向区间数为 10240，相对容差 2×10^{-12}，最大步长 3 s；1 s 是输出间隔，不是积分固定步长。

8



4.3 规定结果与预热规律

表1和表2给出规定的七个时刻、五个径向位置。`result1.xlsx` 从第 1 秒至第 1800 秒，每隔 0.1 cm 输出 21 个位置的温度和干基含水率，共 75600 个数值；计算不提前舍入，输出统一显示四位小数。

表 1 30 分钟内药材中截面温度（°C）

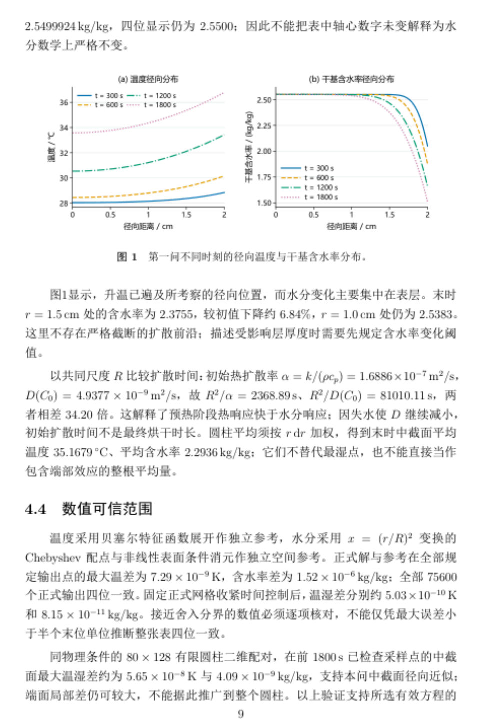
时间 / s	径向距离 / cm				
	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 2 30 分钟内药材中截面干基含水率（kg/kg）

时间 / s	径向距离 / cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

1800 s 时轴心、表面温度分别为 33.5753 °C 和 36.7856 °C，均低于该时刻环境温度 41.5130 °C。表面先响应，轴心因传导而滞后。由未舍入值计算，径向温差为 3.2102 K。同期表面含水率降至 1.5102 kg/kg，轴心实际含水率约为

9



2.5499924 kg/kg，四位显示仍为 2.5500；因此不能把表中轴心数字未变解释为水分数学上严格不变。

图 1 第一问不同时刻的径向温度与干基含水率分布。

图1显示，升温已遍及所考察的径向位置，而水分变化主要集中在表层。末时 $r = 1.5\,\mathrm{cm}$ 处的含水率为 2.3755，较初值下降约 6.84%，$r = 1.0\,\mathrm{cm}$ 处仍为 2.5383。这里不存在严格截断的扩散前沿；描述受影响层厚度时需要先规定含水率变化阈值。

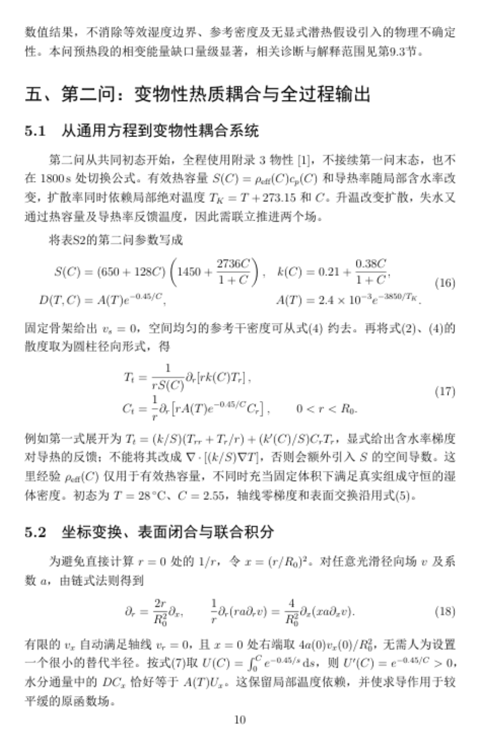
以共同尺度 R 比较扩散时间：初始热扩散率 $\alpha = k/(\rho c_p) = 1.6886 \times 10^{-7}\,\mathrm{m^2/s}$，$D(C_0) = 4.9377 \times 10^{-9}\,\mathrm{m^2/s}$，故 $R^2/\alpha = 2368.89\,\mathrm{s}$、$R^2/D(C_0) = 81010.11\,\mathrm{s}$，两者相差 34.20 倍。这解释了预热阶段热响应快于水分响应；因失水使 D 继续减小，初始扩散时间不是最终烘干时长。圆柱平均须按 $r\,\mathrm{d}r$ 加权，得到末时中截面平均温度 35.1679 °C、平均含水率 2.2936 kg/kg；它们不替代最湿点，也不能直接当作包含端部效应的整根平均量。

4.4 数值可信范围

温度采用贝塞尔特征函数展开作独立参考，水分采用 $x = (r/R)^2$ 变换的 Chebyshev 配点与非线性表面条件消元作独立空间参考。正式解与参考在全部规定输出点的最大温差为 $7.29 \times 10^{-9}\,\mathrm{K}$，含水率差为 $1.52 \times 10^{-6}\,\mathrm{kg/kg}$；全部 75600 个正式输出四位一致。固定正式网格收紧时间控制后，温湿差分别约 $5.03 \times 10^{-10}\,\mathrm{K}$ 和 $8.15 \times 10^{-11}\,\mathrm{kg/kg}$。接近舍入分界的数值必须逐项核对，不能仅凭最大误差小于半个末位单位推断整张表四位一致。

同物理条件的 80×128 有限圆柱二维配对，在前 1800 s 已检查采样点的中截面最大温湿差约为 $5.65 \times 10^{-8}\,\mathrm{K}$ 与 $4.09 \times 10^{-9}\,\mathrm{kg/kg}$，支持本问中截面径向近似；端面局部差仍可较大，不能据此推广到整个圆柱。以上验证支持所选有效方程的

10



数值结果，不消除等效湿度边界、参考密度及无显式潜热假设引入的物理不确定性。本问预热段的相变能量缺口量级显著，相关诊断与解释范围见第9.3节。

五、第二问：变物性热质耦合与全过程输出

5.1 从通用方程到变物性耦合系统

第二问从共同初态开始，全程使用附录 3 物性 [1]，不接续第一问末态，也不在 1800 s 处切换公式。有效热容量 $S(C) = \rho_{\mathrm{eff}}(C)c_p(C)$ 和导热率随局部含水率改变，扩散率同时依赖局部绝对温度 $T_K = T + 273.15$ 和 C。升温改变扩散，失水又通过热容量及导热率反馈温度，因此需联立推进两个场。

将表S2的第二问参数写成

$$\begin{aligned} S(C) &= (650 + 128C)\left(1450 + \frac{2736C}{1+C}\right), & k(C) &= 0.21 + \frac{0.38C}{1+C}, \\ D(T, C) &= A(T)e^{-0.45/C}, & A(T) &= 2.4 \times 10^{-3} e^{-3850/T_K}. \end{aligned} \tag{16}$$

固定骨架给出 $v_s = 0$，空间均匀的参考干密度可从式(4) 约去。再将式(2)、(4)的散度取为圆柱径向形式，得

$$\begin{aligned} T_t &= \frac{1}{rS(C)}\partial_r[rk(C)T_r], \\ C_t &= \frac{1}{r}\partial_r\big[rA(T)e^{-0.45/C}C_r\big], \qquad 0 < r < R_0. \end{aligned} \tag{17}$$

例如第一式展开为 $T_t = (k/S)(T_{rr} + T_r/r) + (k'(C)/S)C_rT_r$，显式给出含水率梯度对导热的反馈；不能将其改成 $\nabla \cdot [(k/S)\nabla T]$，否则会额外引入 S 的空间导数。这里经验 $\rho_{\mathrm{eff}}(C)$ 仅用于有效热容量，不同时充当固定体积下满足真实组成守恒的湿体密度。初态为 $T = 28\,^\circ\mathrm{C}$、$C = 2.55$，轴线零梯度和表面交换沿用式(5)。

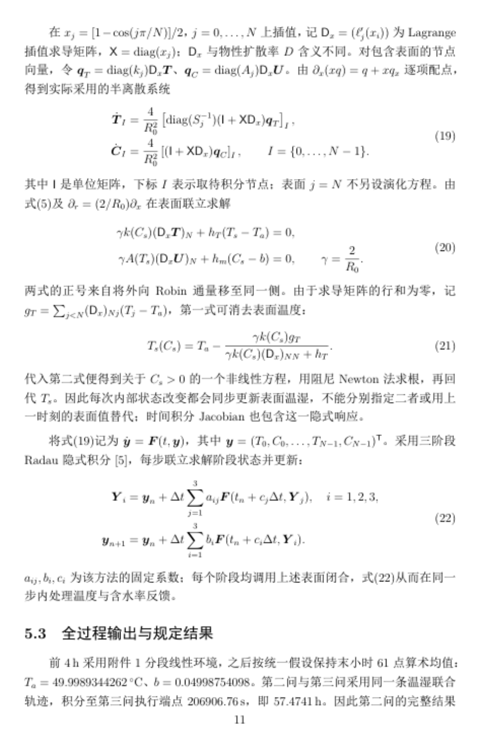
5.2 坐标变换、表面闭合与联合积分

为避免直接计算 $r = 0$ 处的 $1/r$，令 $x = (r/R_0)^2$。对任意光滑径向场 v 及系数 a，由链式法则得到

$$\partial_r = \frac{2r}{R_0^2}\partial_x, \qquad \frac{1}{r}\partial_r(ra\partial_r v) = \frac{4}{R_0^2}\partial_x(xa\partial_x v). \tag{18}$$

有限的 v_x 自动满足轴线 $v_r = 0$，且 $x = 0$ 处右端取 $4a(0)v_x(0)/R_0^2$，无需人为设置一个很小的替代半径。按式(7)取 $U(C) = \int_0^C e^{-0.45/s}\,\mathrm{d}s$，则 $U'(C) = e^{-0.45/C} > 0$，水分通量中的 DC_x 恰好等于 $A(T)U_x$。这保留局部温度依赖，并使求导作用于较平缓的原函数场。

11



在 $x_j = [1 - \cos(j\pi/N)]/2$，$j = 0, \ldots, N$ 上插值，记 $\mathsf{D}_x = (\ell_j'(x_i))$ 为 Lagrange 插值求导矩阵，$\mathsf{X} = \mathrm{diag}(x_j)$；$\mathsf{D}_x$ 与物性扩散率 D 含义不同。对包含表面的节点向量，令 $\boldsymbol{q}_T = \mathrm{diag}(k_j)\mathsf{D}_x\boldsymbol{T}$、$\boldsymbol{q}_C = \mathrm{diag}(A_j)\mathsf{D}_x\boldsymbol{U}$。由 $\partial_x(xq) = q + xq_x$ 逐项配点，得到实际采用的半离散系统

$$\begin{aligned} \dot{\boldsymbol{T}}_I &= \frac{4}{R_0^2}\left[\mathrm{diag}(S_j^{-1})(\mathsf{I} + \mathsf{X}\mathsf{D}_x)\boldsymbol{q}_T\right]_I, \\ \dot{\boldsymbol{C}}_I &= \frac{4}{R_0^2}\left[(\mathsf{I} + \mathsf{X}\mathsf{D}_x)\boldsymbol{q}_C\right]_I, \qquad I = \{0, \ldots, N-1\}. \end{aligned} \tag{19}$$

其中 I 是单位矩阵，下标 I 表示取待积分节点；表面 $j = N$ 不另设演化方程。由式(5)及 $\partial_r = (2/R_0)\partial_x$ 在表面联立求解

$$\begin{aligned} \gamma k(C_s)(\mathsf{D}_x\boldsymbol{T})_N + h_T(T_s - T_a) &= 0, \\ \gamma A(T_s)(\mathsf{D}_x\boldsymbol{U})_N + h_m(C_s - b) &= 0, \qquad \gamma = \frac{2}{R_0}. \end{aligned} \tag{20}$$

两式的正号来自将外向 Robin 通量移至同一侧。由于求导矩阵的行和为零，记 $g_T = \sum_{j<N}(\mathsf{D}_x)_{Nj}(T_j - T_a)$，第一式可消去表面温度：

$$T_s(C_s) = T_a - \frac{\gamma k(C_s)g_T}{\gamma k(C_s)(\mathsf{D}_x)_{NN} + h_T}. \tag{21}$$

代入第二式便得到关于 $C_s > 0$ 的一个非线性方程，用阻尼 Newton 法求根，再回代 T_s。因此每次内部状态改变都会同步更新表面温湿，不能分别指定二者或用上一时刻的表面值替代；时间积分 Jacobian 也包含这一隐式响应。

将式(19)记为 $\dot{\boldsymbol{y}} = \boldsymbol{F}(t, \boldsymbol{y})$，其中 $\boldsymbol{y} = (T_0, C_0, \ldots, T_{N-1}, C_{N-1})^{\mathsf{T}}$。采用三阶段 Radau 隐式积分 [5]，每步联立求解阶段状态并更新：

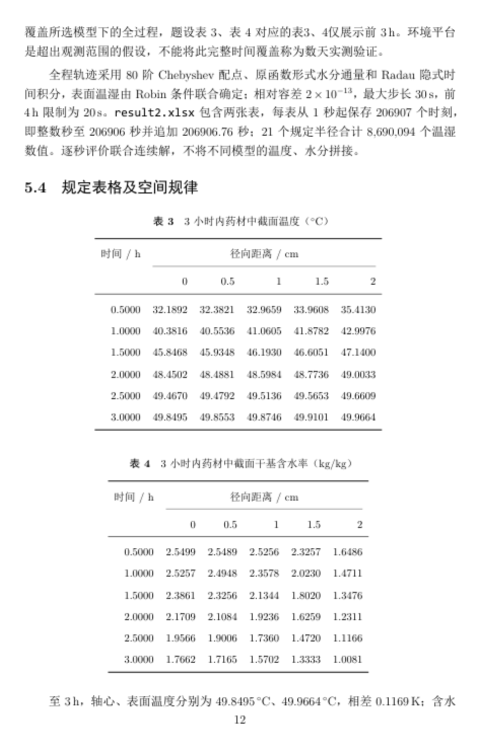
$$\begin{aligned} \boldsymbol{Y}_i &= \boldsymbol{y}_n + \Delta t\sum_{j=1}^{3} a_{ij}\boldsymbol{F}(t_n + c_j\Delta t, \boldsymbol{Y}_j), \qquad i = 1, 2, 3, \\ \boldsymbol{y}_{n+1} &= \boldsymbol{y}_n + \Delta t\sum_{i=1}^{3} b_i\boldsymbol{F}(t_n + c_i\Delta t, \boldsymbol{Y}_i). \end{aligned} \tag{22}$$

a_{ij}, b_i, c_i 为该方法的固定系数；每个阶段均调用上述表面闭合，式(22)从而在同一步内处理温度与含水率反馈。

5.3 全过程输出与规定结果

前 4 h 采用附件 1 分段线性环境，之后按统一假设保持末小时 61 点算术均值：$T_a = 49.9989344262\,^\circ\mathrm{C}$、$b = 0.04998754098$。第二问与第三问采用同一条温湿联合轨迹，积分至第三问执行端点 206906.76 s，即 57.4741 h。因此第二问的完整结果

12



覆盖所选模型下的全过程，题设表 3、表 4 对应的表3、4仅展示前 3 h。环境平台是超出观测范围的假设，不能将此完整时间覆盖称为数天实测验证。

全程轨迹采用 80 阶 Chebyshev 配点、原函数形式水分通量和 Radau 隐式时间积分，表面温湿由 Robin 条件联合确定；相对容差 2×10^{-13}，最大步长 30 s，前 4 h 限制为 20 s。`result2.xlsx` 包含两张表，每表从 1 秒起保存 206907 个时刻，即整数秒至 206906 秒并追加 206906.76 秒；21 个规定半径合计 8,690,094 个温湿数值。逐秒评价联合连续解，不将不同模型的温度、水分拼接。

5.4 规定表格及空间规律

表 3 3 小时内药材中截面温度（°C）

时间 / h	径向距离 / cm				
	0	0.5	1	1.5	2
0.5000	32.1892	32.3821	32.9659	33.9608	35.4130
1.0000	40.3816	40.5536	41.0605	41.8782	42.9976
1.5000	45.8468	45.9348	46.1930	46.6051	47.1400
2.0000	48.4502	48.4881	48.5984	48.7736	49.0033
2.5000	49.4670	49.4792	49.5136	49.5653	49.6609
3.0000	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 3 小时内药材中截面干基含水率（kg/kg）

时间 / h	径向距离 / cm				
	0	0.5	1	1.5	2
0.5000	2.5499	2.5489	2.5256	2.3257	1.6486
1.0000	2.5257	2.4948	2.3578	2.0230	1.4711
1.5000	2.3861	2.3256	2.1344	1.8020	1.3476
2.0000	2.1709	2.1084	1.9236	1.6259	1.2311
2.5000	1.9566	1.9006	1.7360	1.4720	1.1166
3.0000	1.7662	1.7165	1.5702	1.3333	1.0081

至 3 h，轴心、表面温度分别为 49.8495 °C、49.9664 °C，相差 0.1169 K；含水

率分别为 1.7662、1.0081，相差 0.7581 kg/kg。温度接近空间均匀时，水分仍有明显梯度，不能用均匀含水率的集中参数近似替代径向场。按 $r\,\mathrm{d}r$ 加权，平均温度为 49.9043 °C，平均含水率为 1.3825 kg/kg，相对初始含水率下降 45.7833%。该比例是固定参考干密度下的有效水分减少比例，不是实测总质量损失率。

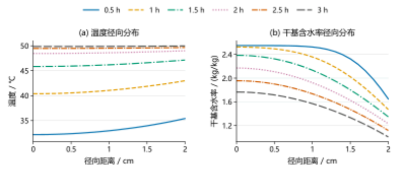


图 2　第二问前 3 小时的中截面径向温湿分布。

图2表明，初期失水集中于外层，随后内部含水率逐步降低。第二问 0.5 h 轴心温度 32.1892 °C，低于第一问同一时刻；这是从初态使用不同物性、尤其初始有效热容量更大的正常结果，不是问间接口错误。第一问和第二问是不同参数情形，不能共用一段预热轨迹。

5.5　升温促进与失水抑制的竞争

将附录 3 扩散系数相对初态取对数，可精确分解为

$$\begin{aligned}\ln\frac{D(t,r)}{D_0}&=H(t,r)+M(t,r),\\ H=3850\left(\frac{1}{301.15}-\frac{1}{T_K}\right),&\qquad M=0.45\left(\frac{1}{2.55}-\frac{1}{C}\right).\end{aligned}\tag{23}$$

其中 $D_0=5.64168037\times10^{-9}\,\mathrm{m^2/s}$。$H$ 度量沿当前轨迹的累计温度贡献，M 度量累计含水率贡献；这是代数分解，不能当作独立改变某一因素的因果贡献百分比。

3 h 时，轴心与表面的 D/D_0 分别为 2.1957 与 1.8207，说明相对初态累计升温促进仍占优势；但最后半小时，两处扩散率分别下降约 1.06% 和 3.16%。因此“扩散系数全程单调增加”不成立。表面还曾在早期每秒采样的第 144 秒达到 $D/D_0=0.983282$ 的局部最低值。温度变化和失水抑制的竞争随阶段变化，前 3 小时的累计主导关系不能直接用于判定末期干燥瓶颈。

在同一 320 区间有限体积网格上单独扰动交换系数，h_m 降低、提高 20% 时，3 h 表面含水率分别为 1.1775、0.8737；相应轴心值为 1.8581、1.6921。对这一输出和扰动区间，水分交换系数的影响明显大于数值误差。该对照为人为情景而非参数置信区间（见第8.2节），不能替代气固水分基准识别 [2, 3]。

5.6　独立验证与适用范围

前 3 小时已有节点有限体积逐级加密至 2560 区间及 Richardson 外推，另用独立空间离散与 Radau 积分核对。全程轨迹与该外推轨迹在前 3 小时共同采样点的最大温差约 1.07×10^{-9} K、含水率差约 8.41×10^{-8} kg/kg；前 3 小时温湿表及规定 60 个论文表值四位一致。有限体积外推是这一前 3 小时交叉验证的来源，不是完整工作簿的生成方法。

长时段另与已完成的 160 阶 BDF 轨迹在全部 60 秒采样点及同一执行端点比较，最大温湿差约 3.27×10^{-9} K 与 9.33×10^{-9} kg/kg，四位输出一致。当前 869 万余格工作簿与同源未舍入轨迹的舍入核对全部通过，该检查支持导出一致性；高阶独立参考覆盖的是所列采样点，不能把导出一致性误写成全部逐秒状态均另作了独立求解。

原有限体积有效水分账本的最大残差约 6.51×10^{-12} kg/kg。热账本核对的是有效热容量积分与边界热流，即比较 $\iint S(C)T_t\,\mathrm{d}V\,\mathrm{d}t$ 与累计边界热量，相对残差约 3.11×10^{-11}，并非 $\int ST\,\mathrm{d}V$ 的末初差，更不构成潜热、携焓及真实总能量已经闭合的证明。同物理 80 × 256 二维配对在前 3 小时已检查采样点的中截面最大温湿差约为 1.692×10^{-3} K、2.247×10^{-5} kg/kg。这支持中截面的工程近似，但不能保证降维后所有四位显示相同；长期全域达标须结合第三问终点证据，不能直接沿用前 3 小时的几何误差。本问早期大通量阶段亦受相变能量缺口约束，相关量级诊断与闭合限制见第9.3节。

六、问题三：由全域含水率约束确定干燥时长

6.1　终止判据与长时计算

问题三要求药材各处的干基含水率严格低于 $C_{\lim}=0.15$ kg/kg[1]。本文沿用问题二的附录 3 物性、初态和环境条件，由同一温湿联立轨迹确定终点，而不以表面含水率或截面平均值代替最大值。因此，本问直接沿用式(17)的控制方程及式(19)、(20)的离散闭合，新增的求解任务是从时间轨迹中定位满足全径向约束的执行点。在一维径向近似中定义

$$M_1(t)=\max_{0\le r\le R_0}C(r,t),\qquad M_1(t_{*,3})=C_{\lim}.\tag{24}$$

所取事件为结束附近的下降穿越。等号根是严格达标时间集合的边界，根本身不满足题设严格不等式；有限圆柱中“各处”的含义则由后述二维计算另行检验。

附件 1 仅覆盖前 4 h，长时计算需延拓环境。本文保持前述末小时 61 点算术均值平台，即环境温度 49.9989344262 °C、有效边界值 $b=0.04998754098$ kg/kg。该平台是假设的后续工况，不是 4 h 以后的观测记录。温度和含水率始终按式(2)–(5)联立推进，不在温度接近环境后直接将温度场替换为常数。

随着含水率降低，扩散系数中的 $\exp(-0.45/C)$ 迅速减小，长时低含水率阶段需要保留局部非线性通量。令

$$\begin{aligned}a_3(T)&=2.4\times10^{-3}\exp\left[-\frac{3850}{T+273.15}\right],\\ F_3(C)&=\int_0^C\exp(-0.45/s)\,\mathrm{d}s,\qquad D\nabla C=a_3(T)\nabla F_3(C).\end{aligned}\tag{25}$$

采用 80 阶 Chebyshev 配点和隐式 Radau 方法推进，表面状态由 Robin 条件联立确定。式(25)只改写通量，不改变题给扩散系数。临界附近同时检查径向重构极值和边界值，确认最大含水率位于轴心。空间加密与不同隐式时间方法作为交叉核验，规定表格和完整结果均由同一未舍入轨迹生成。

6.2　由重构空间极值到时间事件

令 $P_U(x,t)=\sum_{j=0}^{N}F_3(C_j(t))\ell_j(x)$ 为包含表面状态的原函数插值多项式，$x=(r/R_0)^2$。在 $P_U>0$ 的物理解支上，含水率重构为 $C_N(x,t)=F_3^{-1}[P_U(x,t)]$。由于 $F_3'(C)>0$，有 $\partial_xC_N=\partial_xP_U/F_3'(C_N)$，故两者的内部驻点相同。每一时刻的极值候选集为

$$\mathcal{X}(t)=\{0,1\}\cup\{x\in(0,1):\partial_xP_U(x,t)=0\}.\tag{26}$$

求出式(26)中的全部实根，并比较候选点值，得到

$$\widehat{M}_1(t)=\max_{x\in\mathcal{X}(t)}C_N(x,t)=F_3^{-1}\left[\max_{x\in\mathcal{X}(t)}P_U(x,t)\right].\tag{27}$$

该式确定的是重构一维场的最大值；重构误差及与连续方程的差异仍需数值加密检验，不能把多项式极值的求全等同于连续模型严格误差为零。

实际程序先以内部节点事件 $g_{\mathrm{node}}(t)=\max_{j<N}C_j(t)-C_{\lim}$ 检测下降穿越。当相邻接受步满足 $g_{\mathrm{node}}(t_a)>0$、$g_{\mathrm{node}}(t_b)\le0$ 时，利用 Radau 的步内连续插值对 $g_{\mathrm{node}}(t)=0$ 作 Brent 括区求根，再在候选根和执行点按式(27)复核，并用根两侧节点值检查下降穿越。本算例确认根与执行点的重构最大值均位于轴心，节点候选根才可用于所述径向终点。若重构出现更湿的内部点，就必须重新定位事件，不能直接接受节点根。

为解释时间裕量的尺度，在根附近作一阶展开：

$$\widehat{M}_1(t_*+\delta t)-C_{\lim}\approx\dot{\widehat{M}}_1(t_*)\,\delta t,\qquad|\delta t|\approx\frac{|\delta M|}{|\dot{\widehat{M}}_1(t_*)|}.\tag{28}$$

这里 δM 为含水率的局部数值差。已有事件记录给出末期斜率约 -2.9128×10^{-7} (kg/kg)/s；因此显示到四位小数的 0.1500 不能支持亚秒级的达标判定。式(28)仅作局部误差尺度解释，正式时间预算仍取自网格与时间方法对照，不能由该线性近似给出严格界。

6.3　结束时长与径向水分分布

一维模型的临界根为 $t_{*,3}=206906.389932328$ s，即约 57.4739972 h。为区分数值根、执行点和四位小时显示值，取显示时间量子 $\Delta t=0.0001\,\mathrm{h}=0.36\,\mathrm{s}$，并按空间、时间及独立方法差异设置 $\varepsilon_{t,3}=0.03$ s 的经验数值余量，选择

$$t_{\mathrm{exec},3}=\Delta t\left\lceil\frac{t_{*,3}+\varepsilon_{t,3}}{\Delta t}\right\rceil=206906.76\,\mathrm{s}=\boxed{57.4741\,\mathrm{h}}.\tag{29}$$

在该实际时刻重新代入未舍入一维场，得到

$$M_1(t_{\mathrm{exec},3})=0.149999892208\,\mathrm{kg/kg}<0.15\,\mathrm{kg/kg}.\tag{30}$$

执行点比临界根晚约 0.3701 s。这里的经验余量用于当前方程的数值选择，不含环境、边界映射和物性假设的不确定性，也不是实验安全上界。

题设表 5 要求的每 6 h 及结束时刻的径向含水率见表5。18 h 时，表面已降至 0.0845 kg/kg，轴心仍为 0.2988 kg/kg；54 h 时轴心仍为 0.1539 kg/kg。因此，表面先达到要求并不能作为全体材料的终止依据。末期水分向低湿表面迁移，而局部扩散系数随含水率下降，共同形成中心缓慢趋近阈值的尾段。

完整的 `result3.xlsx` 以 60 s 为间隔、0.1 cm 为径向间隔输出，并在最后一个完整分钟 206880 s 之后补入执行点 206906.76 s。末行轴心显示为 0.1500，是式(30)的四位舍入值；达标判断使用未舍入场而不使用表格显示值。160 阶空间加密参考和另一隐式时间方法在全部 72429 个规定水分输出上四位一致，其中空间加密参考与正式轨迹的最大未舍入差约 9.33×10^{-9} kg/kg。该结果支持已选方程的数值稳定性，不替代物理参数验证。

表 5　题设表 5：固定半径时的径向干基含水率（kg/kg）

时间 / h	径向距离 / cm				
	0	0.5	1	1.5	2
6.0000	1.0170	0.9881	0.9015	0.7550	0.5328
12.0000	0.4561	0.4432	0.4031	0.3297	0.1642
18.0000	0.2988	0.2916	0.2687	0.2253	0.0845
24.0000	0.2378	0.2327	0.2167	0.1855	0.0665
30.0000	0.2058	0.2018	0.1892	0.1641	0.0598
36.0000	0.1859	0.1825	0.1718	0.1504	0.0566
42.0000	0.1721	0.1691	0.1597	0.1407	0.0548
48.0000	0.1618	0.1592	0.1507	0.1334	0.0537
54.0000	0.1539	0.1514	0.1436	0.1277	0.0530
57.4741	0.1500	0.1477	0.1402	0.1249	0.0526

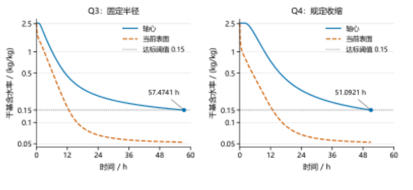


图 3　问题三（左）与问题四（右）的轴心、当前表面含水率演化。纵轴为对数刻度，水平线为 0.15 kg/kg 阈值，终点标记对应各自正式执行时刻。两问采用各自附录物性，问题四另施加观测半径历程，因而曲线差异不能全部归因于收缩。

6.4　有限圆柱检验与工况条件

为检验“各处”的空间含义，在长度 0.25 m 的有限圆柱上增加轴向传递，侧面与端面保持相同物性和交换系数。以 80 个径向区间、256 个轴向区间的同网格一维/二维配对计算比较事件根，二维比配对一维提前约 7.4740 s。这一差值用于识别端部效应，不能把粗网格二维绝对根直接替代精细一维根。

再从临界附近已保存的二维场推进至式(29)的执行时刻，得到全节点最大含水率 0.149998606290 kg/kg，最湿点位于中截面轴心，离散场已经达标。该节点结果与配对根差支持 57.4741 h 的工程保守方向；局部续算仍继承前史离散误差，尚不构成连续二维域的严格数学误差界，也不将该时长解释为精确的二维最短时间。上述证据层级集中见第9.3节。

后段环境对终时的影响大于末位数值差。例如，在相同 4 h 初始全场和其余参数下，将之后温度平台降低 1 K，事件根由约 57.4740 h 延至 59.3413 h；升高 1 K 则变为 55.6884 h。该单因素情景为人为设定而非观测误差范围（见第8.2节）。故 57.4741 h 应与环境平台和有效气固边界共同报告，不能仅凭小数位数给出真实工况下的确定性保证。

七、问题四：观测收缩下的干燥过程

7.1　半径历程与材料运动

问题四将附录 4 物性代入式(2)–(5)，并根据附件 2 把固定几何改为运动材料域 [1]。温度和含水率从 28 °C、2.55 kg/kg 的共同初态重新求解，不接续问题三的干燥末态。附件 2 含 0–72 h 的 145 个半径记录，间隔 1800 s，半径由 2 cm 降至 1.198 cm。本文将半径转换为米后分段线性插值为 $R(t)$，保持其单调性和平台段；所求主终点位于观测时域内。环境继续采用前 4 h 观测插值及之后的共同均值平台。

外半径只规定边界位置，不能唯一确定内部材料运动。采用均匀径向收缩、长度 $L=L_0$ 不变的运动学闭合，定义材料标记 ξ 及映射

$$r=R(t)\xi,\quad z=Z,\qquad v_{s,r}=\frac{R'(t)}{R(t)}r=w_r,\quad v_{s,z}=0,\qquad J=\left(\frac{R(t)}{R_0}\right)^2.\tag{31}$$

其中 $w_r=\partial r/\partial t|_\xi$ 为网格速度，J 为体积 Jacobian。该材料映射是一项附加假设，而不是半径记录直接测得的内部变形。移动边界模型须区分体积水浓度与材料干基含水率，并以当前界面的相对通量写守恒条件 [3]。

7.2　干基守恒与参考域方程

当前体积水浓度为 ρ_dC。对干物质与水分别写守恒式，并采用相对材料的扩散通量，得

$$\begin{aligned}&\partial_t\rho_d+\nabla\cdot(\rho_d\boldsymbol{v}_s)=0,\\ &\partial_t(\rho_dC)+\nabla\cdot(\rho_dC\boldsymbol{v}_s+\boldsymbol{j}_w)=0,\qquad\boldsymbol{j}_w=-\rho_dD\nabla C.\end{aligned}\tag{32}$$

第二式减去第一式的 C 倍，即得到式(4)。由初始干密度均匀和式(31)，$\rho_dJ=\rho_{d,0}$，所以 $\rho_d(t)=\rho_{d,0}[R_0/R(t)]^2$ 为空间常数，可从干基水分方程中约去。对任一场量 $f(r,t)=\widehat{f}(\xi,t)$，链式法则给出

$$\partial_tf|_r=\partial_t\widehat{f}\Big|_\xi-\frac{R'}{R}\xi\partial_\xi\widehat{f},\qquad v_{s,r}\partial_rf=\frac{R'}{R}\xi\partial_\xi\widehat{f}.\tag{33}$$

两项中的材料运动恰好抵消，故 $\mathrm{D}_sf/\mathrm{d}t=\partial_t\widehat{f}|_\xi$。再将 $\partial_r=R^{-1}\partial_\xi$ 代入式(2)和(4)，省略场量帽号后，一维参考域方程为

$$\begin{aligned}\partial_tC|_\xi&=\frac{1}{R(t)^2\xi}\partial_\xi\left[\xi D(T,C)\partial_\xi C\right],\\ \rho_{\mathrm{eff}}(C)c_p(C)\,\partial_tT|_\xi&=\frac{1}{R(t)^2\xi}\partial_\xi\left[\xi k(C)\partial_\xi T\right].\end{aligned}\tag{34}$$

轴线取正则对称条件；将同一空间导数变换代入式(5)，当前侧表面的边界式为

$$-\frac{D(T_s,C_s)}{R}\,\partial_\xi C|_1=h_m(C_s-b),\qquad-\frac{k_s}{R}\,\partial_\xi T|_1=h_T(T_s-T_a).\tag{35}$$

其中 $k_s=k(C_s)$，表面系数均按当前温湿状态计算。

式(34)没有额外的几何浓缩源。无交换的纯收缩使体积水浓度 ρ_dC 增加，但水与干物质质量均不变，因此干基 C 应保持不变。若再加入 $-2R'C/R$ 或另一个网格对流项，就会重复计算已经包含的材料运动。附录 ρ_{eff} 用于有效热容量，ρ_d 用于上述质量账本；二者不能同时通过 $\rho_d=\rho_{\mathrm{eff}}/(1+C)$ 强制等同为真实组分密度。热式仍无显式潜热，数值闭合不构成真实组分总焓守恒的证明（见第9.3节）。

附录 4 的局部扩散系数为

$$D(T,C)=4.2\times10^{-4}\exp(-0.30/C)\exp\left[-\frac{3850}{T+273.15}\right].\tag{36}$$

全部热物性也在各空间位置按当前 C 更新，温湿两场继续联立求解。表面交换系数沿用 $h_T=25\,\mathrm{W/(m^2\cdot K)}$、$h_m=8\times10^{-7}\,\mathrm{m/s}$，式(35)中的质量通量相对于材料界面，不另将表面扫过的体积计为蒸发失水。

7.3　数值求解、账本与结束状态

令 $x=\xi^2$，对任意扩散系数 a 和场量 f 有

$$\frac{1}{R^2\xi}\partial_\xi(a\xi\partial_\xi f)=\frac{4}{R^2}\partial_x(ax\partial_xf)=\frac{4}{R^2}\left[q+x\partial_xq\right],\qquad q=a\partial_xf.\tag{37}$$

该式消去轴线显式奇点，在 $x=0$ 处直接取 $4q(0)/R^2$。依式(7)定义 $F_4'(C)=\exp(-0.30/C)$、$F_4(0)=0$，并令 $A_4(T)=4.2\times10^{-4}\exp[-3850/(T+273.15)]$。沿用问题二的 Chebyshev 微分矩阵 D_x，记 $q_{T,i}=k(C_i)(\mathsf{D}_x\boldsymbol{T})_i$、$q_{C,i}=A_4(T_i)[\mathsf{D}_xF_4(\boldsymbol{C})]_i$ 及 $S_i=\rho_{\mathrm{eff}}(C_i)c_p(C_i)$，则内部节点方程为

$$\begin{aligned}\dot{T}_i&=\frac{4}{R(t)^2S_i}\left[q_{T,i}+x_i(\mathsf{D}_x\boldsymbol{q}_T)_i\right],\\ \dot{C}_i&=\frac{4}{R(t)^2}\left[q_{C,i}+x_i(\mathsf{D}_x\boldsymbol{q}_C)_i\right].\end{aligned}\tag{38}$$

其中 $i=0,\dots,N-1$，表面 $i=N$ 由式(35)确定：

$$\frac{2}{R}q_{T,N}+h_T(T_N-T_a)=0,\qquad\frac{2}{R}q_{C,N}+h_m(C_N-b)=0.\tag{39}$$

这里 $\partial_\xi=2\xi\partial_x$ 使表面因子成为 $2/R$，而内部演化因子为 $4/R^2$；二者均随观测半径更新。取 $N=120$，联立求解表面温湿代数方程后用 Radau 推进内部未知量，在环境及半径输入节点处分段重启。

参考圆环的当前体积与固定干质量分别为 $V_i=\pi R^2L_0(\xi_e^2-\xi_w^2)$、$m_{d,i}=\rho_dV_i$。利用圆环权重与固定干质量构造水量账本 [2]，有

$$\overline{C}(t)=\int_0^1C(x,t)\,dx,\qquad\overline{C}(t)+\int_0^t\frac{2h_m}{R(\tau)}[C_s(\tau)-b(\tau)]\,d\tau=C_0.\tag{40}$$

边界通量积分与空间平均的全程段末最大差为 4.96×10^{-13} kg/kg。72 h 零交换纯收缩检验中，温度与干基含水率保持初态，干物质和水质量的相对变化小于 4.45×10^{-16}。这两项检验针对已选干基守恒与材料运动，不把经验密度的物理解释也视为已验证。

将配点阶数由 120 增加至 160，共同输出点最大含水率差为 8.56×10^{-12} kg/kg；同阶 BDF 时间方法与 Radau 的临界根差约 5.98×10^{-4} s。另用 640 个径向区间的有限体积独立离散，在相同材料位置比较，最大含水率差约 1.58×10^{-5} kg/kg，最后两级空间外推的根与谱法相差约 0.0129 s。前两类检验显示谱解已稳定，有限体积外推提供不同离散方法的支持，外推差仍是经验数值指标而非严格误差界。

终点检测先由内部节点最大值产生等号候选事件，再将式(26)–(27) 中的原函数替换为 F_4，检查重构多项式的全部内部实驻点及两端点。由于 $F_4'(C)>0$，该检查同时给出重构含水率的最大值；本算例在候选根和相邻时刻均确认最大值位于轴心。一维径向极值下降至阈值时，临界根为 $t_{*,4}=183931.211715489$ s。根前后 1 s 的最大值分别约为 0.150000513489 和 0.149999486518，最大位置在轴心。采用 $\varepsilon_{t,4}=0.129$ s 的经验数值预算，按式(29)相同的 0.36 s 量子向上选择执行点，得

$$\begin{aligned}t_{\mathrm{exec},4}&=183931.56\,\mathrm{s}=\boxed{51.0921\,\mathrm{h}},\\ \max_{0\le r\le R(t_{\mathrm{exec},4})}C(r,t_{\mathrm{exec},4})&=0.149999821161\,\mathrm{kg/kg}<0.15\,\mathrm{kg/kg}.\end{aligned}\tag{41}$$

该点晚于一维根约 0.3483 s，当前半径为 1.2000 cm。题设表 6 的规定采样见表6。对规定的实际位置 r_j，以 $P_{U,4}(x,t)$ 表示 $F_4(C)$ 的配点插值多项式，导出值按下式由当前材料场重构：

$$C(r_j,t)=F_4^{-1}\left[P_{U,4}\left(\frac{r_j^2}{R(t)^2},t\right)\right],\qquad0\le r_j\le R(t).\tag{42}$$

当 $r_j>R(t)$ 时不作域外外推；当前表面取 $x=1$ 另列，不能当作固定 2 cm 位置。

表 6　题设表 6：收缩过程中的干基含水率（kg/kg）

时间 / h	径向距离 / cm					当前表面
	0	0.5	1	1.5	2	
6.0000	1.7196	1.5378	1.0226	–	–	0.4207
12.0000	0.7377	0.6546	0.4083	–	–	0.1669
18.0000	0.4088	0.3687	0.2397	–	–	0.0892
24.0000	0.2851	0.2611	0.1790	–	–	0.0673
30.0000	0.2263	0.2094	0.1493	–	–	0.0595
36.0000	0.1928	0.1797	0.1317	–	–	0.0560
42.0000	0.1712	0.1604	0.1200	–	–	0.0541
48.0000	0.1561	0.1468	0.1116	–	–	0.0530
51.0921	0.1500	0.1413	0.1081	–	–	0.0526

表中域外位置以“—”表示不适用，不能解释成零含水率。`result4.xlsx` 按 60 s、0.1 cm 导出，并补入实际执行点；域外单元格留空，另保留表面值与当前半径。末行轴心四位显示 0.1500，严格不等式判断仍基于式(41)的未舍入值。

7.4　有限圆柱全域的数值检验

二维模型在式(34)中加入轴向通量散度，端面使用相同交换系数，长度仍为 L_0。在 80 个径向区间、128 个轴向区间的配对计算中，二维事件根为 183934.287166 s，同径向网格的一维根为 183934.321385 s，相差约 −0.034218 s。这说明端部交换使该配对事件略提前；两个粗网格绝对根共同含有约 3.1 s 的径向偏差，不能把它全算成几何影响。

不同空间与时间设置下，配对提前量约为 0.0286–0.0343 s。精细一维根加该差值得到二维临界根估计 183931.177445–183931.183083 s，但这不是严格上下界。直接将已有 80×128 二维场推进至 183931.56 s 时，其全节点最大含水率为 0.150001406623 kg/kg，仍略高于阈值；该局部续算继承原粗网格前史，不能消除上述共同径向误差。因此，本文保留 51.0921 h 作为一维条件执行值，配对估计提供其二维保守方向的支持；进一步作出充分的有限圆柱全域达标结论，仍需精细二维正式点及误差与正裕量对照。这一证据范围不等于用粗网格结果推翻精细主解，也不等于已经取得连续二维的数学保证。上述证据层级集中见第9.3节。

7.5　收缩作用与模型解释

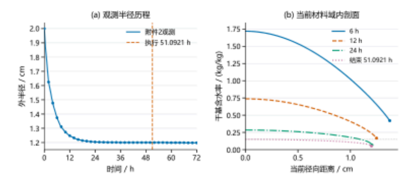


图 4　观测半径历程（左）及附录 4 下不同时间的径向含水率剖面（右）。半径图标出正式执行时刻；剖面采用当前实际半径坐标，各曲线只画至对应时刻的材料表面。

图4给出半径输入与相应的径向含水率变化。为分离施加收缩历程的作用，另保持附录 4 和其余条件不变，将半径固定为 2 cm 作受控对照，临界时间为 129.848360839 h；施加观测收缩后为 51.092003254 h，模型临界时间缩短 78.7564 h，约 60.6526%。这反映当前模型中缩短扩散路径与改变边界交换标度的综合作用。第三、四问还更换了整套物性，故图3两问约 6.382 h 的净差不能全部归因于收缩。

固定 h_m 保持的是材料干基 Robin 系数。在所声明的干质量尺度下，表面质量通量为 $j_s=\rho_d(t)h_m(C_s-b)$，因而每面积、每单位干基差的通量系数为 $K_C=\rho_dh_m$。当 $R/R_0=0.6$ 时，K_C 为初始的 2.7778 倍，式(40)中的 $2h_m/R$ 为初始的 1.6667 倍。所以模型临界时间缩短 60.6526% 是同一有效方程内的条件对照结果，不等于固定真实气膜条件下仅由缩短路径得到的效应，也不能据此推断真实热量收支。

半径重建方式也影响终时。将线性插值改为保形 PCHIP，临界时间为 51.088501642 h，比主线提前约 12.6058 s，明显大于单一插值方案内部的数值预算。输入重建差不能当作半径测量误差的置信区间，适用边界见第9.3节。复现给定的观测 $R(t)$ 不能作为温湿预测的独立验证；改变环境或交换系数而沿用该半径历程时，结果仅为条件响应。

八、模型检验：降维有效性与后段环境灵敏度

8.1　中截面、端部与整根积分

中截面配对需要保持相同物性、环境与边界，才能识别额外轴向交换的影响。问题一 80×128 二维对照在 0–1800 s 中截面采样的最大温度差约为 5.65×10^{-8} K，含水率差约为 4.09×10^{-9} kg/kg。问题二采用最终 80×256 对照，在前 10800 s 的对应差为 1.69165×10^{-3} K 与 2.24706×10^{-5} kg/kg，支持中截面工程近似，但不能宣称所有四位单元均一致。

另一方面，端面交换会改变整根失水。按二维控制体权重与一维圆环权重积分得到表7。表中“失水增加”定义为 $(C_0-\overline{C}_{2D})/(C_0-\overline{C}_{1D})-1$，不是平均含水率的相对误差。在空间均匀干密度的共同假设下，体积权重平均等于干质量权重平均。

表 7　同网格保存场的整根失水对照

情形与时刻	二维网格	$\overline{C}_{2D}$	$\overline{C}_{1D}$	失水增加/%
问题一，1800 s	80×128	2.274929	2.293532	7.2533
问题二，3 h	80×256	1.327482	1.382492	4.7118
问题四，6 h	80×128	1.024545	1.065198	2.7379

图5将同一配对保存场的中截面剖面与整根平均并列，显示局部近似与总体失水应分别判断。

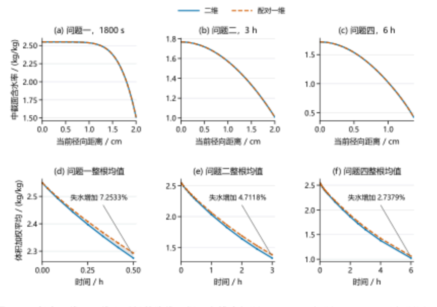


图 5　已保存一维、二维配对场的降维证据：上排为问题一 1800 s、问题二 10800 s 及问题四 21600 s 的中截面径向含水率；下排为整根体积加权平均随时间的变化，终点失水增加与表7一致。问题一、二在已检时段中截面采样最大含水率差分别为 4.09×10^{-9}、2.24706×10^{-5} kg/kg。体积加权平均在空间均匀干密度假设下等于干质量加权平均。来源：`q4_complete_delivery/v1/validation/geometry/` 下的`q1_80x128/q1_80x0`、`q23_80x256_integral/q23_80x0_integral` 及`q4_80x128_integral/q4_80x0_integral` 配对`.npz` 文件；二维半柱坐标的 $z=0$ 为中截面对称面。

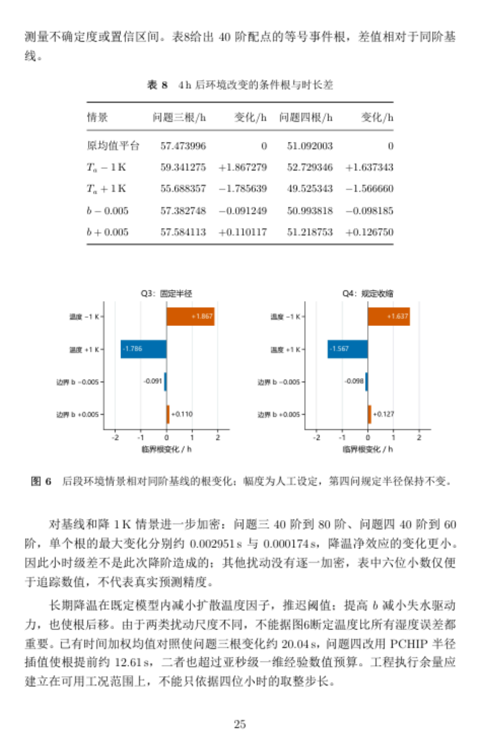
中截面差小与整根总量差可见并不矛盾。若用于全长平均、总失水或端部状态，应采用二维积分或明确忽略端面的近似，不能把中截面表格的精度扩展到全部工程指标。

打开端面后，配对算例显示最湿点更早达到阈值，但这尚不是一般比较定理。温度场与含水率相互耦合；即使 D 随温度增加，两个解之间的变系数通量散度也未必具有统一符号。因此问题三、四的一维执行值、二维节点场、配对校正与连续严格界分别报告。

8.2　后段环境的受控情景

由于式(1)仅为 4 h 后的延拓，保持短时实测过程不变，从保存的 4 h 内部温湿场开展后段联立续算。每问设置原平台、温度 ±1 K 和有效边界 $b\pm0.005$ 五种情景；第四问始终使用同一观测 $R(t)$。扰动幅度为人为选择的压力测试，没有作为

25



测量不确定度或置信区间。表8给出 40 阶配点的等号事件根，差值相对于同阶基线。

表 8　4 h 后环境改变的条件根与时长差

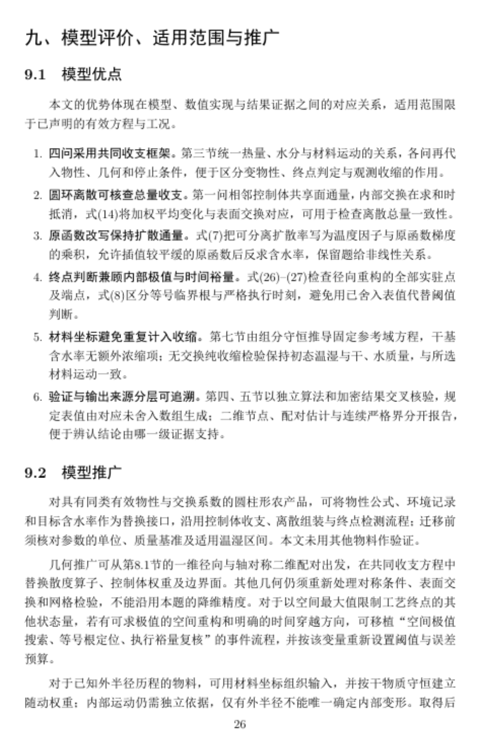
情景	问题三根/h	变化/h	问题四根/h	变化/h
原均值平台	57.473996	0	51.092003	0
$T_a - 1\,\mathrm{K}$	59.341275	+1.867279	52.729346	+1.637343
$T_a + 1\,\mathrm{K}$	55.688357	−1.785639	49.525343	−1.566660
$b - 0.005$	57.382748	−0.091249	50.993818	−0.098185
$b + 0.005$	57.584113	+0.110117	51.218753	+0.126750

图 6　后段环境情景相对同阶基线的根变化；幅度为人工设定，第四问规定半径保持不变。

对基线和降 1 K 情景进一步加密：问题三 40 阶到 80 阶、问题四 40 阶到 60 阶，单个根的最大变化分别约 0.002951 s 与 0.000174 s，降温净效应的变化更小。因此小时级差不是此次降阶造成的；其他扰动没有逐一加密，表中六位小数仅便于追踪数值，不代表真实预测精度。

长期降温在既定模型内减小扩散温度因子，推迟阈值；提高 b 减小失水驱动力，也使根后移。由于两类扰动尺度不同，不能据图6断定温度比所有湿度误差都重要。已有时间加权均值对照使问题三根变化约 20.04 s，问题四改用 PCHIP 半径插值使根提前约 12.61 s，二者也超过亚秒级一维经验数值预算。工程执行余量应建立在可用工况范围上，不能只依据四位小时的取整步长。

26



九、模型评价、适用范围与推广

9.1　模型优点

本文的优势体现在模型、数值实现与结果证据之间的对应关系，适用范围限于已声明的有效方程与工况。

1. **四问采用共同收支框架。**第三节统一热量、水分与材料运动的关系，各问再代入物性、几何和停止条件，便于区分变物性、终点判定与观测收缩的作用。
2. **圆环离散可核查总量收支。**第一问相邻控制体共享面通量，内部交换在求和时抵消，式(14)将加权平均变化与表面交换对应，可用于检查离散总量一致性。
3. **原函数改写保持扩散通量。**式(7)把可分离扩散率写为温度因子与原函数梯度的乘积，允许插值较平缓的原函数后反求含水率，保留题给非线性关系。
4. **终点判断兼顾内部极值与时间裕量。**式(26)–(27)检查径向重构的全部实驻点及端点，式(8)区分等号临界根与严格执行时刻，避免用已舍入表值代替阈值判断。
5. **材料坐标避免重复计入收缩。**第七节由组分守恒推导固定参考域方程，干基含水率无额外浓缩项；无交换纯收缩检验保持初态温湿与干、水质量，与所选材料运动一致。
6. **验证与输出来源分层可追溯。**第四、五节以独立算法和加密结果交叉核验，规定表值由对应未舍入数组生成；二维节点、配对估计与连续严格界分开报告，便于辨认结论由哪一级证据支持。

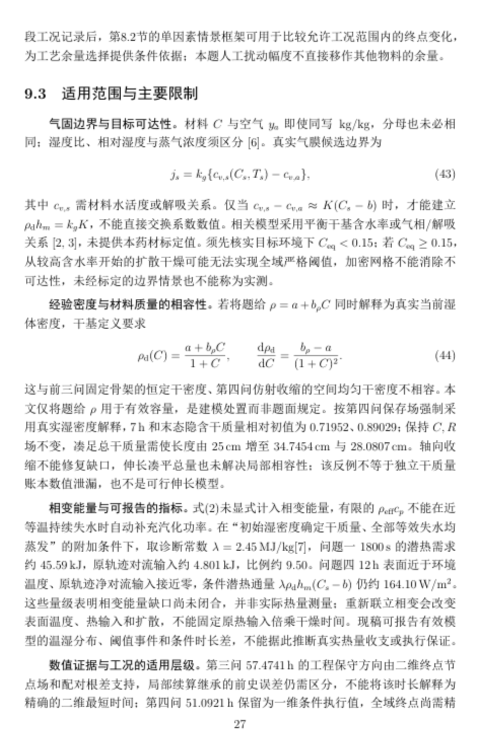
9.2　模型推广

对具有同类有效物性与交换系数的圆柱形农产品，可将物性公式、环境记录和目标含水率作为替换接口，沿用控制体收支、离散组装与终点检测流程；迁移前须核对参数的单位、质量基准及适用温湿区间。本文未用其他物料作验证。

几何推广可从第8.1节的一维径向与轴对称二维配对出发，在共同收支方程中替换散度算子、控制体权重及边界面。其他几何仍须重新处理对称条件、表面交换和网格检验，不能沿用本题的降维精度。对于以空间最大值限制工艺终点的其他状态量，若有可求极值的空间重构和明确的时间穿越方向，可移植“空间极值搜索、等号根定位、执行裕量复核”的事件流程，并按该变量重新设置阈值与误差预算。

对于已知外半径历程的物料，可用材料坐标组织输入，并按干物质守恒建立随动权重；内部运动仍需独立依据，仅有外半径不能唯一确定内部变形。取得后

27



段工况记录后，第8.2节的单因素情景框架可用于比较允许工况范围内的终点变化，为工艺余量选择提供条件依据；本题人工扰动幅度不直接移作其他物料的余量。

9.3　适用范围与主要限制

气固边界与目标可达性。材料 C 与空气 y_a 即使同写 kg/kg，分母也未必相同；湿度比、相对湿度与蒸气浓度须区分 [6]。真实气膜候选边界为

$$j_s = k_g\{c_{v,s}(C_s, T_s) - c_{v,a}\}, \tag{43}$$

其中 $c_{v,s}$ 需材料水活度或解吸关系。仅当 $c_{v,s} - c_{v,a} \approx K(C_s - b)$ 时，才能建立 $\rho_\mathrm{d} h_m = k_g K$，不能直接交换系数数值。相关模型采用平衡干基含水率或气相/解吸关系 [2, 3]，未提供本药材标定值。须先核实目标环境下 $C_\mathrm{eq} < 0.15$；若 $C_\mathrm{eq} \geq 0.15$，从较高含水率开始的扩散干燥可能无法实现全域严格阈值，加密网格不能消除不可达性，未经标定的边界情景也不能称为实测。

经验密度与材料质量的相容性。若将题给 $\rho = a + b_\rho C$ 同时解释为真实当前湿体密度，干基定义要求

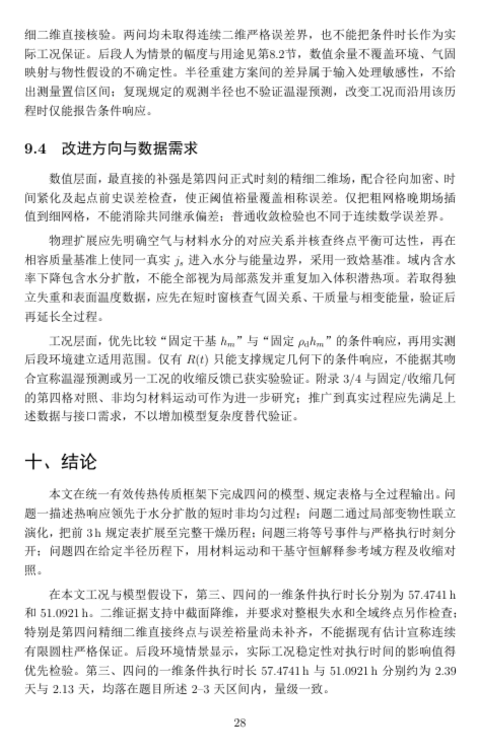
$$\rho_\mathrm{d}(C) = \frac{a + b_\rho C}{1 + C}, \qquad \frac{\mathrm{d}\rho_\mathrm{d}}{\mathrm{d}C} = \frac{b_\rho - a}{(1 + C)^2}. \tag{44}$$

这与前三问固定骨架的恒定干密度、第四问仿射收缩的空间均匀干密度不相容。本文仅将题给 ρ 用于有效容量，是建模处置而非题面规定。按第四问保存场强制采用真实湿密度解释，7 h 和末态隐含干质量相对初值为 0.71952、0.89029；保持 C, R 场不变，凑足总干质量需使长度由 25 cm 增至 34.7454 cm 与 28.0807 cm。轴向收缩不能修复缺口，伸长凑平总量也未解决局部相容性；该反例不等于独立干质量账本数值泄漏，也不是可行伸长模型。

相变能量与可报告的指标。式(2)未显式计入相变能量，有限的 $\rho_\mathrm{eff} c_p$ 不能在近等温持续失水时自动补充汽化功率。在“初始湿密度确定干质量、全部等效失水均蒸发”的附加条件下，取诊断常数 $\lambda = 2.45\,\mathrm{MJ/kg}$[7]，问题一 1800 s 的潜热需求约 45.59 kJ，原轨迹对流输入约 4.801 kJ，比例约 9.50。问题四 12 h 表面近于环境温度、原轨迹净对流输入接近零，条件潜热通量 $\lambda \rho_\mathrm{d} h_m (C_s - b)$ 仍约 164.10 W/m²。这些量级表明相变能量缺口尚未闭合，并非实际热量测量；重新联立相变会改变表面温度、热输入和扩散，不能固定原热输入倍乘干燥时间。现稿可报告有效模型的温湿分布、阈值事件和条件时长差，不能据此推断真实热量收支或执行保证。

数值证据与工况的适用层级。第三问 57.4741 h 的工程保守方向由二维终点节点场和配对根差支持，局部续算继承的前史误差仍需区分，不能将该时长解释为精确的二维最短时间；第四问 51.0921 h 保留为一维条件执行值，全域终点尚需精

28



细二维直接核验。两问均未取得连续二维严格误差界，也不能把条件时长作为实际工况保证。后段人为情景的幅度与用途见第8.2节，数值余量不覆盖环境、气固映射与物性假设的不确定性。半径重建方案间的差异属于输入处理敏感性，不给出测量置信区间；复现规定的观测半径也不验证温湿预测，改变工况而沿用该历程时仅能报告条件响应。

9.4　改进方向与数据需求

数值层面，最直接的补强是第四问正式时刻的精细二维场，配合径向加密、时间紧化及起点前史误差检查，使正阈值裕量覆盖相称误差。仅把粗网格晚期场插值到细网格，不能消除共同继承偏差；普通收敛检验也不同于连续数学误差界。

物理扩展应先明确空气与材料水分的对应关系并核查终点平衡可达性，再在相容质量基准上使同一真实 j_s 进入水分与能量边界，采用一致焓基准。域内含水率下降包含水分扩散，不能全部视为局部蒸发并重复加入体积潜热项。若取得独立失重和表面温度数据，应先在短时窗核查气固关系、干质量与相变能量，验证后再延长全过程。

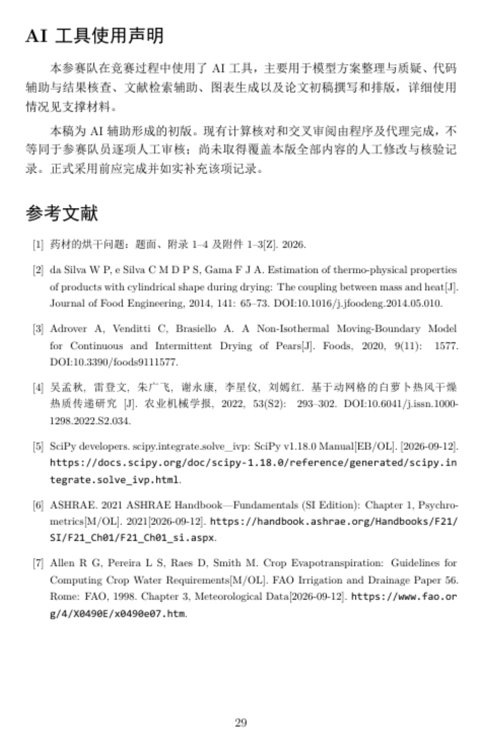
工况层面，优先比较“固定干基 h_m”与“固定 $\rho_\mathrm{d} h_m$”的条件响应，再用实测后段环境建立适用范围。仅有 $R(t)$ 只能支撑规定几何下的条件响应，不能据其吻合宣称温湿预测或另一工况的收缩反馈已获实验验证。附录 3/4 与固定/收缩几何的第四格对照、非均匀材料运动可作为进一步研究；推广到真实过程应先满足上述数据与接口需求，不以增加模型复杂度替代验证。

十、结论

本文在统一有效传热传质框架下完成四问的模型、规定表格与全过程输出。问题一描述热响应领先于水分扩散的短时非均匀过程；问题二通过局部变物性联立演化，把前 3 h 规定表扩展至完整干燥历程；问题三将等号事件与严格执行时刻分开；问题四在给定半径历程下，用材料运动和干基守恒解释参考域方程及收缩对照。

在本文工况与模型假设下，第三、四问的一维条件执行时长分别为 57.4741 h 和 51.0921 h。二维证据支持中截面降维，并要求对整根失水和全域终点另作检查；特别是第四问精细二维直接终点与误差裕量尚未补齐，不能据现有估计宣称连续有限圆柱严格保证。后段环境情景显示，实际工况稳定性对执行时间的影响值得优先检验。第三、四问的一维条件执行时长 57.4741 h 与 51.0921 h 分别约为 2.39 天与 2.13 天，均落在题目所述 2–3 天区间内，量级一致。

29



AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型方案整理与质疑、代码辅助与结果核查、文献检索辅助、图表生成以及论文初稿撰写和排版，详细使用情况见支撑材料。

本稿为 AI 辅助形成的初版。现有计算核对和交叉审阅由程序及代理完成，不等同于参赛队员逐项人工审核；尚未取得覆盖本版全部内容的人工修改与核验记录。正式采用前应完成并如实补充该项记录。

参考文献

[1] 药材的烘干问题：题面、附录 1–4 及附件 1–3[Z]. 2026.

[2] da Silva W P, e Silva C M D P S, Gama F J A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. Journal of Food Engineering, 2014, 141: 65–73. DOI:10.1016/j.jfoodeng.2014.05.010.

[3] Adrover A, Venditti C, Brasiello A. A Non-Isothermal Moving-Boundary Model for Continuous and Intermittent Drying of Pears[J]. Foods, 2020, 9(11): 1577. DOI:10.3390/foods9111577.

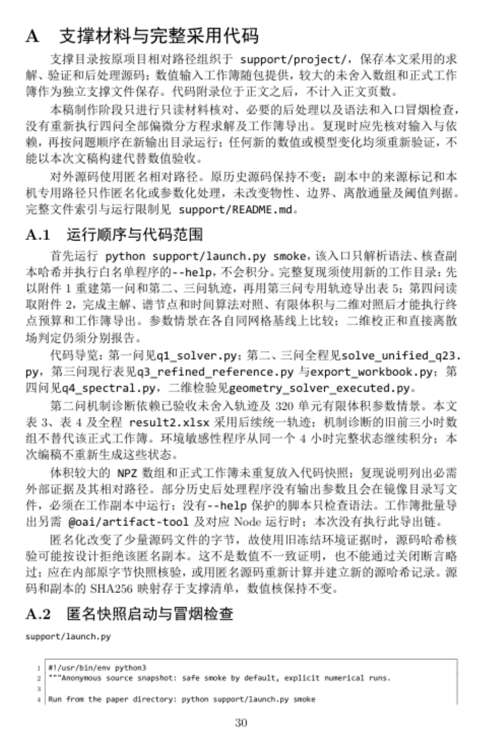
[4] 吴孟秋，雷登文，朱广飞，谢永康，李星仪，刘嫣红．基于动网格的白萝卜热风干燥热质传递研究 [J]．农业机械学报，2022，53(S2)： 293–302．DOI:10.6041/j.issn.1000-1298.2022.S2.034.

[5] SciPy developers. scipy.integrate.solve_ivp: SciPy v1.18.0 Manual[EB/OL]. [2026-09-12]. `https://docs.scipy.org/doc/scipy-1.18.0/reference/generated/scipy.integrate.solve_ivp.html`.

[6] ASHRAE. 2021 ASHRAE Handbook—Fundamentals (SI Edition): Chapter 1, Psychrometrics[M/OL]. 2021[2026-09-12]. `https://handbook.ashrae.org/Handbooks/F21/SI/F21_Ch01/F21_Ch01_si.aspx`.

[7] Allen R G, Pereira L S, Raes D, Smith M. Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements[M/OL]. FAO Irrigation and Drainage Paper 56. Rome: FAO, 1998. Chapter 3, Meteorological Data[2026-09-12]. `https://www.fao.org/4/X0490E/x0490e07.htm`.

30



A　支撑材料与完整采用代码

支撑目录按原项目相对路径组织于 `support/project/`，保存本文采用的求解、验证和后处理源码；数值输入工作簿随包提供，较大的未舍入数组和正式工作簿作为独立支撑文件保存。代码附录位于正文之后，不计入正文页数。

本稿制作阶段只进行只读材料核对、必要的后处理以及语法和入口冒烟检查，没有重新执行四问全部偏微分方程求解及工作簿导出。复现时应先核对输入与依赖，再按问题顺序在新输出目录运行；任何新的数值或模型变化均须重新验证，不能以本次文稿构建代替数值验收。

对外源码使用匿名相对路径。原历史源码保持不变；副本中的来源标记和本机专用路径只作匿名化或参数化处理，未改变物性、边界、离散通量及阈值判据。完整文件索引与运行限制见 `support/README.md`。

A.1　运行顺序与代码范围

首先运行 `python support/launch.py smoke`，该入口只解析语法、核查副本哈希并执行白名单程序的`--help`，不会积分。完整复现须使用新的工作目录：先以附件 1 重建第一问和第二、三问轨迹，再用第三问专用轨迹导出表 5；第四问读取附件 2，完成主解、谱节点和时间算法对照、有限体积与二维对照后才能执行终点预算和工作簿导出。参数情景在各自同网格基线上比较；二维校正和直接离散场判定仍须分别报告。

代码导览：第一问见`q1_solver.py`；第二、三问全程见`solve_unified_q23.py`，第三问现行表见`q3_refined_reference.py` 与`export_workbook.py`；第四问见`q4_spectral.py`，二维检验见`geometry_solver_executed.py`。

第二问机制诊断依赖已验收未舍入轨迹及 320 单元有限体积参数情景。本文表 3、表 4 及全程 `result2.xlsx` 采用后续统一轨迹；机制诊断的旧前三小时数组不替代该正式工作簿。环境敏感性程序从同一个 4 小时完整状态继续积分；本次编稿不重新生成这些状态。

体积较大的 `NPZ` 数组和正式工作簿未重复放入代码快照；复现说明列出必需外部证据及其相对路径。部分历史后处理程序没有输出参数且会在镜像目录写文件，必须在工作副本中运行；没有`--help` 保护的脚本只检查语法。工作簿批量导出另需 `@oai/artifact-tool` 及对应 Node 运行时；本次没有执行此导出链。

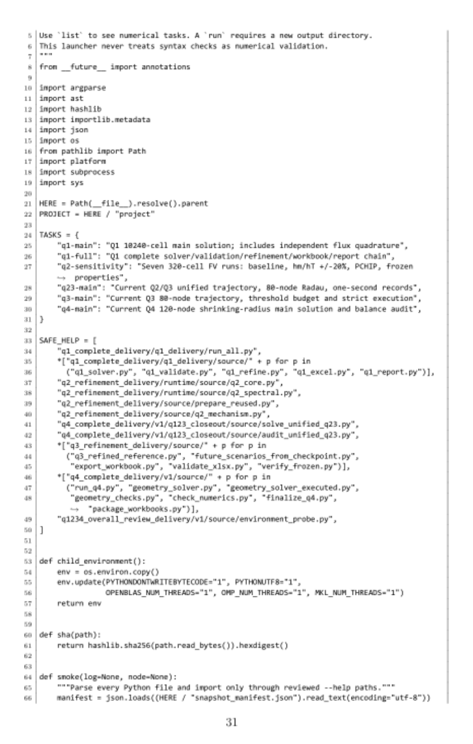
匿名化改变了少量源码文件的字节，故使用旧冻结环境证据时，源码哈希核验可能按设计拒绝该匿名副本。这不是数值不一致证明，也不能通过关闭断言略过；应在内部原字节快照核验，或用匿名源码重新计算并建立新的源哈希记录。源码和副本的 SHA256 映射存于支撑清单，数值核保持不变。

A.2　匿名快照启动与冒烟检查

`support/launch.py`

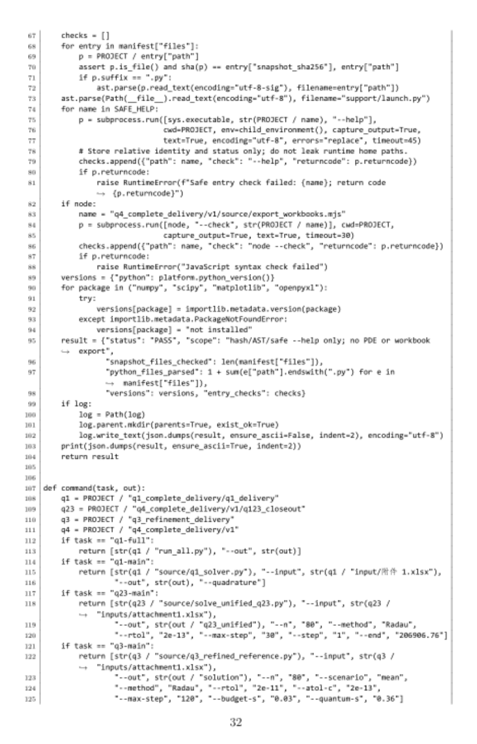
```
#!/usr/bin/env python3
"""Anonymous source snapshot: safe smoke by default, explicit numerical runs.

Run from the paper directory: python support/launch.py smoke
```

31



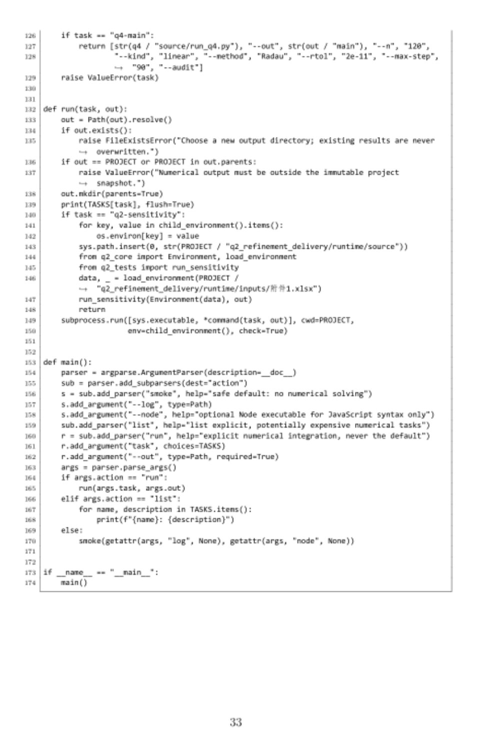
```
Use `list` to see numerical tasks. A `run` requires a new output directory.
This launcher never treats syntax checks as numerical validation.
"""
from __future__ import annotations

import argparse
import ast
import hashlib
import importlib.metadata
import json
import os
from pathlib import Path
import platform
import subprocess
import sys

HERE = Path(__file__).resolve().parent
PROJECT = HERE / "project"

TASKS = {
    "q1-main": "Q1 10240-cell main solution; includes independent flux quadrature",
    "q1-full": "Q1 complete solver/validation/refinement/workbook/report chain",
    "q2-sensitivity": "Seven 320-cell FV runs: baseline, hm/hT +/-20%, PCHIP, frozen
    ↪  properties",
    "q23-main": "Current Q2/Q3 unified trajectory, 80-node Radau, one-second records",
    "q3-main": "Current Q3 80-node trajectory, threshold budget and strict execution",
    "q4-main": "Current Q4 120-node shrinking-radius main solution and balance audit",
}

SAFE_HELP = [
    "q1_complete_delivery/q1_delivery/run_all.py",
    *["q1_complete_delivery/q1_delivery/source/" + p for p in
      ("q1_solver.py", "q1_validate.py", "q1_refine.py", "q1_excel.py", "q1_report.py")],
    "q2_refinement_delivery/runtime/source/q2_core.py",
    "q2_refinement_delivery/runtime/source/q2_spectral.py",
    "q2_refinement_delivery/source/prepare_reused.py",
    "q2_refinement_delivery/source/q2_mechanism.py",
    "q4_complete_delivery/v1/q123_closeout/source/solve_unified_q23.py",
    "q4_complete_delivery/v1/q123_closeout/source/audit_unified_q23.py",
    *["q3_refinement_delivery/source/" + p for p in
      ("q3_refined_reference.py", "future_scenarios_from_checkpoint.py",
       "export_workbook.py", "validate_xlsx.py", "verify_frozen.py")],
    *["q4_complete_delivery/v1/source/" + p for p in
      ("run_q4.py", "geometry_solver.py", "geometry_solver_executed.py",
       "geometry_checks.py", "check_numerics.py", "finalize_q4.py",
       ↪  "package_workbooks.py")],
    "q1234_overall_review_delivery/v1/source/environment_probe.py",
]


def child_environment():
    env = os.environ.copy()
    env.update(PYTHONDONTWRITEBYTECODE="1", PYTHONUTF8="1",
               OPENBLAS_NUM_THREADS="1", OMP_NUM_THREADS="1", MKL_NUM_THREADS="1")
    return env


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def smoke(log=None, node=None):
    """Parse every Python file and import only through reviewed --help paths."""
    manifest = json.loads((HERE / "snapshot_manifest.json").read_text(encoding="utf-8"))
```

32



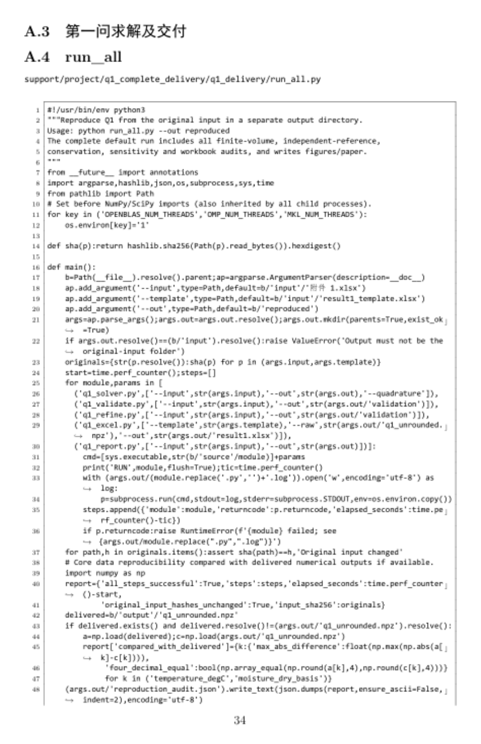
```
    checks = []
    for entry in manifest["files"]:
        p = PROJECT / entry["path"]
        assert p.is_file() and sha(p) == entry["snapshot_sha256"], entry["path"]
        if p.suffix == ".py":
            ast.parse(p.read_text(encoding="utf-8-sig"), filename=entry["path"])
    ast.parse(Path(__file__).read_text(encoding="utf-8"), filename="support/launch.py")
    for name in SAFE_HELP:
        p = subprocess.run([sys.executable, str(PROJECT / name), "--help"],
                            cwd=PROJECT, env=child_environment(), capture_output=True,
                            text=True, encoding="utf-8", errors="replace", timeout=45)
        # Store relative identity and status only; do not leak runtime home paths.
        checks.append({"path": name, "check": "--help", "returncode": p.returncode})
        if p.returncode:
            raise RuntimeError(f"Safe entry check failed: {name}; return code
            ↪  {p.returncode}")
    if node:
        name = "q4_complete_delivery/v1/source/export_workbooks.mjs"
        p = subprocess.run([node, "--check", str(PROJECT / name)], cwd=PROJECT,
                            capture_output=True, text=True, timeout=30)
        checks.append({"path": name, "check": "node --check", "returncode": p.returncode})
        if p.returncode:
            raise RuntimeError("JavaScript syntax check failed")
    versions = {"python": platform.python_version()}
    for package in ("numpy", "scipy", "matplotlib", "openpyxl"):
        try:
            versions[package] = importlib.metadata.version(package)
        except importlib.metadata.PackageNotFoundError:
            versions[package] = "not installed"
    result = {"status": "PASS", "scope": "hash/AST/safe --help only; no PDE or workbook
    ↪  export",
              "snapshot_files_checked": len(manifest["files"]),
              "python_files_parsed": 1 + sum(e["path"].endswith(".py") for e in
              ↪  manifest["files"]),
              "versions": versions, "entry_checks": checks}
    if log:
        log = Path(log)
        log.parent.mkdir(parents=True, exist_ok=True)
        log.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result, ensure_ascii=True, indent=2))
    return result


def command(task, out):
    q1 = PROJECT / "q1_complete_delivery/q1_delivery"
    q23 = PROJECT / "q4_complete_delivery/v1/q123_closeout"
    q3 = PROJECT / "q3_refinement_delivery"
    q4 = PROJECT / "q4_complete_delivery/v1"
    if task == "q1-full":
        return [str(q1 / "run_all.py"), "--out", str(out)]
    if task == "q1-main":
        return [str(q1 / "source/q1_solver.py"), "--input", str(q1 / "input/附件 1.xlsx"),
                "--out", str(out), "--quadrature"]
    if task == "q23-main":
        return [str(q23 / "source/solve_unified_q23.py"), "--input", str(q23 /
        ↪  "inputs/attachment1.xlsx"),
                "--out", str(out / "q23_unified"), "--n", "80", "--method", "Radau",
                "--rtol", "2e-13", "--max-step", "30", "--step", "1", "--end", "206906.76"]
    if task == "q3-main":
        return [str(q3 / "source/q3_refined_reference.py"), "--input", str(q3 /
        ↪  "inputs/attachment1.xlsx"),
                "--out", str(out / "solution"), "--n", "80", "--scenario", "mean",
                "--method", "Radau", "--rtol", "2e-11", "--atol-c", "2e-13",
                "--max-step", "120", "--budget-s", "0.03", "--quantum-s", "0.36"]
```

33



```
    if task == "q4-main":
        return [str(q4 / "source/run_q4.py"), "--out", str(out / "main"), "--n", "120",
                "--kind", "linear", "--method", "Radau", "--rtol", "2e-11", "--max-step",
                ↪  "90", "--audit"]
    raise ValueError(task)


def run(task, out):
    out = Path(out).resolve()
    if out.exists():
        raise FileExistsError("Choose a new output directory; existing results are never
        ↪  overwritten.")
    if out == PROJECT or PROJECT in out.parents:
        raise ValueError("Numerical output must be outside the immutable project
        ↪  snapshot.")
    out.mkdir(parents=True)
    print(TASKS[task], flush=True)
    if task == "q2-sensitivity":
        for key, value in child_environment().items():
            os.environ[key] = value
        sys.path.insert(0, str(PROJECT / "q2_refinement_delivery/runtime/source"))
        from q2_core import Environment, load_environment
        from q2_tests import run_sensitivity
        data, _ = load_environment(PROJECT /
        ↪  "q2_refinement_delivery/runtime/inputs/附件1.xlsx")
        run_sensitivity(Environment(data), out)
        return
    subprocess.run([sys.executable, *command(task, out)], cwd=PROJECT,
                   env=child_environment(), check=True)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    sub = parser.add_subparsers(dest="action")
    s = sub.add_parser("smoke", help="safe default: no numerical solving")
    s.add_argument("--log", type=Path)
    s.add_argument("--node", help="optional Node executable for JavaScript syntax only")
    sub.add_parser("list", help="list explicit, potentially expensive numerical tasks")
    r = sub.add_parser("run", help="explicit numerical integration, never the default")
    r.add_argument("task", choices=TASKS)
    r.add_argument("--out", type=Path, required=True)
    args = parser.parse_args()
    if args.action == "run":
        run(args.task, args.out)
    elif args.action == "list":
        for name, description in TASKS.items():
            print(f"{name}: {description}")
    else:
        smoke(getattr(args, "log", None), getattr(args, "node", None))


if __name__ == "__main__":
    main()
```

34



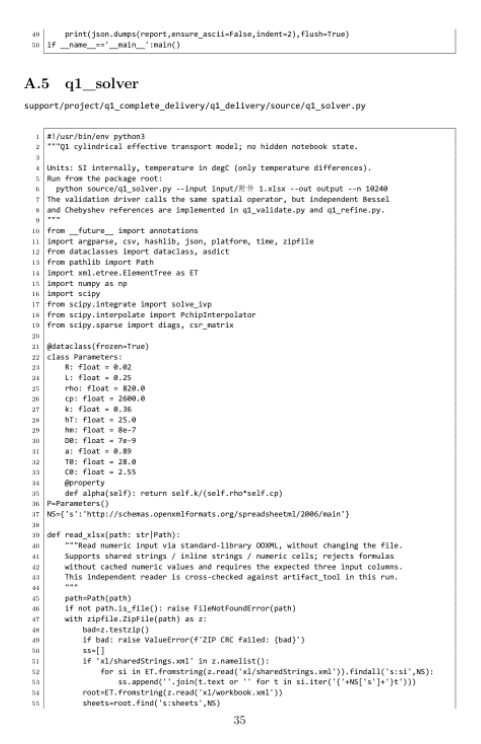
A.3　第一问求解及交付

A.4　run_all

`support/project/q1_complete_delivery/q1_delivery/run_all.py`

```
#!/usr/bin/env python3
"""Reproduce Q1 from the original input in a separate output directory.
Usage: python run_all.py --out reproduced
The complete default run includes all finite-volume, independent-reference,
conservation, sensitivity and workbook audits, and writes figures/paper.
"""
from __future__ import annotations
import argparse,hashlib,json,os,subprocess,sys,time
from pathlib import Path
# Set before NumPy/SciPy imports (also inherited by all child processes).
for key in ('OPENBLAS_NUM_THREADS','OMP_NUM_THREADS','MKL_NUM_THREADS'):
    os.environ[key]='1'

def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()

def main():
    b=Path(__file__).resolve().parent;ap=argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--input',type=Path,default=b/'input'/'附件 1.xlsx')
    ap.add_argument('--template',type=Path,default=b/'input'/'result1_template.xlsx')
    ap.add_argument('--out',type=Path,default=b/'reproduced')
    args=ap.parse_args();args.out=args.out.resolve();args.out.mkdir(parents=True,exist_ok
    ↪  =True)
    if args.out.resolve()==(b/'input').resolve():raise ValueError('Output must not be the
    ↪  original-input folder')
    originals={str(p.resolve()):sha(p) for p in (args.input,args.template)}
    start=time.perf_counter();steps=[]
    for module,params in [
      ('q1_solver.py',['--input',str(args.input),'--out',str(args.out),'--quadrature']),
      ('q1_validate.py',['--input',str(args.input),'--out',str(args.out/'validation')]),
      ('q1_refine.py',['--input',str(args.input),'--out',str(args.out/'validation')]),
      ('q1_excel.py',['--template',str(args.template),'--raw',str(args.out/'q1_unrounded.
      ↪  npz'),'--out',str(args.out/'result1.xlsx')]),
      ('q1_report.py',['--input',str(args.input),'--out',str(args.out)])]:
        cmd=[sys.executable,str(b/'source'/module)]+params
        print('RUN',module,flush=True);tic=time.perf_counter()
        with (args.out/(module.replace('.py','')+'.log')).open('w',encoding='utf-8') as
        ↪  log:
            p=subprocess.run(cmd,stdout=log,stderr=subprocess.STDOUT,env=os.environ.copy())
        steps.append({'module':module,'returncode':p.returncode,'elapsed_seconds':time.pe
        ↪  rf_counter()-tic})
        if p.returncode:raise RuntimeError(f'{module} failed; see
        ↪  {args.out/module.replace(".py",".log")}')
    for path,h in originals.items():assert sha(path)==h,'Original input changed'
    # Core data reproducibility compared with delivered numerical outputs if available.
    import numpy as np
    report={'all_steps_successful':True,'steps':steps,'elapsed_seconds':time.perf_counter
    ↪  ()-start,
            'original_input_hashes_unchanged':True,'input_sha256':originals}
    delivered=b/'output'/'q1_unrounded.npz'
    if delivered.exists() and delivered.resolve()!=(args.out/'q1_unrounded.npz').resolve():
        a=np.load(delivered);c=np.load(args.out/'q1_unrounded.npz')
        report['compared_with_delivered']={k:{'max_abs_difference':float(np.max(np.abs(a[
        ↪  k]-c[k]))),
             'four_decimal_equal':bool(np.array_equal(np.round(a[k],4),np.round(c[k],4)))}
             for k in ('temperature_degC','moisture_dry_basis')}
    (args.out/'reproduction_audit.json').write_text(json.dumps(report,ensure_ascii=False,
    ↪  indent=2),encoding='utf-8')
```

35



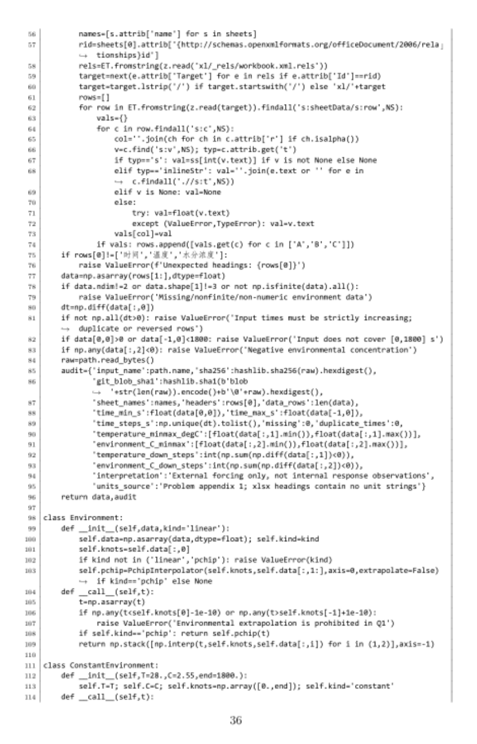
```
    print(json.dumps(report,ensure_ascii=False,indent=2),flush=True)
if __name__=='__main__':main()
```

A.5　q1_solver

`support/project/q1_complete_delivery/q1_delivery/source/q1_solver.py`

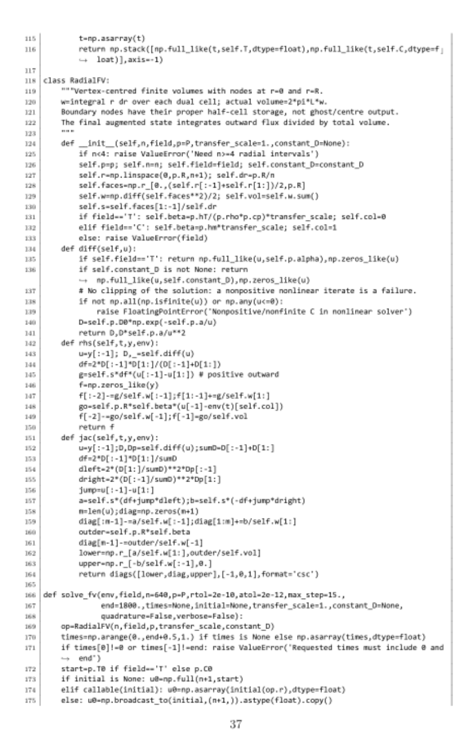
```
#!/usr/bin/env python3
"""Q1 cylindrical effective transport model; no hidden notebook state.

Units: SI internally, temperature in degC (only temperature differences).
Run from the package root:
  python source/q1_solver.py --input input/附件 1.xlsx --out output --n 10240
The validation driver calls the same spatial operator, but independent Bessel
and Chebyshev references are implemented in q1_validate.py and q1_refine.py.
"""
from __future__ import annotations
import argparse, csv, hashlib, json, platform, time, zipfile
from dataclasses import dataclass, asdict
from pathlib import Path
import xml.etree.ElementTree as ET
import numpy as np
import scipy
from scipy.integrate import solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.sparse import diags, csr_matrix

@dataclass(frozen=True)
class Parameters:
    R: float = 0.02
    L: float = 0.25
    rho: float = 820.0
    cp: float = 2600.0
    k: float = 0.36
    hT: float = 25.0
    hm: float = 8e-7
    D0: float = 7e-9
    a: float = 0.89
    T0: float = 28.0
    C0: float = 2.55
    @property
    def alpha(self): return self.k/(self.rho*self.cp)
P=Parameters()
NS={'s':'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}

def read_xlsx(path: str|Path):
    """Read numeric input via standard-library OOXML, without changing the file.
    Supports shared strings / inline strings / numeric cells; rejects formulas
    without cached numeric values and requires the expected three input columns.
    This independent reader is cross-checked against artifact_tool in this run.
    """
    path=Path(path)
    if not path.is_file(): raise FileNotFoundError(path)
    with zipfile.ZipFile(path) as z:
        bad=z.testzip()
        if bad: raise ValueError(f'ZIP CRC failed: {bad}')
        ss=[]
        if 'xl/sharedStrings.xml' in z.namelist():
            for si in ET.fromstring(z.read('xl/sharedStrings.xml')).findall('s:si',NS):
                ss.append(''.join(t.text or '' for t in si.iter('{'+NS['s']+'}t')))
        root=ET.fromstring(z.read('xl/workbook.xml'))
        sheets=root.find('s:sheets',NS)
```

36



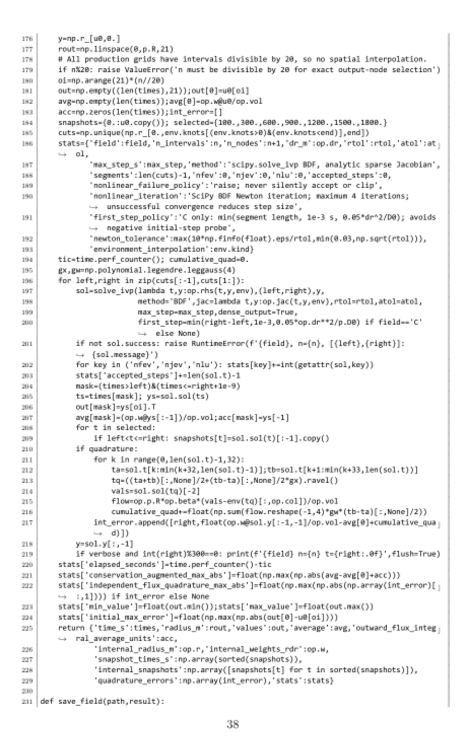
```
        names=[s.attrib['name'] for s in sheets]
        rid=sheets[0].attrib['{http://schemas.openxmlformats.org/officeDocument/2006/rela
        ↪  tionships}id']
        rels=ET.fromstring(z.read('xl/_rels/workbook.xml.rels'))
        target=next(e.attrib['Target'] for e in rels if e.attrib['Id']==rid)
        target=target.lstrip('/') if target.startswith('/') else 'xl/'+target
        rows=[]
        for row in ET.fromstring(z.read(target)).findall('s:sheetData/s:row',NS):
            vals={}
            for c in row.findall('s:c',NS):
                col=''.join(ch for ch in c.attrib['r'] if ch.isalpha())
                v=c.find('s:v',NS); typ=c.attrib.get('t')
                if typ=='s': val=ss[int(v.text)] if v is not None else None
                elif typ=='inlineStr': val=''.join(e.text or '' for e in
                ↪  c.findall('.//s:t',NS))
                elif v is None: val=None
                else:
                    try: val=float(v.text)
                    except (ValueError,TypeError): val=v.text
                vals[col]=val
            if vals: rows.append([vals.get(c) for c in ['A','B','C']])
    if rows[0]!=['时间','温度','水分浓度']:
        raise ValueError(f'Unexpected headings: {rows[0]}')
    data=np.asarray(rows[1:],dtype=float)
    if data.ndim!=2 or data.shape[1]!=3 or not np.isfinite(data).all():
        raise ValueError('Missing/nonfinite/non-numeric environment data')
    dt=np.diff(data[:,0])
    if not np.all(dt>0): raise ValueError('Input times must be strictly increasing;
    ↪  duplicate or reversed rows')
    if data[0,0]>0 or data[-1,0]<1800: raise ValueError('Input does not cover [0,1800] s')
    if np.any(data[:,2]<0): raise ValueError('Negative environmental concentration')
    raw=path.read_bytes()
    audit={'input_name':path.name,'sha256':hashlib.sha256(raw).hexdigest(),
           'git_blob_sha1':hashlib.sha1(b'blob
           ↪  '+str(len(raw)).encode()+b'\0'+raw).hexdigest(),
           'sheet_names':names,'headers':rows[0],'data_rows':len(data),
           'time_min_s':float(data[0,0]),'time_max_s':float(data[-1,0]),
           'time_steps_s':np.unique(dt).tolist(),'missing':0,'duplicate_times':0,
           'temperature_minmax_degC':[float(data[:,1].min()),float(data[:,1].max())],
           'environment_C_minmax':[float(data[:,2].min()),float(data[:,2].max())],
           'temperature_down_steps':int(np.sum(np.diff(data[:,1])<0)),
           'environment_C_down_steps':int(np.sum(np.diff(data[:,2])<0)),
           'interpretation':'External forcing only, not internal response observations',
           'units_source':'Problem appendix 1; xlsx headings contain no unit strings'}
    return data,audit

class Environment:
    def __init__(self,data,kind='linear'):
        self.data=np.asarray(data,dtype=float); self.kind=kind
        self.knots=self.data[:,0]
        if kind not in ('linear','pchip'): raise ValueError(kind)
        self.pchip=PchipInterpolator(self.knots,self.data[:,1:],axis=0,extrapolate=False)
        ↪  if kind=='pchip' else None
    def __call__(self,t):
        t=np.asarray(t)
        if np.any(t<self.knots[0]-1e-10) or np.any(t>self.knots[-1]+1e-10):
            raise ValueError('Environmental extrapolation is prohibited in Q1')
        if self.kind=='pchip': return self.pchip(t)
        return np.stack([np.interp(t,self.knots,self.data[:,i]) for i in (1,2)],axis=-1)

class ConstantEnvironment:
    def __init__(self,T=28.,C=2.55,end=1800.):
        self.T=T; self.C=C; self.knots=np.array([0.,end]); self.kind='constant'
    def __call__(self,t):
```

37



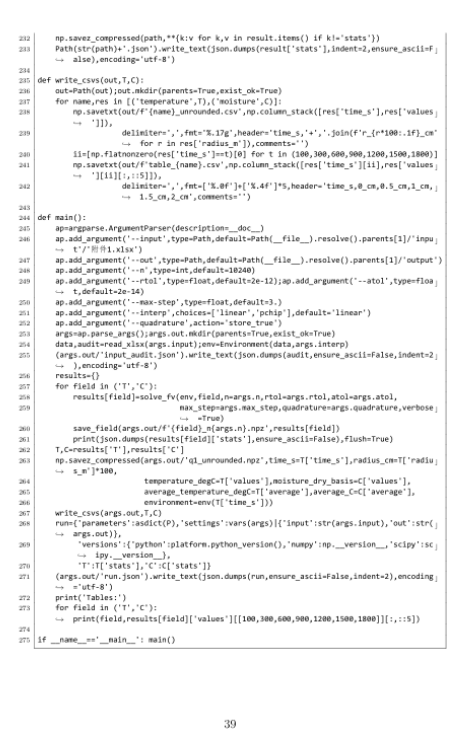
```
        t=np.asarray(t)
        return np.stack([np.full_like(t,self.T,dtype=float),np.full_like(t,self.C,dtype=f
        ↪  loat)],axis=-1)

class RadialFV:
    """Vertex-centred finite volumes with nodes at r=0 and r=R.
    w=integral r dr over each dual cell; actual volume=2*pi*L*w.
    Boundary nodes have their proper half-cell storage, not ghost/centre output.
    The final augmented state integrates outward flux divided by total volume.
    """
    def __init__(self,n,field,p=P,transfer_scale=1.,constant_D=None):
        if n<4: raise ValueError('Need n>=4 radial intervals')
        self.p=p; self.n=n; self.field=field; self.constant_D=constant_D
        self.r=np.linspace(0,p.R,n+1); self.dr=p.R/n
        self.faces=np.r_[0.,(self.r[:-1]+self.r[1:])/2,p.R]
        self.w=np.diff(self.faces**2)/2; self.vol=self.w.sum()
        self.s=self.faces[1:-1]/self.dr
        if field=='T': self.beta=p.hT/(p.rho*p.cp)*transfer_scale; self.col=0
        elif field=='C': self.beta=p.hm*transfer_scale; self.col=1
        else: raise ValueError(field)
    def diff(self,u):
        if self.field=='T': return np.full_like(u,self.p.alpha),np.zeros_like(u)
        if self.constant_D is not None: return
        ↪  np.full_like(u,self.constant_D),np.zeros_like(u)
        # No clipping of the solution: a nonpositive nonlinear iterate is a failure.
        if not np.all(np.isfinite(u)) or np.any(u<=0):
            raise FloatingPointError('Nonpositive/nonfinite C in nonlinear solver')
        D=self.p.D0*np.exp(-self.p.a/u)
        return D,D*self.p.a/u**2
    def rhs(self,t,y,env):
        u=y[:-1]; D,_=self.diff(u)
        df=2*D[:-1]*D[1:]/(D[:-1]+D[1:])
        g=self.s*df*(u[:-1]-u[1:]) # positive outward
        f=np.zeros_like(y)
        f[:-2]-=g/self.w[:-1];f[1:-1]+=g/self.w[1:]
        go=self.p.R*self.beta*(u[-1]-env(t)[self.col])
        f[-2]-=go/self.w[-1];f[-1]=go/self.vol
        return f
    def jac(self,t,y,env):
        u=y[:-1];D,Dp=self.diff(u);sumD=D[:-1]+D[1:]
        df=2*D[:-1]*D[1:]/sumD
        dleft=2*(D[1:]/sumD)**2*Dp[:-1]
        dright=2*(D[:-1]/sumD)**2*Dp[1:]
        jump=u[:-1]-u[1:]
        a=self.s*(df+jump*dleft);b=self.s*(-df+jump*dright)
        m=len(u);diag=np.zeros(m+1)
        diag[:m-1]-=a/self.w[:-1];diag[1:m]+=b/self.w[1:]
        outder=self.p.R*self.beta
        diag[m-1]-=outder/self.w[-1]
        lower=np.r_[a/self.w[1:],outder/self.vol]
        upper=np.r_[-b/self.w[:-1],0.]
        return diags([lower,diag,upper],[-1,0,1],format='csc')

def solve_fv(env,field,n=640,p=P,rtol=2e-10,atol=2e-12,max_step=15.,
             end=1800.,times=None,initial=None,transfer_scale=1.,constant_D=None,
             quadrature=False,verbose=False):
    op=RadialFV(n,field,p,transfer_scale,constant_D)
    times=np.arange(0.,end+0.5,1.) if times is None else np.asarray(times,dtype=float)
    if times[0]!=0 or times[-1]!=end: raise ValueError('Requested times must include 0 and
    ↪  end')
    start=p.T0 if field=='T' else p.C0
    if initial is None: u0=np.full(n+1,start)
    elif callable(initial): u0=np.asarray(initial(op.r),dtype=float)
    else: u0=np.broadcast_to(initial,(n+1,)).astype(float).copy()
```

38



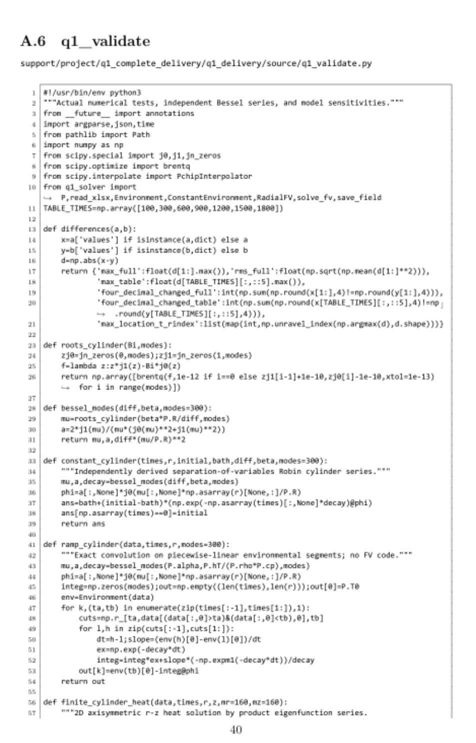
```
    y=np.r_[u0,0.]
    rout=np.linspace(0,p.R,21)
    # All production grids have intervals divisible by 20, so no spatial interpolation.
    if n%20: raise ValueError('n must be divisible by 20 for exact output-node selection')
    oi=np.arange(21)*(n//20)
    out=np.empty((len(times),21));out[0]=u0[oi]
    avg=np.empty(len(times));avg[0]=op.w@u0/op.vol
    acc=np.zeros(len(times));int_error=[]
    snapshots={0.:u0.copy()}; selected={100.,300.,600.,900.,1200.,1500.,1800.}
    cuts=np.unique(np.r_[0.,env.knots[(env.knots>0)&(env.knots<end)],end])
    stats={'field':field,'n_intervals':n,'n_nodes':n+1,'dr_m':op.dr,'rtol':rtol,'atol':at
    ↪  ol,
           'max_step_s':max_step,'method':'scipy.solve_ivp BDF, analytic sparse Jacobian',
           'segments':len(cuts)-1,'nfev':0,'njev':0,'nlu':0,'accepted_steps':0,
           'nonlinear_failure_policy':'raise; never silently accept or clip',
           'nonlinear_iteration':'SciPy BDF Newton iteration; maximum 4 iterations;
           ↪  unsuccessful convergence reduces step size',
           'first_step_policy':'C only: min(segment length, 1e-3 s, 0.05*dr^2/D0); avoids
           ↪  negative initial-step probe',
           'newton_tolerance':max(10*np.finfo(float).eps/rtol,min(0.03,np.sqrt(rtol))),
           'environment_interpolation':env.kind}
    tic=time.perf_counter(); cumulative_quad=0.
    gx,gw=np.polynomial.legendre.leggauss(4)
    for left,right in zip(cuts[:-1],cuts[1:]):
        sol=solve_ivp(lambda t,y:op.rhs(t,y,env),(left,right),y,
                      method='BDF',jac=lambda t,y:op.jac(t,y,env),rtol=rtol,atol=atol,
                      max_step=max_step,dense_output=True,
                      first_step=min(right-left,1e-3,0.05*op.dr**2/p.D0) if field=='C'
                      ↪  else None)
        if not sol.success: raise RuntimeError(f'{field}, n={n}, [{left},{right}]:
        ↪  {sol.message}')
        for key in ('nfev','njev','nlu'): stats[key]+=int(getattr(sol,key))
        stats['accepted_steps']+=len(sol.t)-1
        mask=(times>left)&(times<=right+1e-9)
        ts=times[mask]; ys=sol.sol(ts)
        out[mask]=ys[oi].T
        avg[mask]=(op.w@ys[:-1])/op.vol;acc[mask]=ys[-1]
        for t in selected:
            if left<t<=right: snapshots[t]=sol.sol(t)[:-1].copy()
        if quadrature:
            for k in range(0,len(sol.t)-1,32):
                ta=sol.t[k:min(k+32,len(sol.t)-1)];tb=sol.t[k+1:min(k+33,len(sol.t))]
                tq=((ta+tb)[:,None]/2+(tb-ta)[:,None]/2*gx).ravel()
                vals=sol.sol(tq)[-2]
                flow=op.p.R*op.beta*(vals-env(tq)[:,op.col])/op.vol
                cumulative_quad+=float(np.sum(flow.reshape(-1,4)*gw*(tb-ta)[:,None]/2))
            int_error.append([right,float(op.w@sol.y[:-1,-1]/op.vol-avg[0]+cumulative_qua
            ↪  d)])
        y=sol.y[:,-1]
        if verbose and int(right)%300==0: print(f'{field} n={n} t={right:.0f}',flush=True)
    stats['elapsed_seconds']=time.perf_counter()-tic
    stats['conservation_augmented_max_abs']=float(np.max(np.abs(avg-avg[0]+acc)))
    stats['independent_flux_quadrature_max_abs']=float(np.max(np.abs(np.array(int_error)[
    ↪  :,1]))) if int_error else None
    stats['min_value']=float(out.min());stats['max_value']=float(out.max())
    stats['initial_max_error']=float(np.max(np.abs(out[0]-u0[oi])))
    return {'time_s':times,'radius_m':rout,'values':out,'average':avg,'outward_flux_integ
    ↪  ral_average_units':acc,
            'internal_radius_m':op.r,'internal_weights_rdr':op.w,
            'snapshot_times_s':np.array(sorted(snapshots)),
            'internal_snapshots':np.array([snapshots[t] for t in sorted(snapshots)]),
            'quadrature_errors':np.array(int_error),'stats':stats}

def save_field(path,result):
```

39



```
    np.savez_compressed(path,**{k:v for k,v in result.items() if k!='stats'})
    Path(str(path)+'.json').write_text(json.dumps(result['stats'],indent=2,ensure_ascii=F
    ↪  alse),encoding='utf-8')

def write_csvs(out,T,C):
    out=Path(out);out.mkdir(parents=True,exist_ok=True)
    for name,res in [('temperature',T),('moisture',C)]:
        np.savetxt(out/f'{name}_unrounded.csv',np.column_stack([res['time_s'],res['values
        ↪  ']]),
                   delimiter=',',fmt='%.17g',header='time_s,'+','.join(f'r_{r*100:.1f}_cm'
                   ↪  for r in res['radius_m']),comments='')
        ii=[np.flatnonzero(res['time_s']==t)[0] for t in (100,300,600,900,1200,1500,1800)]
        np.savetxt(out/f'table_{name}.csv',np.column_stack([res['time_s'][ii],res['values
        ↪  '][ii][:,::5]]),
                   delimiter=',',fmt=['%.0f']+['%.4f']*5,header='time_s,0_cm,0.5_cm,1_cm,
                   ↪  1.5_cm,2_cm',comments='')

def main():
    ap=argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--input',type=Path,default=Path(__file__).resolve().parents[1]/'inpu
    ↪  t'/'附件1.xlsx')
    ap.add_argument('--out',type=Path,default=Path(__file__).resolve().parents[1]/'output')
    ap.add_argument('--n',type=int,default=10240)
    ap.add_argument('--rtol',type=float,default=2e-12);ap.add_argument('--atol',type=floa
    ↪  t,default=2e-14)
    ap.add_argument('--max-step',type=float,default=3.)
    ap.add_argument('--interp',choices=['linear','pchip'],default='linear')
    ap.add_argument('--quadrature',action='store_true')
    args=ap.parse_args();args.out.mkdir(parents=True,exist_ok=True)
    data,audit=read_xlsx(args.input);env=Environment(data,args.interp)
    (args.out/'input_audit.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2
    ↪  ),encoding='utf-8')
    results={}
    for field in ('T','C'):
        results[field]=solve_fv(env,field,n=args.n,rtol=args.rtol,atol=args.atol,
                                max_step=args.max_step,quadrature=args.quadrature,verbose
                                ↪  =True)
        save_field(args.out/f'{field}_n{args.n}.npz',results[field])
        print(json.dumps(results[field]['stats'],ensure_ascii=False),flush=True)
    T,C=results['T'],results['C']
    np.savez_compressed(args.out/'q1_unrounded.npz',time_s=T['time_s'],radius_cm=T['radiu
    ↪  s_m']*100,
                        temperature_degC=T['values'],moisture_dry_basis=C['values'],
                        average_temperature_degC=T['average'],average_C=C['average'],
                        environment=env(T['time_s']))
    write_csvs(args.out,T,C)
    run={'parameters':asdict(P),'settings':vars(args)|{'input':str(args.input),'out':str(
    ↪  args.out)},
         'versions':{'python':platform.python_version(),'numpy':np.__version__,'scipy':sc
         ↪  ipy.__version__},
         'T':T['stats'],'C':C['stats']}
    (args.out/'run.json').write_text(json.dumps(run,ensure_ascii=False,indent=2),encoding
    ↪  ='utf-8')
    print('Tables:')
    for field in ('T','C'):
    ↪  print(field,results[field]['values'][[100,300,600,900,1200,1500,1800]][:,::5])

if __name__=='__main__': main()
```

40

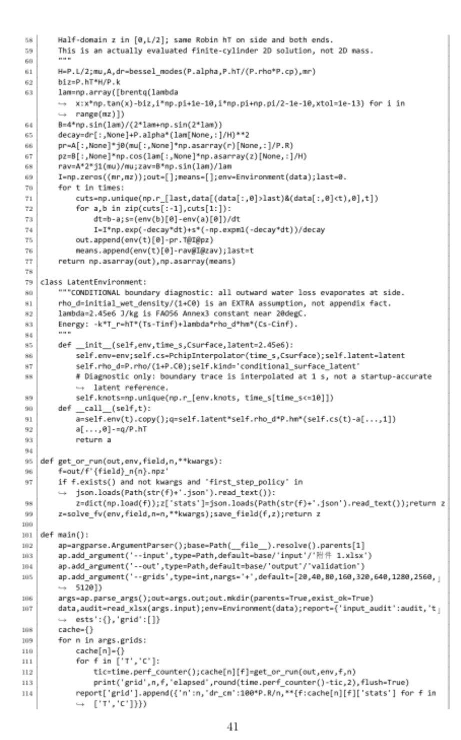


A.6 q1_validate

support/project/q1_complete_delivery/q1_delivery/source/q1_validate.py

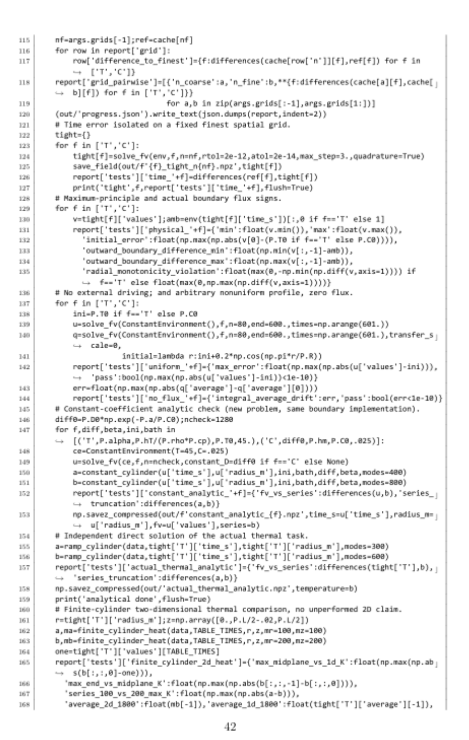
```
#!/usr/bin/env python3
"""Actual numerical tests, independent Bessel series, and model sensitivities."""
from __future__ import annotations
import argparse,json,time
from pathlib import Path
import numpy as np
from scipy.special import j0,j1,jn_zeros
from scipy.optimize import brentq
from scipy.interpolate import PchipInterpolator
from q1_solver import
↪  P,read_xlsx,Environment,ConstantEnvironment,RadialFV,solve_fv,save_field
TABLE_TIMES=np.array([100,300,600,900,1200,1500,1800])

def differences(a,b):
    x=a['values'] if isinstance(a,dict) else a
    y=b['values'] if isinstance(b,dict) else b
    d=np.abs(x-y)
    return {'max_full':float(d[1:].max()),'rms_full':float(np.sqrt(np.mean(d[1:]**2))),
            'max_table':float(d[TABLE_TIMES][:,::5].max()),
            'four_decimal_changed_full':int(np.sum(np.round(x[1:],4)!=np.round(y[1:],4))),
            'four_decimal_changed_table':int(np.sum(np.round(x[TABLE_TIMES][:,::5],4)!=np
            ↪  .round(y[TABLE_TIMES][:,::5],4))),
            'max_location_t_rindex':list(map(int,np.unravel_index(np.argmax(d),d.shape)))}

def roots_cylinder(Bi,modes):
    zj0=jn_zeros(0,modes);zj1=jn_zeros(1,modes)
    f=lambda z:z*j1(z)-Bi*j0(z)
    return np.array([brentq(f,1e-12 if i==0 else zj1[i-1]+1e-10,zj0[i]-1e-10,xtol=1e-13)
    ↪  for i in range(modes)])

def bessel_modes(diff,beta,modes=300):
    mu=roots_cylinder(beta*P.R/diff,modes)
    a=2*j1(mu)/(mu*(j0(mu)**2+j1(mu)**2))
    return mu,a,diff*(mu/P.R)**2

def constant_cylinder(times,r,initial,bath,diff,beta,modes=300):
    """Independently derived separation-of-variables Robin cylinder series."""
    mu,a,decay=bessel_modes(diff,beta,modes)
    phi=a[:,None]*j0(mu[:,None]*np.asarray(r)[None,:]/P.R)
    ans=bath+(initial-bath)*(np.exp(-np.asarray(times)[:,None]*decay)@phi)
    ans[np.asarray(times)==0]=initial
    return ans

def ramp_cylinder(data,times,r,modes=300):
    """Exact convolution on piecewise-linear environmental segments; no FV code."""
    mu,a,decay=bessel_modes(P.alpha,P.hT/(P.rho*P.cp),modes)
    phi=a[:,None]*j0(mu[:,None]*np.asarray(r)[None,:]/P.R)
    integ=np.zeros(modes);out=np.empty((len(times),len(r)));out[0]=P.T0
    env=Environment(data)
    for k,(ta,tb) in enumerate(zip(times[:-1],times[1:]),1):
        cuts=np.r_[ta,data[(data[:,0]>ta)&(data[:,0]<tb),0],tb]
        for l,h in zip(cuts[:-1],cuts[1:]):
            dt=h-l;slope=(env(h)[0]-env(l)[0])/dt
            ex=np.exp(-decay*dt)
            integ=integ*ex+slope*(-np.expm1(-decay*dt))/decay
        out[k]=env(tb)[0]-integ@phi
    return out

def finite_cylinder_heat(data,times,r,z,mr=160,mz=160):
    """2D axisymmetric r-z heat solution by product eigenfunction series.
```

41



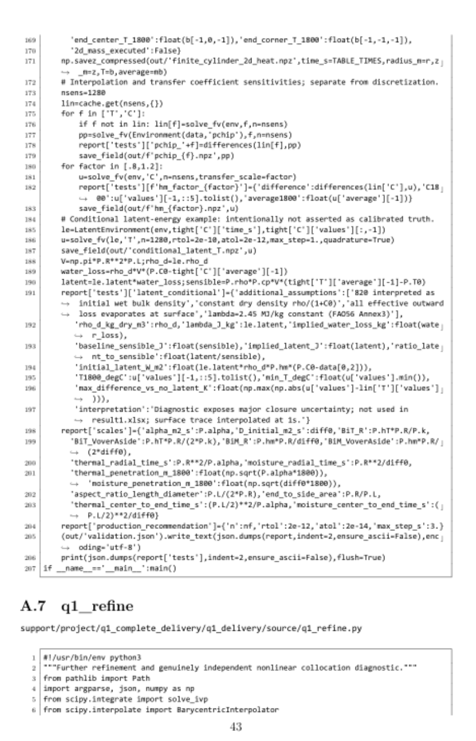
```
    Half-domain z in [0,L/2]; same Robin hT on side and both ends.
    This is an actually evaluated finite-cylinder 2D solution, not 2D mass.
    """
    H=P.L/2;mu,A,dr=bessel_modes(P.alpha,P.hT/(P.rho*P.cp),mr)
    biz=P.hT*H/P.k
    lam=np.array([brentq(lambda
    ↪  x:x*np.tan(x)-biz,i*np.pi+1e-10,i*np.pi+np.pi/2-1e-10,xtol=1e-13) for i in
    ↪  range(mz)])
    B=4*np.sin(lam)/(2*lam+np.sin(2*lam))
    decay=dr[:,None]+P.alpha*(lam[None,:]/H)**2
    pr=A[:,None]*j0(mu[:,None]*np.asarray(r)[None,:]/P.R)
    pz=B[:,None]*np.cos(lam[:,None]*np.asarray(z)[None,:]/H)
    rav=A*2*j1(mu)/mu;zav=B*np.sin(lam)/lam
    I=np.zeros((mr,mz));out=[];means=[];env=Environment(data);last=0.
    for t in times:
        cuts=np.unique(np.r_[last,data[(data[:,0]>last)&(data[:,0]<t),0],t])
        for a,b in zip(cuts[:-1],cuts[1:]):
            dt=b-a;s=(env(b)[0]-env(a)[0])/dt
            I=I*np.exp(-decay*dt)+s*(-np.expm1(-decay*dt))/decay
        out.append(env(t)[0]-pr.T@I@pz)
        means.append(env(t)[0]-rav@I@zav);last=t
    return np.asarray(out),np.asarray(means)

class LatentEnvironment:
    """CONDITIONAL boundary diagnostic: all outward water loss evaporates at side.
    rho_d=initial_wet_density/(1+C0) is an EXTRA assumption, not appendix fact.
    lambda=2.45e6 J/kg is FAO56 Annex3 constant near 20degC.
    Energy: -k*T_r=hT*(Ts-Tinf)+lambda*rho_d*hm*(Cs-Cinf).
    """
    def __init__(self,env,time_s,Csurface,latent=2.45e6):
        self.env=env;self.cs=PchipInterpolator(time_s,Csurface);self.latent=latent
        self.rho_d=P.rho/(1+P.C0);self.kind='conditional_surface_latent'
        # Diagnostic only: boundary trace is interpolated at 1 s, not a startup-accurate
        ↪  latent reference.
        self.knots=np.unique(np.r_[env.knots, time_s[time_s<=10]])
    def __call__(self,t):
        a=self.env(t).copy();q=self.latent*self.rho_d*P.hm*(self.cs(t)-a[...,1])
        a[...,0]-=q/P.hT
        return a

def get_or_run(out,env,field,n,**kwargs):
    f=out/f'{field}_n{n}.npz'
    if f.exists() and not kwargs and 'first_step_policy' in
    ↪  json.loads(Path(str(f)+'.json').read_text()):
        z=dict(np.load(f));z['stats']=json.loads(Path(str(f)+'.json').read_text());return z
    z=solve_fv(env,field,n=n,**kwargs);save_field(f,z);return z

def main():
    ap=argparse.ArgumentParser();base=Path(__file__).resolve().parents[1]
    ap.add_argument('--input',type=Path,default=base/'input'/'附件 1.xlsx')
    ap.add_argument('--out',type=Path,default=base/'output'/'validation')
    ap.add_argument('--grids',type=int,nargs='+',default=[20,40,80,160,320,640,1280,2560,
    ↪  5120])
    args=ap.parse_args();out=args.out;out.mkdir(parents=True,exist_ok=True)
    data,audit=read_xlsx(args.input);env=Environment(data);report={'input_audit':audit,'t
    ↪  ests':{},'grid':[]}
    cache={}
    for n in args.grids:
        cache[n]={}
        for f in ['T','C']:
            tic=time.perf_counter();cache[n][f]=get_or_run(out,env,f,n)
            print('grid',n,f,'elapsed',round(time.perf_counter()-tic,2),flush=True)
        report['grid'].append({'n':n,'dr_cm':100*P.R/n,**{f:cache[n][f]['stats'] for f in
        ↪  ['T','C']}})
```

42



```
    nf=args.grids[-1];ref=cache[nf]
    for row in report['grid']:
        row['difference_to_finest']={f:differences(cache[row['n']][f],ref[f]) for f in
        ↪  ['T','C']}
    report['grid_pairwise']=[{'n_coarse':a,'n_fine':b,**{f:differences(cache[a][f],cache[
    ↪  b][f]) for f in ['T','C']}}
                             for a,b in zip(args.grids[:-1],args.grids[1:])]
    (out/'progress.json').write_text(json.dumps(report,indent=2))
    # Time error isolated on a fixed finest spatial grid.
    tight={}
    for f in ['T','C']:
        tight[f]=solve_fv(env,f,n=nf,rtol=2e-12,atol=2e-14,max_step=3.,quadrature=True)
        save_field(out/f'{f}_tight_n{nf}.npz',tight[f])
        report['tests']['time_'+f]=differences(ref[f],tight[f])
        print('tight',f,report['tests']['time_'+f],flush=True)
    # Maximum-principle and actual boundary flux signs.
    for f in ['T','C']:
        v=tight[f]['values'];amb=env(tight[f]['time_s'])[:,0 if f=='T' else 1]
        report['tests']['physical_'+f]={'min':float(v.min()),'max':float(v.max()),
          'initial_error':float(np.max(np.abs(v[0]-(P.T0 if f=='T' else P.C0)))),
          'outward_boundary_difference_min':float(np.min(v[:,-1]-amb)),
          'outward_boundary_difference_max':float(np.max(v[:,-1]-amb)),
          'radial_monotonicity_violation':float(max(0,-np.min(np.diff(v,axis=1)))) if
          ↪  f=='T' else float(max(0,np.max(np.diff(v,axis=1))))}
    # No external driving; and arbitrary nonuniform profile, zero flux.
    for f in ['T','C']:
        ini=P.T0 if f=='T' else P.C0
        u=solve_fv(ConstantEnvironment(),f,n=80,end=600.,times=np.arange(601.))
        q=solve_fv(ConstantEnvironment(),f,n=80,end=600.,times=np.arange(601.),transfer_s
        ↪  cale=0,
                   initial=lambda r:ini+0.2*np.cos(np.pi*r/P.R))
        report['tests']['uniform_'+f]={'max_error':float(np.max(np.abs(u['values']-ini))),
        ↪  'pass':bool(np.max(np.abs(u['values']-ini))<1e-10)}
        err=float(np.max(np.abs(q['average']-q['average'][0])))
        report['tests']['no_flux_'+f]={'integral_average_drift':err,'pass':bool(err<1e-10)}
    # Constant-coefficient analytic check (new problem, same boundary implementation).
    diff0=P.D0*np.exp(-P.a/P.C0);ncheck=1280
    for f,diff,beta,ini,bath in
    ↪  [('T',P.alpha,P.hT/(P.rho*P.cp),P.T0,45.),('C',diff0,P.hm,P.C0,.025)]:
        ce=ConstantEnvironment(T=45,C=.025)
        u=solve_fv(ce,f,n=ncheck,constant_D=diff0 if f=='C' else None)
        a=constant_cylinder(u['time_s'],u['radius_m'],ini,bath,diff,beta,modes=400)
        b=constant_cylinder(u['time_s'],u['radius_m'],ini,bath,diff,beta,modes=800)
        report['tests']['constant_analytic_'+f]={'fv_vs_series':differences(u,b),'series_
        ↪  truncation':differences(a,b)}
        np.savez_compressed(out/f'constant_analytic_{f}.npz',time_s=u['time_s'],radius_m=
        ↪  u['radius_m'],fv=u['values'],series=b)
    # Independent direct solution of the actual thermal task.
    a=ramp_cylinder(data,tight['T']['time_s'],tight['T']['radius_m'],modes=300)
    b=ramp_cylinder(data,tight['T']['time_s'],tight['T']['radius_m'],modes=600)
    report['tests']['actual_thermal_analytic']={'fv_vs_series':differences(tight['T'],b),
    ↪  'series_truncation':differences(a,b)}
    np.savez_compressed(out/'actual_thermal_analytic.npz',temperature=b)
    print('analytical done',flush=True)
    # Finite-cylinder two-dimensional thermal comparison, no unperformed 2D claim.
    r=tight['T']['radius_m'];z=np.array([0.,P.L/2-.02,P.L/2])
    a,ma=finite_cylinder_heat(data,TABLE_TIMES,r,z,mr=100,mz=100)
    b,mb=finite_cylinder_heat(data,TABLE_TIMES,r,z,mr=200,mz=200)
    one=tight['T']['values'][TABLE_TIMES]
    report['tests']['finite_cylinder_2d_heat']={'max_midplane_vs_1d_K':float(np.max(np.ab
    ↪  s(b[:,:,0]-one))),
      'max_end_vs_midplane_K':float(np.max(np.abs(b[:,:,-1]-b[:,:,0]))),
      'series_100_vs_200_max_K':float(np.max(np.abs(a-b))),
      'average_2d_1800':float(mb[-1]),'average_1d_1800':float(tight['T']['average'][-1]),
```

43



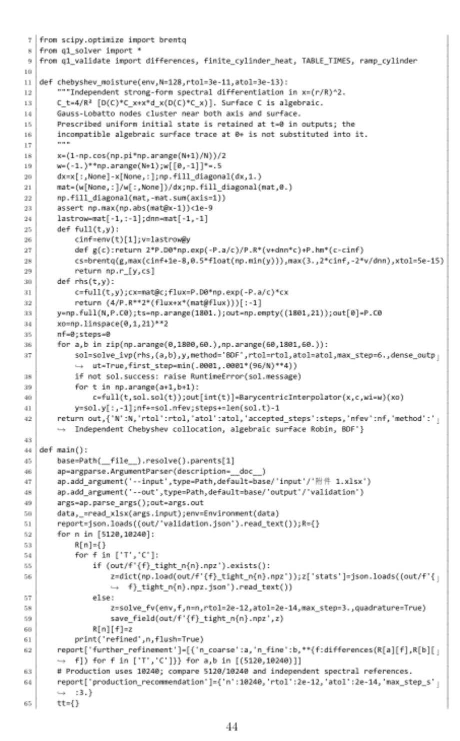
```
      'end_center_T_1800':float(b[-1,0,-1]),'end_corner_T_1800':float(b[-1,-1,-1]),
      '2d_mass_executed':False}
    np.savez_compressed(out/'finite_cylinder_2d_heat.npz',time_s=TABLE_TIMES,radius_m=r,z
    ↪  _m=z,T=b,average=mb)
    # Interpolation and transfer coefficient sensitivities; separate from discretization.
    nsens=1280
    lin=cache.get(nsens,{})
    for f in ['T','C']:
        if f not in lin: lin[f]=solve_fv(env,f,n=nsens)
        pp=solve_fv(Environment(data,'pchip'),f,n=nsens)
        report['tests']['pchip_'+f]=differences(lin[f],pp)
        save_field(out/f'pchip_{f}.npz',pp)
    for factor in [.8,1.2]:
        u=solve_fv(env,'C',n=nsens,transfer_scale=factor)
        report['tests'][f'hm_factor_{factor}']={'difference':differences(lin['C'],u),'C18
        ↪  00':u['values'][-1,::5].tolist(),'average1800':float(u['average'][-1])}
        save_field(out/f'hm_{factor}.npz',u)
    # Conditional latent-energy example: intentionally not asserted as calibrated truth.
    le=LatentEnvironment(env,tight['C']['time_s'],tight['C']['values'][:,-1])
    u=solve_fv(le,'T',n=1280,rtol=2e-10,atol=2e-12,max_step=1.,quadrature=True)
    save_field(out/'conditional_latent_T.npz',u)
    V=np.pi*P.R**2*P.L;rho_d=le.rho_d
    water_loss=rho_d*V*(P.C0-tight['C']['average'][-1])
    latent=le.latent*water_loss;sensible=P.rho*P.cp*V*(tight['T']['average'][-1]-P.T0)
    report['tests']['latent_conditional']={'additional_assumptions':['820 interpreted as
    ↪  initial wet bulk density','constant dry density rho/(1+C0)','all effective outward
    ↪  loss evaporates at surface','lambda=2.45 MJ/kg constant (FAO56 Annex3)'],
      'rho_d_kg_dry_m3':rho_d,'lambda_J_kg':le.latent,'implied_water_loss_kg':float(wate
      ↪  r_loss),
      'baseline_sensible_J':float(sensible),'implied_latent_J':float(latent),'ratio_late
      ↪  nt_to_sensible':float(latent/sensible),
      'initial_latent_W_m2':float(le.latent*rho_d*P.hm*(P.C0-data[0,2])),
      'T1800_degC':u['values'][-1,::5].tolist(),'min_T_degC':float(u['values'].min()),
      'max_difference_vs_no_latent_K':float(np.max(np.abs(u['values']-lin['T']['values']
      ↪  ))),
      'interpretation':'Diagnostic exposes major closure uncertainty; not used in
      ↪  result1.xlsx; surface trace interpolated at 1s.'}
    report['scales']={'alpha_m2_s':P.alpha,'D_initial_m2_s':diff0,'BiT_R':P.hT*P.R/P.k,
      'BiT_VoverAside':P.hT*P.R/(2*P.k),'BiM_R':P.hm*P.R/diff0,'BiM_VoverAside':P.hm*P.R/
      ↪  (2*diff0),
      'thermal_radial_time_s':P.R**2/P.alpha,'moisture_radial_time_s':P.R**2/diff0,
      'thermal_penetration_m_1800':float(np.sqrt(P.alpha*1800)),
      ↪  'moisture_penetration_m_1800':float(np.sqrt(diff0*1800)),
      'aspect_ratio_length_diameter':P.L/(2*P.R),'end_to_side_area':P.R/P.L,
      'thermal_center_to_end_time_s':(P.L/2)**2/P.alpha,'moisture_center_to_end_time_s':(
      ↪  P.L/2)**2/diff0}
    report['production_recommendation']={'n':nf,'rtol':2e-12,'atol':2e-14,'max_step_s':3.}
    (out/'validation.json').write_text(json.dumps(report,indent=2,ensure_ascii=False),enc
    ↪  oding='utf-8')
    print(json.dumps(report['tests'],indent=2,ensure_ascii=False),flush=True)
if __name__=='__main__':main()
```

A.7 q1_refine

support/project/q1_complete_delivery/q1_delivery/source/q1_refine.py

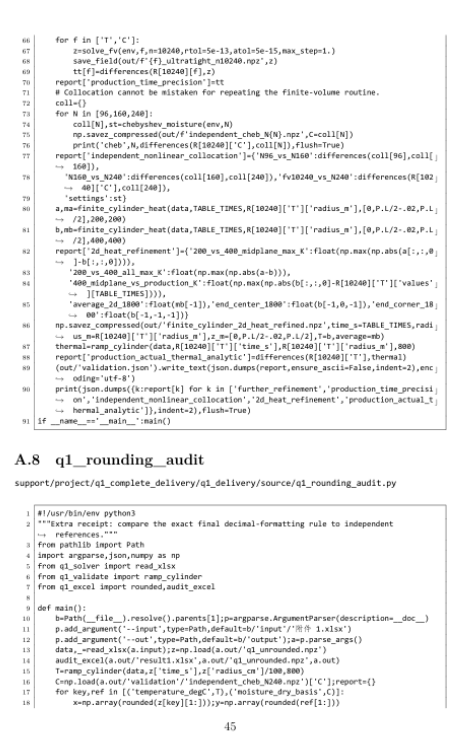
```
#!/usr/bin/env python3
"""Further refinement and genuinely independent nonlinear collocation diagnostic."""
from pathlib import Path
import argparse, json, numpy as np
from scipy.integrate import solve_ivp
from scipy.interpolate import BarycentricInterpolator
```

44



```
from scipy.optimize import brentq
from q1_solver import *
from q1_validate import differences, finite_cylinder_heat, TABLE_TIMES, ramp_cylinder

def chebyshev_moisture(env,N=128,rtol=3e-11,atol=3e-13):
    """Independent strong-form spectral differentiation in x=(r/R)^2.
    C_t=4/R² [D(C)*C_x+x*d_x(D(C)*C_x)]. Surface C is algebraic.
    Gauss-Lobatto nodes cluster near both axis and surface.
    Prescribed uniform initial state is retained at t=0 in outputs; the
    incompatible algebraic surface trace at 0+ is not substituted into it.
    """
    x=(1-np.cos(np.pi*np.arange(N+1)/N))/2
    w=(-1.)**np.arange(N+1);w[[0,-1]]*=.5
    dx=x[:,None]-x[None,:];np.fill_diagonal(dx,1.)
    mat=(w[None,:]/w[:,None])/dx;np.fill_diagonal(mat,0.)
    np.fill_diagonal(mat,-mat.sum(axis=1))
    assert np.max(np.abs(mat@x-1))<1e-9
    lastrow=mat[-1,:-1];dnn=mat[-1,-1]
    def full(t,y):
        cinf=env(t)[1];v=lastrow@y
        def g(c):return 2*P.D0*np.exp(-P.a/c)/P.R*(v+dnn*c)+P.hm*(c-cinf)
        cs=brentq(g,max(cinf+1e-8,0.5*float(np.min(y))),max(3.,2*cinf,-2*v/dnn),xtol=5e-15)
        return np.r_[y,cs]
    def rhs(t,y):
        c=full(t,y);cx=mat@c;flux=P.D0*np.exp(-P.a/c)*cx
        return (4/P.R**2*(flux+x*(mat@flux)))[:-1]
    y=np.full(N,P.C0);ts=np.arange(1801.);out=np.empty((1801,21));out[0]=P.C0
    xo=np.linspace(0,1,21)**2
    nf=0;steps=0
    for a,b in zip(np.arange(0,1800,60.),np.arange(60,1801,60.)):
        sol=solve_ivp(rhs,(a,b),y,method='BDF',rtol=rtol,atol=atol,max_step=6.,dense_outp
        ↪  ut=True,first_step=min(.0001,.0001*(96/N)**4))
        if not sol.success: raise RuntimeError(sol.message)
        for t in np.arange(a+1,b+1):
            c=full(t,sol.sol(t));out[int(t)]=BarycentricInterpolator(x,c,wi=w)(xo)
        y=sol.y[:,-1];nf+=sol.nfev;steps+=len(sol.t)-1
    return out,{'N':N,'rtol':rtol,'atol':atol,'accepted_steps':steps,'nfev':nf,'method':'
    ↪  Independent Chebyshev collocation, algebraic surface Robin, BDF'}

def main():
    base=Path(__file__).resolve().parents[1]
    ap=argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--input',type=Path,default=base/'input'/'附件 1.xlsx')
    ap.add_argument('--out',type=Path,default=base/'output'/'validation')
    args=ap.parse_args();out=args.out
    data,_=read_xlsx(args.input);env=Environment(data)
    report=json.loads((out/'validation.json').read_text());R={}
    for n in [5120,10240]:
        R[n]={}
        for f in ['T','C']:
            if (out/f'{f}_tight_n{n}.npz').exists():
                z=dict(np.load(out/f'{f}_tight_n{n}.npz'));z['stats']=json.loads((out/f'{
                ↪  f}_tight_n{n}.npz.json').read_text())
            else:
                z=solve_fv(env,f,n=n,rtol=2e-12,atol=2e-14,max_step=3.,quadrature=True)
                save_field(out/f'{f}_tight_n{n}.npz',z)
            R[n][f]=z
        print('refined',n,flush=True)
    report['further_refinement']=[{'n_coarse':a,'n_fine':b,**{f:differences(R[a][f],R[b][
    ↪  f]) for f in ['T','C']}} for a,b in [(5120,10240)]]
    # Production uses 10240; compare 5120/10240 and independent spectral references.
    report['production_recommendation']={'n':10240,'rtol':2e-12,'atol':2e-14,'max_step_s'
    ↪  :3.}
    tt={}
```

45



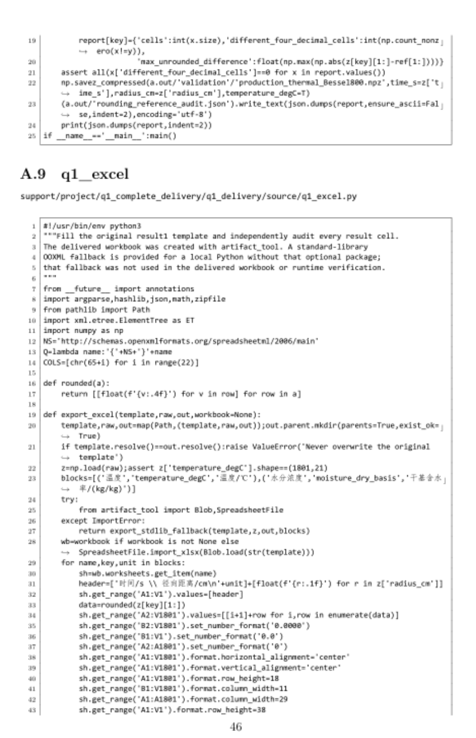
```
    for f in ['T','C']:
        z=solve_fv(env,f,n=10240,rtol=5e-13,atol=5e-15,max_step=1.)
        save_field(out/f'{f}_ultratight_n10240.npz',z)
        tt[f]=differences(R[10240][f],z)
    report['production_time_precision']=tt
    # Collocation cannot be mistaken for repeating the finite-volume routine.
    coll={}
    for N in [96,160,240]:
        coll[N],st=chebyshev_moisture(env,N)
        np.savez_compressed(out/f'independent_cheb_N{N}.npz',C=coll[N])
        print('cheb',N,differences(R[10240]['C'],coll[N]),flush=True)
    report['independent_nonlinear_collocation']={'N96_vs_N160':differences(coll[96],coll[
    ↪  160]),
      'N160_vs_N240':differences(coll[160],coll[240]),'fv10240_vs_N240':differences(R[102
      ↪  40]['C'],coll[240]),
      'settings':st}
    a,ma=finite_cylinder_heat(data,TABLE_TIMES,R[10240]['T']['radius_m'],[0,P.L/2-.02,P.L
    ↪  /2],200,200)
    b,mb=finite_cylinder_heat(data,TABLE_TIMES,R[10240]['T']['radius_m'],[0,P.L/2-.02,P.L
    ↪  /2],400,400)
    report['2d_heat_refinement']={'200_vs_400_midplane_max_K':float(np.max(np.abs(a[:,:,0
    ↪  ]-b[:,:,0]))),
       '200_vs_400_all_max_K':float(np.max(np.abs(a-b))),
       '400_midplane_vs_production_K':float(np.max(np.abs(b[:,:,0]-R[10240]['T']['values'
       ↪  ][TABLE_TIMES]))),
       'average_2d_1800':float(mb[-1]),'end_center_1800':float(b[-1,0,-1]),'end_corner_18
       ↪  00':float(b[-1,-1,-1])}
    np.savez_compressed(out/'finite_cylinder_2d_heat_refined.npz',time_s=TABLE_TIMES,radi
    ↪  us_m=R[10240]['T']['radius_m'],z_m=[0,P.L/2-.02,P.L/2],T=b,average=mb)
    thermal=ramp_cylinder(data,R[10240]['T']['time_s'],R[10240]['T']['radius_m'],800)
    report['production_actual_thermal_analytic']=differences(R[10240]['T'],thermal)
    (out/'validation.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),enc
    ↪  oding='utf-8')
    print(json.dumps({k:report[k] for k in ['further_refinement','production_time_precisi
    ↪  on','independent_nonlinear_collocation','2d_heat_refinement','production_actual_t
    ↪  hermal_analytic']},indent=2),flush=True)
if __name__=='__main__':main()
```

A.8 q1_rounding_audit

support/project/q1_complete_delivery/q1_delivery/source/q1_rounding_audit.py

```
#!/usr/bin/env python3
"""Extra receipt: compare the exact final decimal-formatting rule to independent
↪  references."""
from pathlib import Path
import argparse,json,numpy as np
from q1_solver import read_xlsx
from q1_validate import ramp_cylinder
from q1_excel import rounded,audit_excel

def main():
    b=Path(__file__).resolve().parents[1];p=argparse.ArgumentParser(description=__doc__)
    p.add_argument('--input',type=Path,default=b/'input'/'附件 1.xlsx')
    p.add_argument('--out',type=Path,default=b/'output');a=p.parse_args()
    data,_=read_xlsx(a.input);z=np.load(a.out/'q1_unrounded.npz')
    audit_excel(a.out/'result1.xlsx',a.out/'q1_unrounded.npz',a.out)
    T=ramp_cylinder(data,z['time_s'],z['radius_cm']/100,800)
    C=np.load(a.out/'validation'/'independent_cheb_N240.npz')['C'];report={}
    for key,ref in [('temperature_degC',T),('moisture_dry_basis',C)]:
        x=np.array(rounded(z[key][1:]));y=np.array(rounded(ref[1:]))
```

46



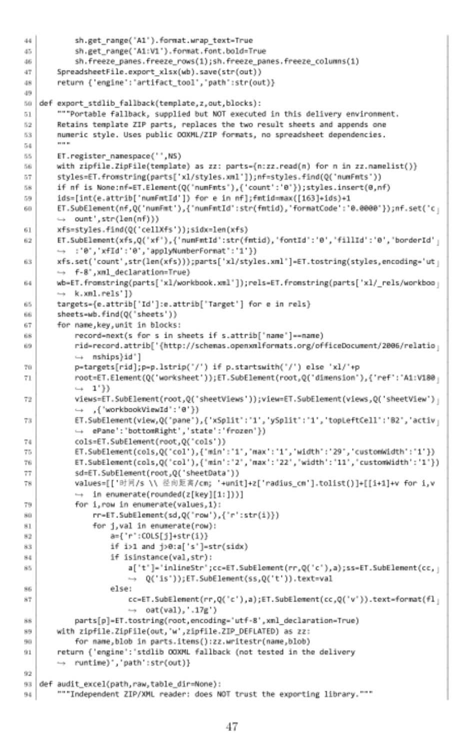
```
        report[key]={'cells':int(x.size),'different_four_decimal_cells':int(np.count_nonz
        ↪  ero(x!=y)),
                     'max_unrounded_difference':float(np.max(np.abs(z[key][1:]-ref[1:])))}
    assert all(x['different_four_decimal_cells']==0 for x in report.values())
    np.savez_compressed(a.out/'validation'/'production_thermal_Bessel800.npz',time_s=z['t
    ↪  ime_s'],radius_cm=z['radius_cm'],temperature_degC=T)
    (a.out/'rounding_reference_audit.json').write_text(json.dumps(report,ensure_ascii=Fal
    ↪  se,indent=2),encoding='utf-8')
    print(json.dumps(report,indent=2))
if __name__=='__main__':main()
```

A.9 q1_excel

support/project/q1_complete_delivery/q1_delivery/source/q1_excel.py

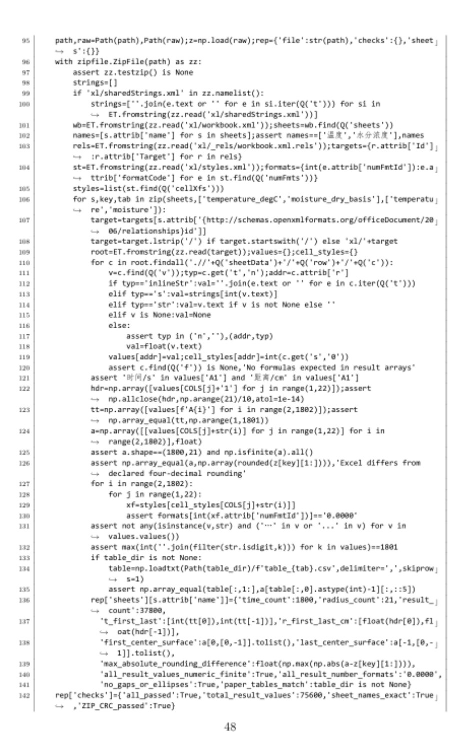
```
#!/usr/bin/env python3
"""Fill the original result1 template and independently audit every result cell.
The delivered workbook was created with artifact_tool. A standard-library
OOXML fallback is provided for a local Python without that optional package;
that fallback was not used in the delivered workbook or runtime verification.
"""
from __future__ import annotations
import argparse,hashlib,json,math,zipfile
from pathlib import Path
import xml.etree.ElementTree as ET
import numpy as np
NS='http://schemas.openxmlformats.org/spreadsheetml/2006/main'
Q=lambda name:'{'+NS+'}'+name
COLS=[chr(65+i) for i in range(22)]

def rounded(a):
    return [[float(f'{v:.4f}') for v in row] for row in a]

def export_excel(template,raw,out,workbook=None):
    template,raw,out=map(Path,(template,raw,out));out.parent.mkdir(parents=True,exist_ok=
    ↪  True)
    if template.resolve()==out.resolve():raise ValueError('Never overwrite the original
    ↪  template')
    z=np.load(raw);assert z['temperature_degC'].shape==(1801,21)
    blocks=[('温度','temperature_degC','温度/℃'),('水分浓度','moisture_dry_basis','干基含水
    ↪  率/(kg/kg)')]
    try:
        from artifact_tool import Blob,SpreadsheetFile
    except ImportError:
        return export_stdlib_fallback(template,z,out,blocks)
    wb=workbook if workbook is not None else
    ↪  SpreadsheetFile.import_xlsx(Blob.load(str(template)))
    for name,key,unit in blocks:
        sh=wb.worksheets.get_item(name)
        header=['时间/s \\ 径向距离/cm\n'+unit]+[float(f'{r:.1f}') for r in z['radius_cm']]
        sh.get_range('A1:V1').values=[header]
        data=rounded(z[key][1:])
        sh.get_range('A2:V1801').values=[[i+1]+row for i,row in enumerate(data)]
        sh.get_range('B2:V1801').set_number_format('0.0000')
        sh.get_range('B1:V1').set_number_format('0.0')
        sh.get_range('A2:A1801').set_number_format('0')
        sh.get_range('A1:V1801').format.horizontal_alignment='center'
        sh.get_range('A1:V1801').format.vertical_alignment='center'
        sh.get_range('A1:V1801').format.row_height=18
        sh.get_range('B1:V1801').format.column_width=11
        sh.get_range('A1:A1801').format.column_width=29
        sh.get_range('A1:V1').format.row_height=38
```

47



```
        sh.get_range('A1').format.wrap_text=True
        sh.get_range('A1:V1').format.font.bold=True
        sh.freeze_panes.freeze_rows(1);sh.freeze_panes.freeze_columns(1)
    SpreadsheetFile.export_xlsx(wb).save(str(out))
    return {'engine':'artifact_tool','path':str(out)}

def export_stdlib_fallback(template,z,out,blocks):
    """Portable fallback, supplied but NOT executed in this delivery environment.
    Retains template ZIP parts, replaces the two result sheets and appends one
    numeric style. Uses public OOXML/ZIP formats, no spreadsheet dependencies.
    """
    ET.register_namespace('',NS)
    with zipfile.ZipFile(template) as zz: parts={n:zz.read(n) for n in zz.namelist()}
    styles=ET.fromstring(parts['xl/styles.xml']);nf=styles.find(Q('numFmts'))
    if nf is None:nf=ET.Element(Q('numFmts'),{'count':'0'});styles.insert(0,nf)
    ids=[int(e.attrib['numFmtId']) for e in nf];fmtid=max([163]+ids)+1
    ET.SubElement(nf,Q('numFmt'),{'numFmtId':str(fmtid),'formatCode':'0.0000'});nf.set('c
    ↪  ount',str(len(nf)))
    xfs=styles.find(Q('cellXfs'));sidx=len(xfs)
    ET.SubElement(xfs,Q('xf'),{'numFmtId':str(fmtid),'fontId':'0','fillId':'0','borderId'
    ↪  :'0','xfId':'0','applyNumberFormat':'1'})
    xfs.set('count',str(len(xfs)));parts['xl/styles.xml']=ET.tostring(styles,encoding='ut
    ↪  f-8',xml_declaration=True)
    wb=ET.fromstring(parts['xl/workbook.xml']);rels=ET.fromstring(parts['xl/_rels/workboo
    ↪  k.xml.rels'])
    targets={e.attrib['Id']:e.attrib['Target'] for e in rels}
    sheets=wb.find(Q('sheets'))
    for name,key,unit in blocks:
        record=next(s for s in sheets if s.attrib['name']==name)
        rid=record.attrib['{http://schemas.openxmlformats.org/officeDocument/2006/relatio
        ↪  nships}id']
        p=targets[rid];p=p.lstrip('/') if p.startswith('/') else 'xl/'+p
        root=ET.Element(Q('worksheet'));ET.SubElement(root,Q('dimension'),{'ref':'A1:V180
        ↪  1'})
        views=ET.SubElement(root,Q('sheetViews'));view=ET.SubElement(views,Q('sheetView')
        ↪  ,{'workbookViewId':'0'})
        ET.SubElement(view,Q('pane'),{'xSplit':'1','ySplit':'1','topLeftCell':'B2','activ
        ↪  ePane':'bottomRight','state':'frozen'})
        cols=ET.SubElement(root,Q('cols'))
        ET.SubElement(cols,Q('col'),{'min':'1','max':'1','width':'29','customWidth':'1'})
        ET.SubElement(cols,Q('col'),{'min':'2','max':'22','width':'11','customWidth':'1'})
        sd=ET.SubElement(root,Q('sheetData'))
        values=[['时间/s \\ 径向距离/cm; '+unit]+z['radius_cm'].tolist()]+[[i+1]+v for i,v
        ↪  in enumerate(rounded(z[key][1:]))]
        for i,row in enumerate(values,1):
            rr=ET.SubElement(sd,Q('row'),{'r':str(i)})
            for j,val in enumerate(row):
                a={'r':COLS[j]+str(i)}
                if i>1 and j>0:a['s']=str(sidx)
                if isinstance(val,str):
                    a['t']='inlineStr';cc=ET.SubElement(rr,Q('c'),a);ss=ET.SubElement(cc,
                    ↪  Q('is'));ET.SubElement(ss,Q('t')).text=val
                else:
                    cc=ET.SubElement(rr,Q('c'),a);ET.SubElement(cc,Q('v')).text=format(fl
                    ↪  oat(val),'.17g')
        parts[p]=ET.tostring(root,encoding='utf-8',xml_declaration=True)
    with zipfile.ZipFile(out,'w',zipfile.ZIP_DEFLATED) as zz:
        for name,blob in parts.items():zz.writestr(name,blob)
    return {'engine':'stdlib OOXML fallback (not tested in the delivery
    ↪  runtime)','path':str(out)}

def audit_excel(path,raw,table_dir=None):
    """Independent ZIP/XML reader: does NOT trust the exporting library."""
```

48



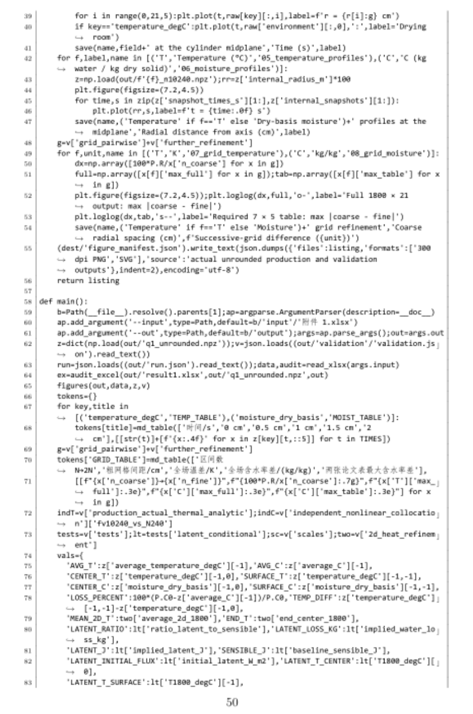
```
    path,raw=Path(path),Path(raw);z=np.load(raw);rep={'file':str(path),'checks':{},'sheet
    ↪  s':{}}
    with zipfile.ZipFile(path) as zz:
        assert zz.testzip() is None
        strings=[]
        if 'xl/sharedStrings.xml' in zz.namelist():
            strings=[''.join(e.text or '' for e in si.iter(Q('t'))) for si in
            ↪  ET.fromstring(zz.read('xl/sharedStrings.xml'))]
        wb=ET.fromstring(zz.read('xl/workbook.xml'));sheets=wb.find(Q('sheets'))
        names=[s.attrib['name'] for s in sheets];assert names==['温度','水分浓度'],names
        rels=ET.fromstring(zz.read('xl/_rels/workbook.xml.rels'));targets={r.attrib['Id']
        ↪  :r.attrib['Target'] for r in rels}
        st=ET.fromstring(zz.read('xl/styles.xml'));formats={int(e.attrib['numFmtId']):e.a
        ↪  ttrib['formatCode'] for e in st.find(Q('numFmts'))}
        styles=list(st.find(Q('cellXfs')))
        for s,key,tab in zip(sheets,['temperature_degC','moisture_dry_basis'],['temperatu
        ↪  re','moisture']):
            target=targets[s.attrib['{http://schemas.openxmlformats.org/officeDocument/20
            ↪  06/relationships}id']]
            target=target.lstrip('/') if target.startswith('/') else 'xl/'+target
            root=ET.fromstring(zz.read(target));values={};cell_styles={}
            for c in root.findall('.//'+Q('sheetData')+'/'+Q('row')+'/'+Q('c')):
                v=c.find(Q('v'));typ=c.get('t','n');addr=c.attrib['r']
                if typ=='inlineStr':val=''.join(e.text or '' for e in c.iter(Q('t')))
                elif typ=='s':val=strings[int(v.text)]
                elif typ=='str':val=v.text if v is not None else ''
                elif v is None:val=None
                else:
                    assert typ in ('n',''),(addr,typ)
                    val=float(v.text)
                values[addr]=val;cell_styles[addr]=int(c.get('s','0'))
                assert c.find(Q('f')) is None,'No formulas expected in result arrays'
            assert '时间/s' in values['A1'] and '距离/cm' in values['A1']
            hdr=np.array([values[COLS[j]+'1'] for j in range(1,22)]);assert
            ↪  np.allclose(hdr,np.arange(21)/10,atol=1e-14)
            tt=np.array([values[f'A{i}'] for i in range(2,1802)]);assert
            ↪  np.array_equal(tt,np.arange(1,1801))
            a=np.array([[values[COLS[j]+str(i)] for j in range(1,22)] for i in
            ↪  range(2,1802)],float)
            assert a.shape==(1800,21) and np.isfinite(a).all()
            assert np.array_equal(a,np.array(rounded(z[key][1:]))),'Excel differs from
            ↪  declared four-decimal rounding'
            for i in range(2,1802):
                for j in range(1,22):
                    xf=styles[cell_styles[COLS[j]+str(i)]]
                    assert formats[int(xf.attrib['numFmtId'])]=='0.0000'
            assert not any(isinstance(v,str) and ('…' in v or '...' in v) for v in
            ↪  values.values())
            assert max(int(''.join(filter(str.isdigit,k))) for k in values)==1801
            if table_dir is not None:
                table=np.loadtxt(Path(table_dir)/f'table_{tab}.csv',delimiter=',',skiprow
                ↪  s=1)
                assert np.array_equal(table[:,1:],a[table[:,0].astype(int)-1][:,::5])
            rep['sheets'][s.attrib['name']]={'time_count':1800,'radius_count':21,'result_
            ↪  count':37800,
              't_first_last':[int(tt[0]),int(tt[-1])],'r_first_last_cm':[float(hdr[0]),fl
              ↪  oat(hdr[-1])],
              'first_center_surface':a[0,[0,-1]].tolist(),'last_center_surface':a[-1,[0,-
              ↪  1]].tolist(),
              'max_absolute_rounding_difference':float(np.max(np.abs(a-z[key][1:]))),
              'all_result_values_numeric_finite':True,'all_result_number_formats':'0.0000',
              'no_gaps_or_ellipses':True,'paper_tables_match':table_dir is not None}
    rep['checks']={'all_passed':True,'total_result_values':75600,'sheet_names_exact':True
    ↪  ,'ZIP_CRC_passed':True}
```

49

A.10 q1_report

support/project/q1_complete_delivery/q1_delivery/source/q1_report.py

50



51

52

A.11 第二问机制与参数情景

A.12 q2_core

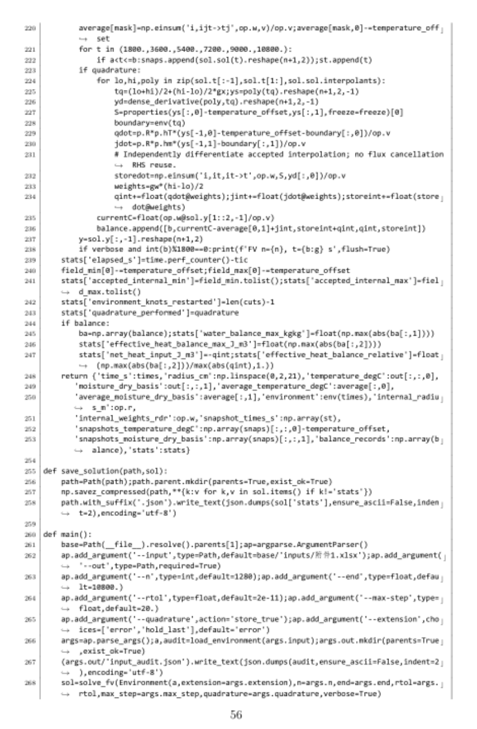
support/project/q2_refinement_delivery/runtime/source/q2_core.py

53

54

55

56



57

A.13 q2_spectral

support/project/q2_refinement_delivery/runtime/source/q2_spectral.py

58

59

60

A.14 q2_tests

support/project/q2_refinement_delivery/runtime/source/q2_tests.py

61

62

63

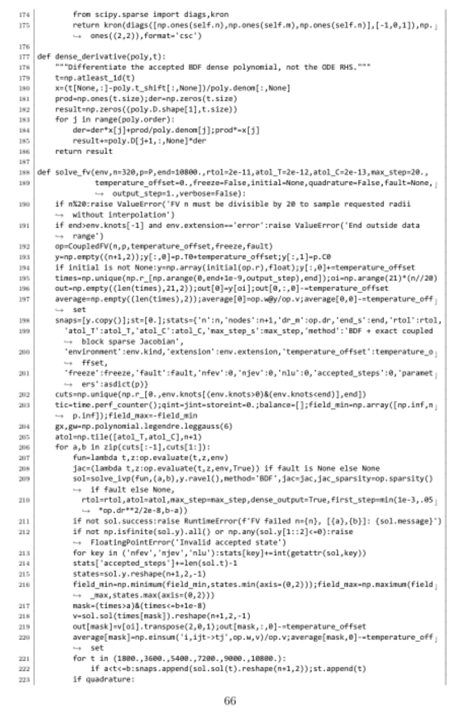
A.15 q2_core_original

support/project/q2_refinement_delivery/reused/q2_core_original.py

64

65

66



67

68

A.16 prepare_reused

support/project/q2_refinement_delivery/source/prepare_reused.py

69

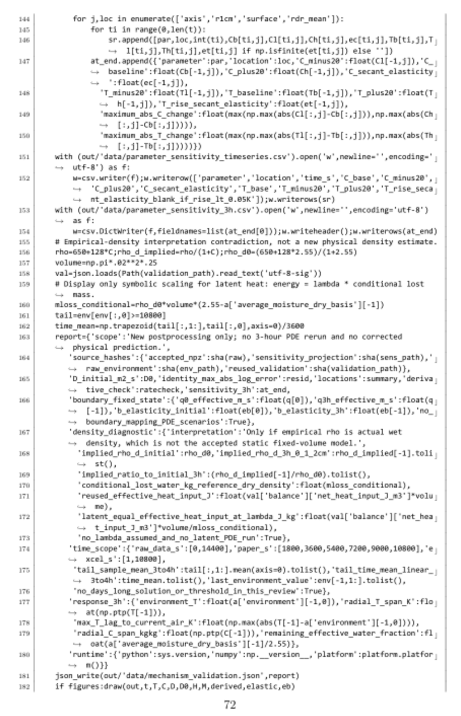
A.17 q2_mechanism

support/project/q2_refinement_delivery/source/q2_mechanism.py

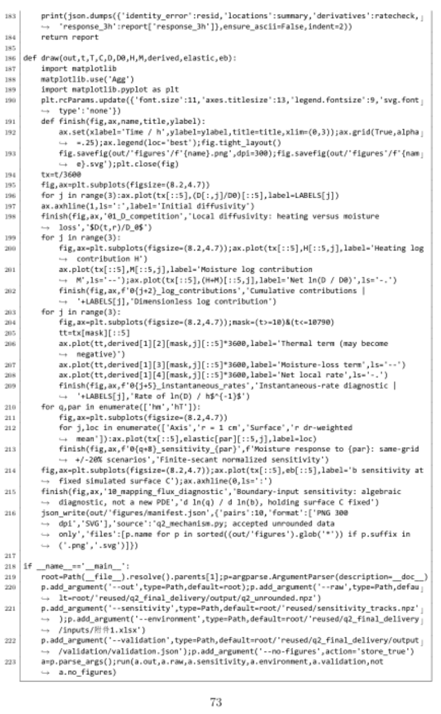
70

71

72



73



74

75

76

77

78

79

80

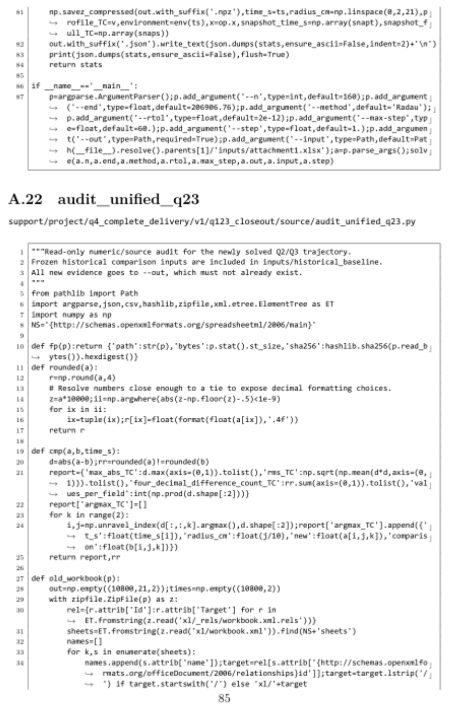
81

82

83

84

85



86

87

88

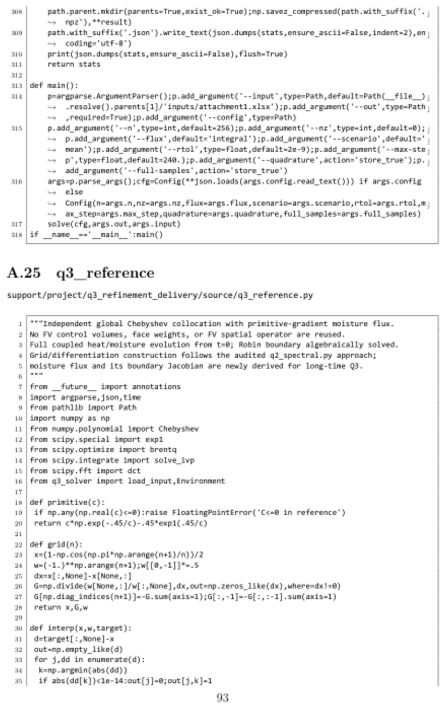
89

90

91

92

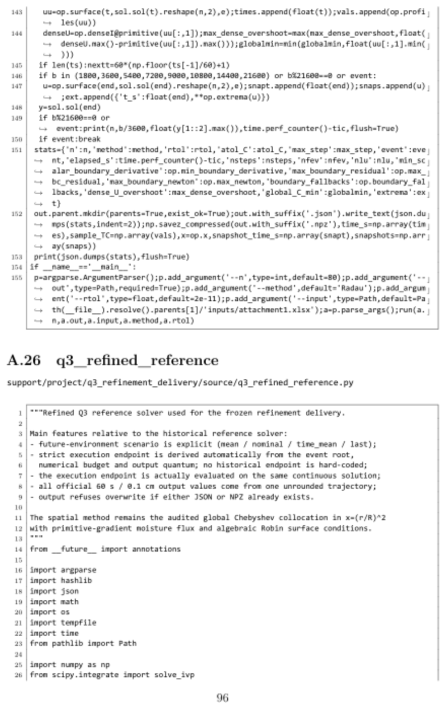
93



94

95

96

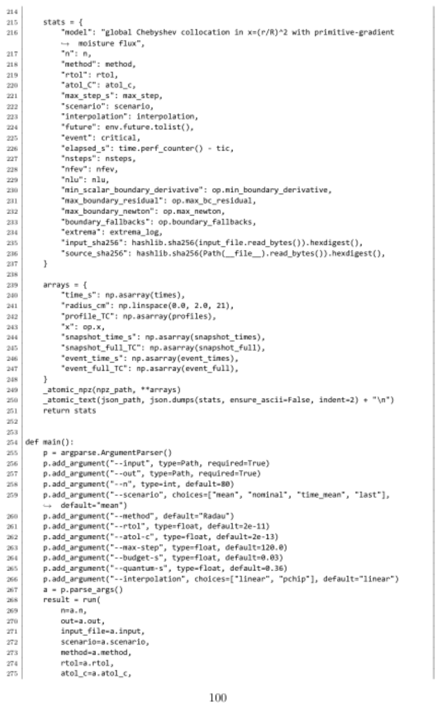


97

98

99

100



101

102

103

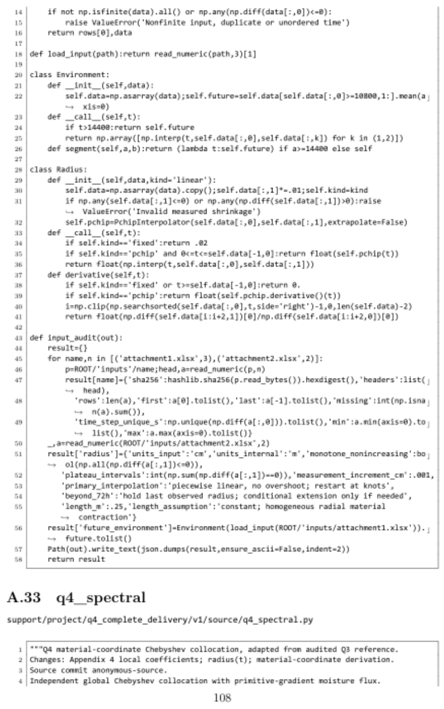
104

105

106

107

108



109

```python
No FV control volumes, face weights, or FV spatial operator are reused.
Full coupled heat/moisture evolution from t=0; Robin boundary algebraically solved.
Grid/differentiation construction follows the audited q2_spectral.py approach;
moisture flux and its boundary Jacobian are newly derived for long-time Q3.
"""
from __future__ import annotations
import argparse,json,time
from pathlib import Path
import numpy as np
from numpy.polynomial import Chebyshev
from scipy.special import exp1
from scipy.optimize import brentq
from scipy.integrate import solve_ivp
from scipy.fft import dct
from q4_common import load_input,Environment

def primitive(c):
 if np.any(np.real(c)<=0):raise FloatingPointError('C<=0 in reference')
 return c*np.exp(-.30/c)-.30*exp1(.30/c)

def grid(n):
 x=(1-np.cos(np.pi*np.arange(n+1)/n))/2
 w=(-1.)**np.arange(n+1);w[[0,-1]]*=.5
 dx=x[:,None]-x[None,:]
 G=np.divide(w[None,:]/w[:,None],dx,out=np.zeros_like(dx),where=dx!=0)
 G[np.diag_indices(n+1)]=-G.sum(axis=1);G[:,-1]=-G[:,:-1].sum(axis=1)
 return x,G,w

def interp(x,w,target):
 d=target[:,None]-x
 out=np.empty_like(d)
 for j,dd in enumerate(d):
  k=np.argmin(abs(dd))
  if abs(dd[k])<1e-14:out[j]=0;out[j,k]=1
  else:v=w/dd;out[j]=v/v.sum()
 return out

def inverse_primitive(v,guess):
 c=np.asarray(guess).copy()
 for _ in range(40):
  delta=(primitive(c)-v)/np.exp(-.30/c)
  cnew=c-delta
  # Globalize Newton without altering the equation or accepted value.
  bad=np.real(cnew)<=0
  while np.any(bad):delta[bad]*=.5;cnew=c-delta;bad=np.real(cnew)<=0
  c=cnew
  if np.max(abs(delta))<3e-14:break
 else:raise RuntimeError('Inverse primitive failed')
 return c

class Reference:
 def __init__(self,n,radius):
  self.radius=radius
  self.n=n;self.x,self.G,self.w=grid(n);self.row=self.G[-1,:-1];self.diag=self.G[-1,-1]
  self.fac=4/.02**2;self.bd=2/.02
  self.BT0=np.zeros((n+1,2*n));self.BC0=np.zeros_like(self.BT0)
  self.BT0[np.arange(n),2*np.arange(n)]=1;self.BC0[np.arange(n),2*np.arange(n)+1]=1
  self.I=interp(self.x,self.w,np.linspace(0,1,21)**2)
  self.denseI=interp(self.x,self.w,np.linspace(0,1,8*n+1))
  self.min_boundary_derivative=float('inf');self.max_bc_residual=0.;self.max_newton=0;self.boundary_fallbacks=0
 def surface(self,t,u,env,jac=False):
  R=self.radius(t);self.fac=4/R**2;self.bd=2/R
  Ti,Ci=u.T;et,ec=env(t);Ui=primitive(Ci);anchor=Ci[-1];ua=Ui[-1]
```

110

```python
  gt=self.row@(Ti-et);gu=self.row@(Ui-ua)
  def calc(c):
   k=.12+.20*c/(1+c);kc=.20/(1+c)**2
   den=self.bd*k*self.diag+25
   Ts=et-self.bd*k*gt/den
   Tsc=-self.bd*kc*gt*25/den**2
   A=.00042*np.exp(-3850/(Ts+273.15));At=A*3850/(Ts+273.15)**2
   ux=gu+self.diag*(primitive(c)-ua)
   f=np.exp(-.30/c)
   val=self.bd*A*ux+8e-7*(c-ec)
   deriv=self.bd*(At*Tsc*ux+A*self.diag*f)+8e-7
   return val,deriv,Ts,k,kc,A,At,ux
  c=anchor
  for it in range(25):
   val,deriv,*_=calc(c);step=val/deriv
   while c-step<=0:step*=.5
   c=c-step
   if abs(step)<2e-14*(1+abs(c)):break
  else:
   self.boundary_fallbacks+=1
   lo=min(ec,float(Ci.min()))*.1;hi=max(ec,float(Ci.max()))*1.1
   c=brentq(lambda z:calc(z)[0],lo,hi,xtol=5e-15)
  val,deriv,Ts,k,kc,A,At,ux=calc(c)
  self.max_newton=max(self.max_newton,it+1);self.min_boundary_derivative=min(self.min_boundary_derivative,float(deriv));self.max_bc_residual=max(self.max_bc_residual,float(abs(val)))
  full=np.vstack([u,[Ts,c]])
  if not jac:return full
  tx=self.row@(Ti-Ts)
  B=np.array([[self.bd*k*self.diag+25,self.bd*kc*tx],[self.bd*At*ux,self.bd*A*self.diag*np.exp(-.30/c)+8e-7]])
  Q=np.zeros((2,2*self.n));Q[0,::2]=self.bd*k*self.row;Q[1,1::2]=self.bd*A*self.row*np.exp(-.30/Ci)
  sens=-np.linalg.solve(B,Q);BT=self.BT0.copy();BC=self.BC0.copy();BT[-1]=sens[0];BC[-1]=sens[1]
  return full,BT,BC
 def evaluate(self,t,y,env,jac=False):
  if jac:full,BT,BC=self.surface(t,y.reshape(self.n,2),env,True)
  else:full=self.surface(t,y.reshape(self.n,2),env)
  T,C=full.T
  if np.any(T<=-273.15):raise FloatingPointError('Kelvin<=0')
  rho=760+90*C;cp=1850+2150*C/(1+C);S=rho*cp;Sc=90*cp+rho*2150/(1+C)**2
  k=.12+.20*C/(1+C);kc=.20/(1+C)**2;A=.00042*np.exp(-3850/(T+273.15));At=A*3850/(T+273.15)**2
  U=primitive(C);gt=self.G@(T-T[-1]);gu=self.G@(U-U[-1]);qt=k*gt;qm=A*gu
  ht=qt+self.x*(self.G@(qt-qt[-1]));hm=qm+self.x*(self.G@(qm-qm[-1]))
  fT=self.fac*ht/S;fC=self.fac*hm
  if not jac:return np.column_stack([fT[:-1],fC[:-1]]).ravel()
  BU=np.exp(-.30/C)[:,None]*BC
  gtj=self.G@(BT-BT[-1]);guj=self.G@(BU-BU[-1])
  qtj=k[:,None]*gtj+(kc*gt)[:,None]*BC
  qmj=A[:,None]*guj+(At*gu)[:,None]*BT
  htj=qtj+self.x[:,None]*(self.G@(qtj-qtj[-1]));hmj=qmj+self.x[:,None]*(self.G@(qmj-qmj[-1]))
  J=np.empty((2*self.n,2*self.n));J[::2]=(self.fac*htj/S[:,None]-(fT*Sc/S)[:,None]*BC)[:-1];J[1::2]=(self.fac*hmj)[:-1]
  return J
 def profiles(self,u):
  T,C=u.T;U=primitive(C);v=self.I@U;guess=self.I@C
  return np.column_stack([self.I@T,inverse_primitive(v,guess)])
 def extrema(self,u):
  # Interpolate U, not steep C. All real stationary points of the polynomial.
  U=primitive(u[:,1]);co=dct(U[::-1],type=1)/self.n;co[[0,-1]]*=.5
```

111

```python
  p=Chebyshev(co);roots=p.deriv().roots();roots=roots[np.isreal(roots)].real;roots=roots[(roots>-1)&(roots<1)]
  pts=np.r_[-1,roots,1];vals=p(pts);idx=np.argmax(vals)
  maxi=inverse_primitive(np.array([vals[idx]]),np.array([u[:,1].max()]))[0]
  return {'max_C':float(maxi),'x_max':float((pts[idx]+1)/2),'stationary_points':len(roots),'minimum_U':float(vals.min())}
```

A.34 run_q4

support/project/q4_complete_delivery/v1/source/run_q4.py

```python
"""Cold-start fully coupled Q4 material-coordinate calculation.

The effective equations are S(C) D_s T=div(k grad T), D_s C=div(D grad C).
With v_s=w=(R'/R)r, x=(r/R)^2, radial div=4/R² d_x(x flux_x).
No mass-density dilution term is added to dry-basis C. Rho(C) is only in S.
"""
import argparse,json,time,hashlib,platform
from pathlib import Path
import numpy as np
import scipy
from scipy.integrate import solve_ivp
from numpy.polynomial.chebyshev import chebvander
from q4_common import ROOT,Environment,Radius,read_numeric,load_input,input_audit
from q4_spectral import Reference,interp,primitive,inverse_primitive

def run(n,out,kind='linear',method='Radau',rtol=2e-10,max_step=180.,audit=False):
    out=Path(out)
    if out.with_suffix('.json').exists() or out.with_suffix('.npz').exists():raise FileExistsError(out)
    out.parent.mkdir(parents=True,exist_ok=True)
    data=load_input(ROOT/'inputs/attachment1.xlsx');rd=read_numeric(ROOT/'inputs/attachment2.xlsx',2)[1]
    env=Environment(data);radius=Radius(rd,kind);op=Reference(n,radius)
    weights=np.linalg.solve(chebvander(2*op.x-1,n).T,np.array([0. if j%2 else 1/(1-j*j) for j in range(n+1)]))
    target=np.linspace(0,.02,21);times=[0.];states=[np.tile([28.,2.55],(n+1,1))]
    radii=[.02];fixed=[np.tile([28.,2.55],(21,1))];surf=[[28.,2.55]];meanC=[2.55]
    maxima=[2.55];positions=[0.];rate_checks=[];cumulative_out=0.;balance=[]
    y=np.tile([28.,2.55],n);tic=time.perf_counter();steps=nfev=nlu=0;event_info=None
    cuts=np.unique(np.r_[data[:,0],rd[:,0],np.arange(259200,518401,1800.)]);next_out=60.
    gauss_x,gauss_w=np.polynomial.legendre.leggauss(4)
    def record(t,u,e):
        R=radius(t);inside=target<=R+1e-14;v=np.full((21,2),np.nan)
        I=interp(op.x,op.w,(target[inside]/R)**2)
        v[inside,0]=I@u[:,0];v[inside,1]=inverse_primitive(I@primitive(u[:,1]),I@u[:,1])
        ex=op.extrema(u)
        times.append(float(t));states.append(u);radii.append(R);fixed.append(v);surf.append(u[-1]);meanC.append(float(weights@u[:,1]));maxima.append(ex['max_C']);positions.append(float(R*np.sqrt(ex['x_max'])))
    for a,b in zip(cuts[:-1],cuts[1:]):
        e=env.segment(a,b);fun=lambda t,y:op.evaluate(t,y,e)
        def ev(t,y):return float(np.max(y[1::2])-.15)
        ev.direction=-1;ev.terminal=False
        sol=solve_ivp(fun,(a,b),y,method=method,jac=lambda t,y:op.evaluate(t,y,e,True),
             rtol=rtol,atol=np.tile([rtol*.1,rtol*.01],n),max_step=min(max_step,30.) if a<14400 else max_step,dense_output=True,events=ev)
        if not sol.success:raise RuntimeError(sol.message)
        steps+=len(sol.t)-1;nfev+=sol.nfev;nlu+=sol.nlu;end=b
        if len(sol.t_events[0]):
            root=float(sol.t_events[0][0]);end=.36*np.ceil(root/.36)
            # Exact evaluations on an explicitly stored grid allow the final
```

112

```python
            # execution margin to be selected AFTER comparing numerical runs.
            near=np.unique(np.r_[root+np.array([-2.,-1.,0.,1.,2.]),
                    np.arange(np.floor(root/.36)-3,np.ceil(root/.36)+10)*.36])
            if near.min()<a or near.max()>b:raise RuntimeError('Event neighborhood crosses segment; refine cuts')
            near_full=np.array([op.surface(t,sol.sol(t).reshape(n,2),e) for t in near])
            event_info={'critical_s':root,'critical_h':root/3600,'preliminary_execution_s':end,
                'preliminary_execution_max_C':op.extrema(op.surface(end,sol.sol(end).reshape(n,2),e))['max_C'],
                'critical_slope':float(fun(root,sol.sol(root))[1]),'root_extrema':op.extrema(op.surface(root,sol.sol(root).reshape(n,2),e)),
                'radius_m':radius(root),'length_m':.25,'root_is_1d':True}
        ts=np.arange(next_out,end+1e-9,60.)
        if event_info:ts=np.r_[ts[ts<end-1e-8],end]
        for t in ts:record(t,op.surface(t,sol.sol(t).reshape(n,2),e),e)
        if len(ts):next_out=60*(np.floor(ts[-1]/60)+1)
        # Independent quadrature of the boundary loss; dry mass cancels.
        if audit:
            for sa,sb in zip(sol.t[:-1],sol.t[1:]):
                sb=min(sb,end)
                if sa>=end:break
                tq=(sa+sb)/2+(sb-sa)/2*gauss_x
                rates=[]
                for t in tq:
                    u=op.surface(t,sol.sol(t).reshape(n,2),e)
                    rates.append(2/radius(t)*8e-7*(u[-1,1]-e(t)[1]))
                cumulative_out+=(sb-sa)/2*np.dot(gauss_w,rates)
            u=op.surface(end,sol.sol(end).reshape(n,2),e)
            balance.append([float(end),float(weights@u[:,1]),float(cumulative_out),float(weights@u[:,1]+cumulative_out-2.55)])
        if b%21600==0 or event_info:
            u=op.surface(end,sol.sol(end).reshape(n,2),e)
            f=fun(end,sol.sol(end)).reshape(n,2)
            # Pointwise differentiated polynomial at the boundary supplies rate
            # solely for this spatial quadrature identity, not a PDE unknown.
            T,C=u.T;R=radius(end);S=(760+90*C)*(1850+2150*C/(1+C));k=.12+.20*C/(1+C)
            U=primitive(C);gt=op.G@(T-T[-1]);gu=op.G@(U-U[-1]);qt=k*gt;qm=.00042*np.exp(-3850/(T+273.15))*gu
            ft=4/R**2*(qt+op.x*(op.G@(qt-qt[-1])))/S;fc=4/R**2*(qm+op.x*(op.G@(qm-qm[-1])))
            rate_checks.append({'t_s':end,'water_rate_residual':float(weights@fc+2/R*8e-7*(C[-1]-e(end)[1])),
                'effective_heat_rate_residual':float(weights@(S*ft)+2/R*25*(T[-1]-e(end)[0]))})
            print(json.dumps({'n':n,'kind':kind,'time_h':end/3600,'max_C':float(C.max()),'elapsed_s':time.perf_counter()-tic}),flush=True)
        y=sol.sol(end)
        if event_info:break
    if event_info is None:raise RuntimeError('No threshold by 144h; no fabricated endpoint')
    stats={'n':n,'method':method,'rtol':rtol,'atol_TC':[rtol*.1,rtol*.01],'max_step_s':max_step,'radius_interpolation':kind,
       'future_environment':env.future.tolist(),'event':event_info,'elapsed_s':time.perf_counter()-tic,'steps':steps,'nfev':nfev,'nlu':nlu,
       'max_boundary_residual':op.max_bc_residual,'boundary_fallbacks':op.boundary_fallbacks,'rate_checks':rate_checks,
       'mass_balance_max_abs_in_mean_C':float(np.max(np.abs(np.array(balance)[:,-1]))) if balance else None,
       'numpy':np.__version__,'scipy':scipy.__version__,'python':platform.python_version(),
       'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
    np.savez_compressed(out.with_suffix('.npz'),time_s=times,radius_m=radii,full_TC=states,x=op.x,
         fixed_radius_m=target,sample_TC=fixed,surface_TC=surf,mean_C=meanC,max_C=maxima,max_radius_m=positions,
```

113

```python
         event_time_s=near,event_full_TC=near_full,quadrature_weights=weights,balance=balance)
    out.with_suffix('.json').write_text(json.dumps(stats,ensure_ascii=False,indent=2))
    print(json.dumps(stats,ensure_ascii=False),flush=True)
    return stats

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--n',type=int,default=80);p.add_argument('--out',type=Path,required=True)
    p.add_argument('--kind',choices=['linear','fixed','pchip'],default='linear');p.add_argument('--method',default='Radau')
    p.add_argument('--rtol',type=float,default=2e-10);p.add_argument('--max-step',type=float,default=180.);p.add_argument('--audit',action='store_true')
    a=p.parse_args();run(a.n,a.out,a.kind,a.method,a.rtol,a.max_step,a.audit)
```

A.35 check_numerics

support/project/q4_complete_delivery/v1/source/check_numerics.py

```python
"""Read-only comparisons of real saved runs. No output-state edits."""
import argparse,json,hashlib
from pathlib import Path
from decimal import Decimal,ROUND_HALF_UP
import numpy as np
from q4_common import ROOT
from q4_spectral import interp,primitive,inverse_primitive

def check(base=ROOT):
    base=Path(base);v=base/'validation';z=np.load(base/'output/main.npz');m=json.loads((base/'output/main.json').read_text())
    result={'main':'output/main.npz','comparisons':{},'main_mass_balance_max_mean_C':m['mass_balance_max_abs_in_mean_C'],
       'main_max_boundary_residual':m['max_boundary_residual'],'main_total_loss_per_unit_dry_mass':float(2.55-z['mean_C'][-1]),
       'main_end_mean_C':float(z['mean_C'][-1]),'domain_masks':{}}
    for name in ['spectral80','spectral160','time120_bdf']:
        p=v/(name+'.npz')
        if not p.exists():result['comparisons'][name]={'status':'not complete; excluded'};continue
        q=np.load(p);d=json.loads(p.with_suffix('.json').read_text());common,ia,ib=np.intersect1d(z['time_s'],q['time_s'],return_indices=True)
        err=z['sample_TC'][ia]-q['sample_TC'][ib];finite=np.isfinite(err)
        rnd=lambda x:Decimal(str(float(x))).quantize(Decimal('.0001'),rounding=ROUND_HALF_UP)
        mismatch=sum(rnd(a)!=rnd(b) for a,b in zip(z['sample_TC'][ia][finite],q['sample_TC'][ib][finite]))
        result['comparisons'][name]={'status':'actual saved run read','common_times':len(common),
            'max_abs_T_C':float(np.nanmax(abs(err[:,:,0]))),'max_abs_C':float(np.nanmax(abs(err[:,:,1]))),
            'four_decimal_mismatch_count_TC':int(mismatch),'root_difference_s':d['event']['critical_s']-m['event']['critical_s']}
    g=v/'geometry';p=g/'q4_640x0_integral_rtol2e-11.npz'
    if p.exists():
        f=np.load(p);common,ia,ib=np.intersect1d(z['time_s'],f['time_s'],return_indices=True)
        n=len(z['x'])-1;w=(-1.)**np.arange(n+1);w[[0,-1]]*=.5
        I=interp(z['x'],w,np.linspace(0,1,21)**2);u=z['full_TC'][ia]
        mt=np.einsum('ij,tj->ti',I,u[:,:,0]);mc=inverse_primitive(np.einsum('ij,tj->ti',I,primitive(u[:,:,1])),np.einsum('ij,tj->ti',I,u[:,:,1]))
        result['independent_fv']={'path':str(p.relative_to(base)),'common_times':len(common),'coordinate':'matched material xi=0:.05:1, not fixed physical r',
```

114

```python
            'max_abs_T':float(np.max(abs(mt-f['mid'][ib,:,0]))),'max_abs_C':float(np.max(abs(mc-f['mid'][ib,:,1]))),
            'last_pair_extrapolation_is_estimate':True}
    mask=z['fixed_radius_m'][None,:]>z['radius_m'][:,None]+1e-14
    result['domain_masks']={'outside_count':int(mask.sum()),'outside_are_nan':bool(np.isnan(z['sample_TC'][:,:,1][mask]).all()),
       'inside_are_finite':bool(np.isfinite(z['sample_TC'][:,:,1][~mask]).all())}
    eq=np.isclose(z['fixed_radius_m'][None,:],z['radius_m'][:,None],atol=1e-14,rtol=0);ii,jj=np.where(eq)
    result['surface_fixed_coincidence']={'count':len(ii),'max_abs_C_difference':float(np.max(abs(z['sample_TC'][ii,jj,1]-z['surface_TC'][ii,1])))}
    result['polynomial_maximum']={'max_C_minus_axis_C':float(np.max(abs(z['max_C']-z['full_TC'][:,0,1]))),
       'root_extrema':m['event']['root_extrema'],'early_argmax_note':'nearly uniform profiles have floating-point nonunique locations'}
    (v/'numeric_comparison.json').write_text(json.dumps(result,ensure_ascii=False,indent=2));print(json.dumps(result,ensure_ascii=False,indent=2))
    return result
if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--base',type=Path,default=ROOT);a=p.parse_args();check(a.base)
```

A.36 finalize_q4

support/project/q4_complete_delivery/v1/source/finalize_q4.py

```python
"""Select a strict execution point using computed numerical discrepancies.
All reported cells, tables and trajectories originate from the same main run.
The budget is empirical numerical evidence, not a rigorous error bound.
"""
import argparse,json,math,csv
from pathlib import Path
import numpy as np
from q4_common import ROOT

def finalize(base):
    base=Path(base);out=base/'output';v=base/'validation';g=v/'geometry'
    if (out/'result4_data.json').exists():raise FileExistsError('Refuse overwrite final derived results')
    read=lambda p:json.loads(Path(p).read_text())
    main=read(out/'main.json');root=main['event']['critical_s']
    a=read(v/'spectral80.json')['event']['critical_s']
    b=read(v/'time120_bdf.json')['event']['critical_s']
    f320=read(g/'q4_320x0_integral_rtol2e-10.json')['event_root_s']
    f640=read(g/'q4_640x0_integral_rtol2e-10.json')['event_root_s']
    floose=read(g/'q4_320x0_integral.json')['event_root_s']
    fex=(4*f640-f320)/3
    discrepancies={'spectral80_to_main_s':abs(a-root),'same_n_time_method_change_s':abs(b-root),
        'independent_fv_extrapolation_to_main_s':abs(fex-root),
        'independent_fv_temporal_change_s':abs(f320-floose),
        'independent_fv_last_spatial_correction_s':abs(f640-fex)}
    budget=math.ceil(2000*max(discrepancies.values()))/1000
    execute=.36*math.ceil((root+budget)/.36)
    z=dict(np.load(out/'main.npz'));candidates=z['event_time_s'];idx=int(np.argmin(abs(candidates-execute)))
    if abs(candidates[idx]-execute)>1e-7:raise ValueError('Execution point not actually solved')
    full=z['event_full_TC'][idx]
    # Current implementation records exact candidates around the event. A shifted
    # final endpoint must first be regenerated from that complete state.
```

115

```python
    if abs(z['time_s'][-1]-execute)>1e-7:raise ValueError('Regenerate final row from event full state before export')
    if float(full[:,1].max())>=.15:raise AssertionError('Strict C<.15 failed')
    before=z['event_full_TC'][np.argmin(abs(candidates-(root-1)))]
    after=z['event_full_TC'][np.argmin(abs(candidates-(root+1)))]
    final={'critical_s':root,'critical_h':root/3600,'critical_display_h':round(root/3600,4),
       'execution_s':execute,'execution_h':execute/3600,'execution_max_C':float(full[:,1].max()),
       'actual_margin_s':execute-root,'empirical_budget_s':budget,'budget_rule':'ceil(2*max(recorded discrepancies), 0.001 s)',
       'budget_is_rigorous_bound':False,'discrepancies':discrepancies,'fv_extrapolated_root_s':fex,
       'one_second_before_max_C':float(before[:,1].max()),'one_second_after_max_C':float(after[:,1].max()),
       'radius_m':float(z['radius_m'][-1]),'length_m':.25,'domain':'0<=r<=R(t); -0.125<=z<=0.125 m',
       'source':'output/main.npz; official 1D endpoint, matched 2D applicability and whole-domain evidence are reported separately'}
    (out/'end_event.json').write_text(json.dumps(final,ensure_ascii=False,indent=2))
    keep=z['time_s']>0;Cf=z['sample_TC'][keep,:,1]
    doc={'time_s':z['time_s'][keep].tolist(),'radius_m':z['radius_m'][keep].tolist(),
       'C_fixed':[[None if not np.isfinite(x) else float(x) for x in row] for row in Cf],
       'C_surface':z['surface_TC'][keep,1].tolist()}
    (out/'result4_data.json').write_text(json.dumps(doc,ensure_ascii=False,allow_nan=False))
    with (out/'trajectory_60s_unrounded.csv').open('w',newline='') as f:
        w=csv.writer(f);w.writerow(['time_s','radius_m','max_C','max_radius_m','mean_C','T_surface_C','C_surface']+
          [f'T_r{j/10:.1f}cm_C' for j in range(21)]+[f'C_r{j/10:.1f}cm' for j in range(21)])
        for i,t in enumerate(z['time_s']):
            vals=[t,z['radius_m'][i],z['max_C'][i],z['max_radius_m'][i],z['mean_C'][i],*z['surface_TC'][i],*z['sample_TC'][i,:,0],*z['sample_TC'][i,:,1]]
            w.writerow(['' if not np.isfinite(x) else format(float(x),'.17g') for x in vals])
    rows=[]
    for t in np.r_[np.arange(21600,execute,21600),execute]:
        i=int(np.argmin(abs(z['time_s']-t)))
        if abs(z['time_s'][i]-t)>1e-7:raise ValueError('Missing actual table time')
        rows.append([t/3600,*z['sample_TC'][i,::5,1],z['surface_TC'][i,1],z['radius_m'][i]*100])
    with (out/'table6_unrounded.csv').open('w',newline='') as f:
        w=csv.writer(f);w.writerow(['time_h','r0cm','r0.5cm','r1cm','r1.5cm','r2cm','surface','radius_cm'])
        for row in rows:w.writerow(['' if not np.isfinite(x) else format(float(x),'.17g') for x in row])
    md='|时间/h|0 cm|0.5 cm|1 cm|1.5 cm|2 cm|药材表面|半径/cm|\n|---:|---:|---:|---:|---:|---:|---:|---:|\n'
    for row in rows:md+='|'+'|'.join('—' if not np.isfinite(x) else f'{x:.4f}' for x in row)+'|\n'
    md+='\n“—”表示固定位置已在当前材料域外，末行轴心显示0.1500是四位舍入；未舍入最大值见end_event.json，严格小于0.15。\n'
    (out/'表 6.md').write_text(md)
    print(json.dumps(final,ensure_ascii=False,indent=2));print(md)
    return final
if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--base',type=Path,default=ROOT);a=p.parse_args();finalize(a.base)
```

116

A.37 package_workbooks

support/project/q4_complete_delivery/v1/source/package_workbooks.py

```python
"""Prepare arrays, execute artifact-tool chunks, package their unchanged OOXML.

This module does not author spreadsheet cells: all cells and their final row
addresses are generated by export_workbooks.mjs. Standard-library zipfile only
joins already-authored sheetData rows after checking common styles/strings.
Openpyxl is used exclusively for reading the final file.
"""
from __future__ import annotations
import argparse
import hashlib
import json
import os
from pathlib import Path
import re
import shutil
import subprocess
import time
import threading
import zipfile
import xml.etree.ElementTree as ET
from xml.sax.saxutils import escape
from concurrent.futures import ThreadPoolExecutor
from decimal import Decimal, ROUND_HALF_UP
import numpy as np
import openpyxl


def sha(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for b in iter(lambda: f.read(2**20), b""):
            h.update(b)
    return h.hexdigest()


def rounded4(array):
    a = np.asarray(array, dtype=float)
    scaled = np.abs(a) * 1e4
    out = np.sign(a) * np.floor(scaled + 0.5) / 1e4
    ties = np.abs(scaled - np.floor(scaled) - 0.5) <= 1e-8
    for idx in zip(*np.where(ties)):
        out[idx] = float(Decimal(str(float(a[idx]))).quantize(Decimal("0.0001"), rounding=ROUND_HALF_UP))
    return out


def verify_common_parts(base, part):
    for common in ["xl/styles.xml", "xl/sharedStrings.xml"]:
        if part.read(common)!=base.read(common):
            raise ValueError(f"Chunk metadata differs: {common}")
    # Each export legitimately creates fresh relationship IDs. Keep the first
    # package's workbook and .rels unchanged; compare their resolved meaning.
    def signature(z):
        rel=ET.fromstring(z.read("xl/_rels/workbook.xml.rels"))
        targets={e.attrib["Id"]:e.attrib["Target"] for e in rel}
        book=ET.fromstring(z.read("xl/workbook.xml"))
        main="http://schemas.openxmlformats.org/spreadsheetml/2006/main"
        relation="http://schemas.openxmlformats.org/officeDocument/2006/relationships"
        sheets=[(e.attrib["name"],targets[e.attrib[f"{{{relation}}}id"]]) for e in book.findall(f"{{{main}}}sheets/{{{main}}}sheet")]
```

117

```python
        return sheets,sorted((e.attrib["Type"],e.attrib["Target"]) for e in rel)
    if signature(base)!=signature(part):
        raise ValueError("Chunk workbook/sheet relationship meaning differs")


def combine_chunks(chunks, destination):
    """Transport artifact-authored XML rows verbatim; never calculate a cell."""
    destination = Path(destination)
    temp = destination.with_suffix(".partial.xlsx")
    row_re = re.compile(rb'<x:row\b[^>]*\br="(\d+)"[^>]*(?:/>|>.*?</x:row>)', re.S)
    evidence = {"method": "artifact-tool authoring; verbatim sheetData row packaging", "chunks": [], "sheets": {}}
    with zipfile.ZipFile(chunks[0][2]) as base, zipfile.ZipFile(temp, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=6, allowZip64=True) as final:
        names = base.namelist()
        sheets = [n for n in names if re.fullmatch(r"xl/worksheets/sheet[12]\.xml", n)]
        for name in names:
            if name not in sheets:
                final.writestr(name, base.read(name))
        for sheet in sheets:
            previous = 0
            count = 0
            with final.open(sheet, "w", force_zip64=True) as output:
                base_xml = base.read(sheet)
                before, rest = base_xml.split(b"<x:sheetData>", 1)
                _, after = rest.split(b"</x:sheetData>", 1)
                # artifact-tool currently omits dimension. Reject a future
                # per-chunk dimension rather than silently authoring metadata.
                if b"<x:dimension" in before:
                    raise ValueError("Unexpected per-chunk dimension metadata")
                output.write(before + b"<x:sheetData>")
                for lo, hi, filename in chunks:
                    with zipfile.ZipFile(filename) as part:
                        verify_common_parts(base,part)
                        xml = part.read(sheet)
                    expected = lo + 1
                    for m in row_re.finditer(xml):
                        row = int(m[1])
                        if row == 1 and previous == 0:
                            output.write(m[0]); previous = 1; count += 1
                        elif lo + 1 <= row <= hi:
                            if row != expected or row != previous + 1:
                                raise ValueError(f"Missing/duplicate row {row}; expected {expected}")
                            output.write(m[0]); previous = row; expected += 1; count += 1
                    if expected != hi + 1:
                        raise ValueError(f"Incomplete chunk: {filename}")
                output.write(b"</x:sheetData>" + after)
            evidence["sheets"][sheet] = {"total_rows_including_header": count, "last_row": previous}
    temp.replace(destination)
    for lo, hi, p in chunks:
        evidence["chunks"].append({"source_index_lo": lo, "source_index_hi_exclusive": hi, "sha256": sha(p), "bytes": Path(p).stat().st_size})
    evidence["final_sha256"] = sha(destination)
    return evidence


def validate_q2(path, time_s, profiles, radius_cm):
    report = {"file": str(path), "sha256": sha(path), "rounding": "decimal ROUND_HALF_UP, 4 places; time unrounded", "sheets": {}}
    wb = openpyxl.load_workbook(path, read_only=True, data_only=False)
    if wb.sheetnames != [" 温度", " 水分浓度"]:
        raise ValueError("Template sheet names/order changed")
```

118

```python
    for name, component in [(" 温度", 0), (" 水分浓度", 1)]:
        s = wb[name]
        rows = s.iter_rows(values_only=True)
        header = next(rows)
        if header != tuple([" 时间\\到药材中心的距离", *radius_cm]):
            raise ValueError(f"Unexpected header {header}")
        expected = rounded4(profiles[1:, :, component])
        mismatch = 0; maxdiff = 0.0; n = 0; badtime = 0; badtypes = 0
        for n, row in enumerate(rows, 1):
            if n >= len(time_s):
                raise ValueError("Unexpected trailing row")
            if len(row) != 22:
                raise ValueError(f"Wrong column count at row {n+1}")
            badtime += row[0] != float(time_s[n])
            badtypes += sum(not isinstance(v, (int, float)) for v in row)
            d = np.abs(np.asarray(row[1:], dtype=float) - expected[n-1])
            mismatch += int(np.count_nonzero(d)); maxdiff = max(maxdiff, float(d.max()))
        report["sheets"][name] = {"data_rows": n, "expected_data_rows": len(time_s)-1, "numeric_cells_compared": n*21, "time_mismatches": badtime, "value_mismatches": mismatch, "max_abs_difference_after_rounding": maxdiff, "non_numeric_cells": badtypes, "first_time_s": float(time_s[1]), "last_time_s": float(time_s[-1])}
        if n != len(time_s)-1 or mismatch or badtime or badtypes:
            raise ValueError(json.dumps(report["sheets"][name]))
    wb.close()
    report["passed"] = True
    return report


def validate_q4(path, source):
    indexes=[i for i,t in enumerate(source["time_s"]) if t>0]
    expected_times=np.array([source["time_s"][i] for i in indexes])
    if not np.array_equal(expected_times[:-1],np.arange(60,expected_times[-1],60)):
        raise ValueError("Q4 time grid is not every 60 s plus final time")
    wb=openpyxl.load_workbook(path,read_only=True,data_only=False)
    if wb.sheetnames != ["Sheet1"]:
        raise ValueError("Q4 template sheet name changed")
    it=wb["Sheet1"].iter_rows(max_col=25,values_only=True)
    header=next(it)
    if header[:23]!=tuple(["时间\\到药材中心的距离",*[j/10 for j in range(21)],"药材表面"]):
        raise ValueError("Q4 template core header changed")
    errors=[]; cells=0; blanks=0; coincident=0; radius_error=0.0; n=0
    for n,row in enumerate(it,1):
        if n>len(indexes):
            raise ValueError("Q4 unexpected trailing data row")
        i=indexes[n-1]; radius=source["radius_m"][i]
        if row[0]!=source["time_s"][i]:
            errors.append([n+1,"time"])
        want=source["C_fixed"][i]+[source["C_surface"][i]]
        expected=[None if x is None else float(Decimal(str(x)).quantize(Decimal('0.0001'),rounding=ROUND_HALF_UP)) for x in want]
        if list(row[1:23])!=expected:
            errors.append([n+1,"value"])
        for j in range(21):
            outside=j/1000>radius+1e-12
            if (row[j+1] is None)!=outside:
                errors.append([n+1,j+2,"domain_mask"])
            if outside: blanks+=1
            else: cells+=1
            if abs(j/1000-radius)<=1e-12:
                coincident+=1
                if row[j+1]!=row[22]: errors.append([n+1,j+2,"surface_coincidence"])
        cells+=1
```

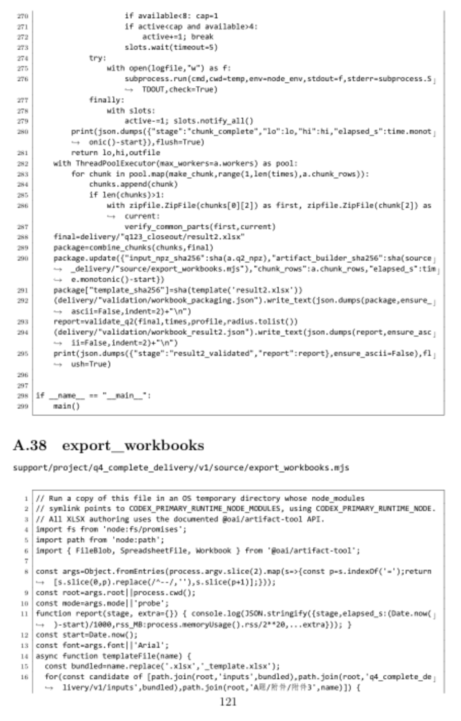
119

```python
        if row[23] is not None: errors.append([n+1,"spacer"])
        radius_error=max(radius_error,abs(row[24]-radius*100))
    wb.close()
    report={"file":str(path),"sha256":sha(path),"rounding":"decimal ROUND_HALF_UP, 4 places; time unrounded","data_rows":n,"expected_data_rows":len(indexes),"numeric_moisture_cells_compared":cells,"outside_domain_blank_cells":blanks,"surface_coincidences_checked":coincident,"time_grid":"60 s, 120 s, ... plus actual final time","first_time_s":float(expected_times[0]),"last_time_s":float(expected_times[-1]),"radius_metadata_max_abs_error_cm":radius_error,"errors":errors[:100],"passed":n==len(indexes) and not errors and radius_error<1e-12}
    if not report["passed"]: raise ValueError(json.dumps(report))
    return report


def main():
    p=argparse.ArgumentParser()
    p.add_argument("--root", type=Path, default=Path(__file__).resolve().parents[1])
    p.add_argument("--q2-npz", type=Path)
    p.add_argument("--q4-json", type=Path)
    p.add_argument("--out-dir",type=Path,help="New delivery directory for a cold-start rebuild; refuses to overwrite an existing formal XLSX")
    p.add_argument("--chunk-rows", type=int, default=10000)
    p.add_argument("--font", default="Arial")
    p.add_argument("--resume", action="store_true")
    p.add_argument("--workers",type=int,choices=[1,2],default=2)
    a=p.parse_args()
    root=a.root.resolve(); source_delivery=root if (root/"source/export_workbooks.mjs").exists() else root/"q4_complete_delivery/v1"
    delivery=a.out_dir.resolve() if a.out_dir else source_delivery
    for relative in ["output","validation","q123_closeout"]:
        (delivery/relative).mkdir(parents=True,exist_ok=True)
    targets=[]
    if a.q4_json: targets.append(delivery/"output/result4.xlsx")
    if a.q2_npz: targets.append(delivery/"q123_closeout/result2.xlsx")
    for target in targets:
        if target.exists(): raise FileExistsError(f"Refusing to overwrite {target}; choose a new --out-dir")
    temp=Path(os.environ.get("DRYING_WORKBOOK_TEMP", str(Path.cwd()/"workbook_temp"))); temp.mkdir(parents=True,exist_ok=True)
    node=os.environ["CODEX_PRIMARY_RUNTIME_NODE"]
    modules=Path(os.environ["CODEX_PRIMARY_RUNTIME_NODE_MODULES"])
    if not (temp/"node_modules").exists():
        (temp/"node_modules").symlink_to(modules, target_is_directory=True)
    shutil.copyfile(source_delivery/"source/export_workbooks.mjs", temp/"export_workbooks.mjs")
    font_dir=source_delivery/"inputs/fonts"
    node_env=os.environ.copy()
    if font_dir.exists():
        config=temp/"fonts.conf"
        config.write_text(f'<?xml version="1.0"?><!DOCTYPE fontconfig SYSTEM "urn:fontconfig:fonts.dtd"><fontconfig><include ignore_missing="yes">/etc/fonts/fonts.conf</include><dir>{escape(str(font_dir))}</dir><cachedir>{escape(str(temp/"font-cache"))}</cachedir></fontconfig>')
        node_env["FONTCONFIG_FILE"]=str(config)
    def template(name):
        supplied=source_delivery/"inputs"/name.replace('.xlsx','_template.xlsx')
        return supplied if supplied.exists() else root/"A 题/附件/附件 3"/name
    if a.q4_json:
        previewdir=delivery/"validation/workbook_previews"; previewdir.mkdir(exist_ok=True)
        final=delivery/"output/result4.xlsx"
        with open(delivery/"validation/workbook_result4_build.log","w") as f:
```

120

```python
            subprocess.run([node,"--max-old-space-size=2048",str(temp/"export_workbooks.mjs"),"--mode=q4",f"--root={root}",f"--input={a.q4_json.resolve()}",f"--out={final}",f"--font={a.font}",f"--previewDir={previewdir}"],cwd=temp,env=node_env,stdout=f,stderr=subprocess.STDOUT,check=True)
        source=json.loads(a.q4_json.read_text())
        report=validate_q4(final,source)
        report["input_json_sha256"]=sha(a.q4_json)
        report["template_sha256"]=sha(template('result4.xlsx'))
        sidecar=Path(str(final)+".inspect.ndjson")
        if sidecar.exists(): sidecar.replace(delivery/"validation/workbook_result4_inspect.ndjson")
        (delivery/"validation/workbook_result4.json").write_text(json.dumps(report,ensure_ascii=False,indent=2)+"\n")
        print(json.dumps({"stage":"result4_validated","report":report},ensure_ascii=False),flush=True)
        if not a.q2_npz: return
    if not a.q2_npz:
        p.error("--q2-npz or --q4-json is required")
    with np.load(a.q2_npz) as data:
        times=np.asarray(data["time_s"], dtype="<f8")
        profile=np.asarray(data["profile_TC"], dtype="<f8")
        radius=np.asarray(data["radius_cm"], dtype=float)
    assert profile.shape == (len(times),21,2)
    assert times[0]==0 and times[-1]==206906.76 and np.array_equal(times[1:-1], np.arange(1,206907))
    source_hash=sha(a.q2_npz); builder_hash=sha(source_delivery/"source/export_workbooks.mjs")
    metadata=temp/"q2_meta.json"
    if a.resume and metadata.exists():
        previous=json.loads(metadata.read_text())
        if previous.get("source_sha256")!=source_hash or previous.get("builder_sha256")!=builder_hash:
            raise ValueError("Resume cache belongs to different inputs or builder; omit --resume for a fresh rebuild")
    times.tofile(temp/"q2_time.f64"); profile.tofile(temp/"q2_profile.f64")
    metadata.write_text(json.dumps({"shape":profile.shape,"radius_cm":radius.tolist(),"source_sha256":source_hash,"builder_sha256":builder_hash}))
    chunks=[]; start=time.monotonic()
    previewdir=delivery/"validation/workbook_previews"; previewdir.mkdir(exist_ok=True)
    logdir=delivery/"validation/workbook_chunks"; logdir.mkdir(exist_ok=True)
    slots=threading.Condition(); active=0
    def make_chunk(lo):
        nonlocal active
        hi=min(lo+a.chunk_rows,len(times)); outfile=temp/f"q2_chunk_{lo}_{hi}.xlsx"
        logfile=logdir/f"q2_chunk_{lo}_{hi}.log"
        reusable=False
        if a.resume and outfile.exists():
            try:
                with zipfile.ZipFile(outfile) as z:
                    reusable=z.testzip() is None
            except zipfile.BadZipFile:
                reusable=False
        if not reusable:
            cmd=[node,"--max-old-space-size=4096",str(temp/"export_workbooks.mjs"),"--mode=q2chunk",f"--root={root}",f"--inputDir={temp}",f"--lo={lo}",f"--hi={hi}",f"--out={outfile}",f"--font={a.font}",f"--previewDir={previewdir}",f"--preview={'true' if lo==1 else 'false'}"]
            with slots:
                while True:
                    control=temp/"q2_workers.txt"
                    cap=min(a.workers,int(control.read_text().strip())) if control.exists() else a.workers
                    available=int(re.search(r"MemAvailable:\s+(\d+)",Path('/proc/meminfo').read_text())[1])/1024**2
```

121



```
                if available<8: cap=1
                if active<cap and available>4:
                    active+=1; break
                slots.wait(timeout=5)
            try:
                with open(logfile,"w") as f:
                    subprocess.run(cmd,cwd=temp,env=node_env,stdout=f,stderr=subprocess.S
                    ↪  TDOUT,check=True)
            finally:
                with slots:
                    active-=1; slots.notify_all()
        print(json.dumps({"stage":"chunk_complete","lo":lo,"hi":hi,"elapsed_s":time.monot
        ↪  onic()-start}),flush=True)
        return lo,hi,outfile
    with ThreadPoolExecutor(max_workers=a.workers) as pool:
        for chunk in pool.map(make_chunk,range(1,len(times),a.chunk_rows)):
            chunks.append(chunk)
            if len(chunks)>1:
                with zipfile.ZipFile(chunks[0][2]) as first, zipfile.ZipFile(chunk[2]) as
                ↪  current:
                    verify_common_parts(first,current)
    final=delivery/"q123_closeout/result2.xlsx"
    package=combine_chunks(chunks,final)
    package.update({"input_npz_sha256":sha(a.q2_npz),"artifact_builder_sha256":sha(source
    ↪  _delivery/"source/export_workbooks.mjs"),"chunk_rows":a.chunk_rows,"elapsed_s":tim
    ↪  e.monotonic()-start})
    package["template_sha256"]=sha(template('result2.xlsx'))
    (delivery/"validation/workbook_packaging.json").write_text(json.dumps(package,ensure_
    ↪  ascii=False,indent=2)+"\n")
    report=validate_q2(final,times,profile,radius.tolist())
    (delivery/"validation/workbook_result2.json").write_text(json.dumps(report,ensure_asc
    ↪  ii=False,indent=2)+"\n")
    print(json.dumps({"stage":"result2_validated","report":report},ensure_ascii=False),fl
    ↪  ush=True)


if __name__ == "__main__":
    main()
```

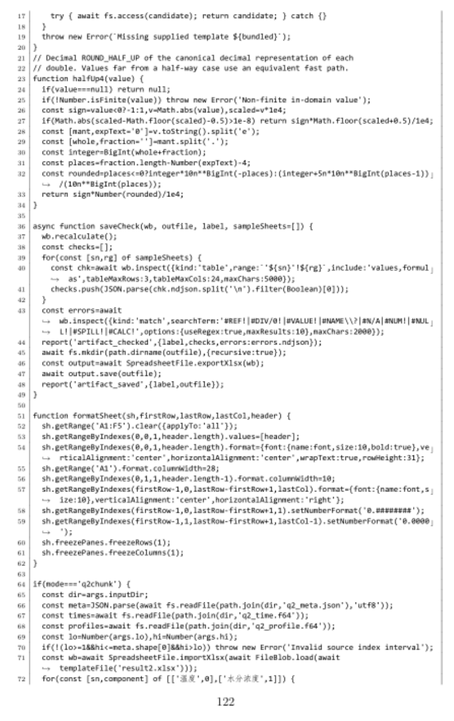
A.38 export_workbooks

support/project/q4_complete_delivery/v1/source/export_workbooks.mjs

```
// Run a copy of this file in an OS temporary directory whose node_modules
// symlink points to CODEX_PRIMARY_RUNTIME_NODE_MODULES, using CODEX_PRIMARY_RUNTIME_NODE.
// All XLSX authoring uses the documented @oai/artifact-tool API.
import fs from 'node:fs/promises';
import path from 'node:path';
import { FileBlob, SpreadsheetFile, Workbook } from '@oai/artifact-tool';

const args=Object.fromEntries(process.argv.slice(2).map(s=>{const p=s.indexOf('=');return
↪  [s.slice(0,p).replace(/^--/,''),s.slice(p+1)];}));
const root=args.root||process.cwd();
const mode=args.mode||'probe';
function report(stage, extra={}) { console.log(JSON.stringify({stage,elapsed_s:(Date.now(
↪  )-start)/1000,rss_MB:process.memoryUsage().rss/2**20,...extra})); }
const start=Date.now();
const font=args.font||'Arial';
async function templateFile(name) {
  const bundled=name.replace('.xlsx','_template.xlsx');
  for(const candidate of [path.join(root,'inputs',bundled),path.join(root,'q4_complete_de
  ↪  livery/v1/inputs',bundled),path.join(root,'A题/附件/附件3',name)]) {
```

121

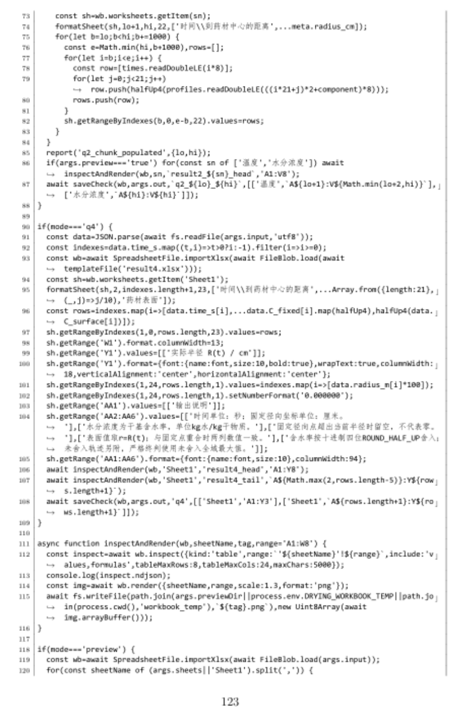
122



```
    try { await fs.access(candidate); return candidate; } catch {}
  }
  throw new Error(`Missing supplied template ${bundled}`);
}
// Decimal ROUND_HALF_UP of the canonical decimal representation of each
// double. Values far from a half-way case use an equivalent fast path.
function halfUp4(value) {
  if(value===null) return null;
  if(!Number.isFinite(value)) throw new Error('Non-finite in-domain value');
  const sign=value<0?-1:1,v=Math.abs(value),scaled=v*1e4;
  if(Math.abs(scaled-Math.floor(scaled)-0.5)>1e-8) return sign*Math.floor(scaled+0.5)/1e4;
  const [mant,expText='0']=v.toString().split('e');
  const [whole,fraction='']=mant.split('.');
  const integer=BigInt(whole+fraction);
  const places=fraction.length-Number(expText)-4;
  const rounded=places<=0?integer*10n**BigInt(-places):(integer+5n*10n**BigInt(places-1))
  ↪  /(10n**BigInt(places));
  return sign*Number(rounded)/1e4;
}

async function saveCheck(wb, outfile, label, sampleSheets=[]) {
  wb.recalculate();
  const checks=[];
  for(const [sn,rg] of sampleSheets) {
    const chk=await wb.inspect({kind:'table',range:`'${sn}'!${rg}`,include:'values,formul
    ↪  as',tableMaxRows:3,tableMaxCols:24,maxChars:5000});
    checks.push(JSON.parse(chk.ndjson.split('\n').filter(Boolean)[0]));
  }
  const errors=await
  ↪  wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#N/A|#NUM!|#NUL
  ↪  L!|#SPILL!|#CALC!',options:{useRegex:true,maxResults:10},maxChars:2000});
  report('artifact_checked',{label,checks,errors:errors.ndjson});
  await fs.mkdir(path.dirname(outfile),{recursive:true});
  const output=await SpreadsheetFile.exportXlsx(wb);
  await output.save(outfile);
  report('artifact_saved',{label,outfile});
}

function formatSheet(sh,firstRow,lastRow,lastCol,header) {
  sh.getRange('A1:F5').clear({applyTo:'all'});
  sh.getRangeByIndexes(0,0,1,header.length).values=[header];
  sh.getRangeByIndexes(0,0,1,header.length).format={font:{name:font,size:10,bold:true},ve
  ↪  rticalAlignment:'center',horizontalAlignment:'center',wrapText:true,rowHeight:31};
  sh.getRange('A1').format.columnWidth=28;
  sh.getRangeByIndexes(0,1,1,header.length-1).format.columnWidth=10;
  sh.getRangeByIndexes(firstRow-1,0,lastRow-firstRow+1,lastCol).format={font:{name:font,s
  ↪  ize:10},verticalAlignment:'center',horizontalAlignment:'right'};
  sh.getRangeByIndexes(firstRow-1,0,lastRow-firstRow+1,1).setNumberFormat('0.########');
  sh.getRangeByIndexes(firstRow-1,1,lastRow-firstRow+1,lastCol-1).setNumberFormat('0.0000
  ↪  ');
  sh.freezePanes.freezeRows(1);
  sh.freezePanes.freezeColumns(1);
}

if(mode==='q2chunk') {
  const dir=args.inputDir;
  const meta=JSON.parse(await fs.readFile(path.join(dir,'q2_meta.json'),'utf8'));
  const times=await fs.readFile(path.join(dir,'q2_time.f64'));
  const profiles=await fs.readFile(path.join(dir,'q2_profile.f64'));
  const lo=Number(args.lo),hi=Number(args.hi);
  if(!(lo>=1&&hi<=meta.shape[0]&&hi>lo)) throw new Error('Invalid source index interval');
  const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(await
  ↪  templateFile('result2.xlsx')));
  for(const [sn,component] of [['温度',0],['水分浓度',1]]) {
```

122

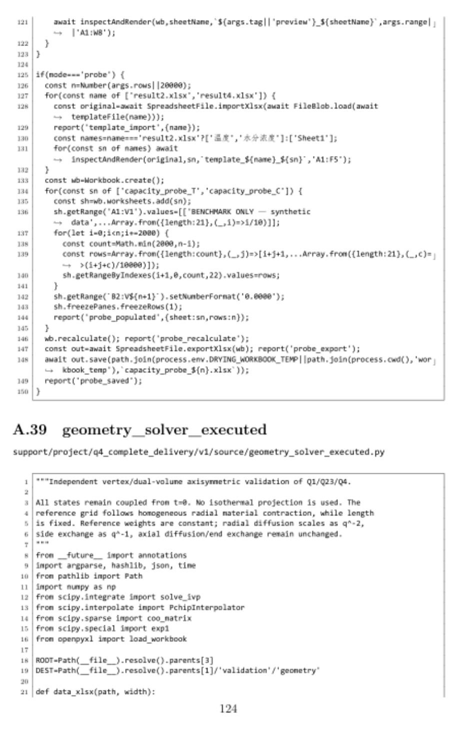
123



```
    const sh=wb.worksheets.getItem(sn);
    formatSheet(sh,lo+1,hi,22,['时间\\到药材中心的距离',...meta.radius_cm]);
    for(let b=lo;b<hi;b+=1000) {
      const e=Math.min(hi,b+1000),rows=[];
      for(let i=b;i<e;i++) {
        const row=[times.readDoubleLE(i*8)];
        for(let j=0;j<21;j++)
        ↪  row.push(halfUp4(profiles.readDoubleLE(((i*21+j)*2+component)*8)));
        rows.push(row);
      }
      sh.getRangeByIndexes(b,0,e-b,22).values=rows;
    }
  }
  report('q2_chunk_populated',{lo,hi});
  if(args.preview==='true') for(const sn of ['温度','水分浓度']) await
  ↪  inspectAndRender(wb,sn,`result2_${sn}_head`,'A1:V8');
  await saveCheck(wb,args.out,`q2_${lo}_${hi}`,[['温度',`A${lo+1}:V${Math.min(lo+2,hi)}`],
  ↪  ['水分浓度',`A${hi}:V${hi}`]]);
}

if(mode==='q4') {
  const data=JSON.parse(await fs.readFile(args.input,'utf8'));
  const indexes=data.time_s.map((t,i)=>t>0?i:-1).filter(i=>i>=0);
  const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(await
  ↪  templateFile('result4.xlsx')));
  const sh=wb.worksheets.getItem('Sheet1');
  formatSheet(sh,2,indexes.length+1,23,['时间\\到药材中心的距离',...Array.from({length:21},
  ↪  (_,j)=>j/10),'药材表面']);
  const rows=indexes.map(i=>[data.time_s[i],...data.C_fixed[i].map(halfUp4),halfUp4(data.
  ↪  C_surface[i])]);
  sh.getRangeByIndexes(1,0,rows.length,23).values=rows;
  sh.getRange('W1').format.columnWidth=13;
  sh.getRange('Y1').values=[['实际半径 R(t) / cm']];
  sh.getRange('Y1').format={font:{name:font,size:10,bold:true},wrapText:true,columnWidth:
  ↪  18,verticalAlignment:'center',horizontalAlignment:'center'};
  sh.getRangeByIndexes(1,24,rows.length,1).values=indexes.map(i=>[data.radius_m[i]*100]);
  sh.getRangeByIndexes(1,24,rows.length,1).setNumberFormat('0.000000');
  sh.getRange('AA1').values=[['输出说明']];
  sh.getRange('AA2:AA6').values=[['时间单位：秒；固定径向坐标单位：厘米。
  ↪  '],['水分浓度为干基含水率，单位kg水/kg干物质。'],['固定径向点超出当前半径时留空，不代表零。
  ↪  '],['表面值取r=R(t)；与固定点重合时两列数值一致。'],['含水率按十进制四位ROUND_HALF_UP舍入；
  ↪  未舍入轨迹另附，严格终判使用未舍入全域最大值。']];
  sh.getRange('AA1:AA6').format={font:{name:font,size:10},columnWidth:94};
  await inspectAndRender(wb,'Sheet1','result4_head','A1:Y8');
  await inspectAndRender(wb,'Sheet1','result4_tail',`A${Math.max(2,rows.length-5)}:Y${row
  ↪  s.length+1}`);
  await saveCheck(wb,args.out,'q4',[['Sheet1','A1:Y3'],['Sheet1',`A${rows.length+1}:Y${ro
  ↪  ws.length+1}`]]);
}

async function inspectAndRender(wb,sheetName,tag,range='A1:W8') {
  const inspect=await wb.inspect({kind:'table',range:`'${sheetName}'!${range}`,include:'v
  ↪  alues,formulas',tableMaxRows:8,tableMaxCols:24,maxChars:5000});
  console.log(inspect.ndjson);
  const img=await wb.render({sheetName,range,scale:1.3,format:'png'});
  await fs.writeFile(path.join(args.previewDir||process.env.DRYING_WORKBOOK_TEMP||path.jo
  ↪  in(process.cwd(),'workbook_temp'),`${tag}.png`),new Uint8Array(await
  ↪  img.arrayBuffer()));
}

if(mode==='preview') {
  const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(args.input));
  for(const sheetName of (args.sheets||'Sheet1').split(',')) {
```

123

124



```
    await inspectAndRender(wb,sheetName,`${args.tag||'preview'}_${sheetName}`,args.range|
    ↪  |'A1:W8');
  }
}

if(mode==='probe') {
  const n=Number(args.rows||20000);
  for(const name of ['result2.xlsx','result4.xlsx']) {
    const original=await SpreadsheetFile.importXlsx(await FileBlob.load(await
    ↪  templateFile(name)));
    report('template_import',{name});
    const names=name==='result2.xlsx'?['温度','水分浓度']:['Sheet1'];
    for(const sn of names) await
    ↪  inspectAndRender(original,sn,`template_${name}_${sn}`,'A1:F5');
  }
  const wb=Workbook.create();
  for(const sn of ['capacity_probe_T','capacity_probe_C']) {
    const sh=wb.worksheets.add(sn);
    sh.getRange('A1:V1').values=[['BENCHMARK ONLY — synthetic
    ↪  data',...Array.from({length:21},(_,i)=>i/10)]];
    for(let i=0;i<n;i+=2000) {
      const count=Math.min(2000,n-i);
      const rows=Array.from({length:count},(_,j)=>[i+j+1,...Array.from({length:21},(_,c)=
      ↪  >(i+j+c)/10000)]);
      sh.getRangeByIndexes(i+1,0,count,22).values=rows;
    }
    sh.getRange(`B2:V${n+1}`).setNumberFormat('0.0000');
    sh.freezePanes.freezeRows(1);
    report('probe_populated',{sheet:sn,rows:n});
  }
  wb.recalculate(); report('probe_recalculate');
  const out=await SpreadsheetFile.exportXlsx(wb); report('probe_export');
  await out.save(path.join(process.env.DRYING_WORKBOOK_TEMP||path.join(process.cwd(),'wor
  ↪  kbook_temp'),`capacity_probe_${n}.xlsx`));
  report('probe_saved');
}
```

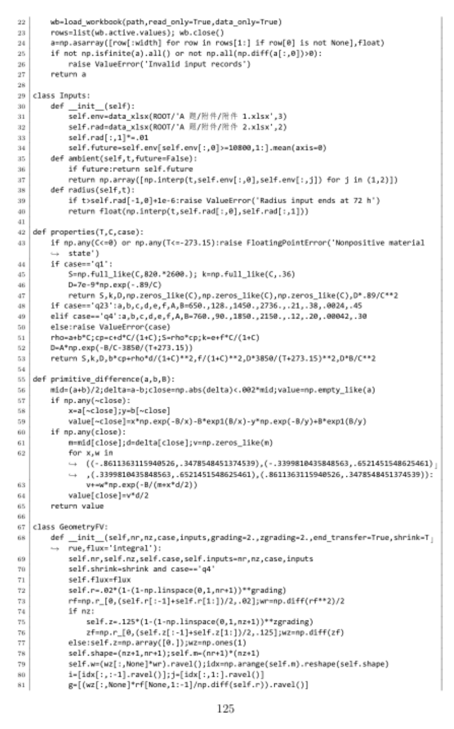
A.39 geometry_solver_executed

support/project/q4_complete_delivery/v1/source/geometry_solver_executed.py

```
"""Independent vertex/dual-volume axisymmetric validation of Q1/Q23/Q4.

All states remain coupled from t=0. No isothermal projection is used. The
reference grid follows homogeneous radial material contraction, while length
is fixed. Reference weights are constant; radial diffusion scales as q^-2,
side exchange as q^-1, axial diffusion/end exchange remain unchanged.
"""
from __future__ import annotations
import argparse, hashlib, json, time
from pathlib import Path
import numpy as np
from scipy.integrate import solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.sparse import coo_matrix
from scipy.special import exp1
from openpyxl import load_workbook

ROOT=Path(__file__).resolve().parents[3]
DEST=Path(__file__).resolve().parents[1]/'validation'/'geometry'

def data_xlsx(path, width):
```

124

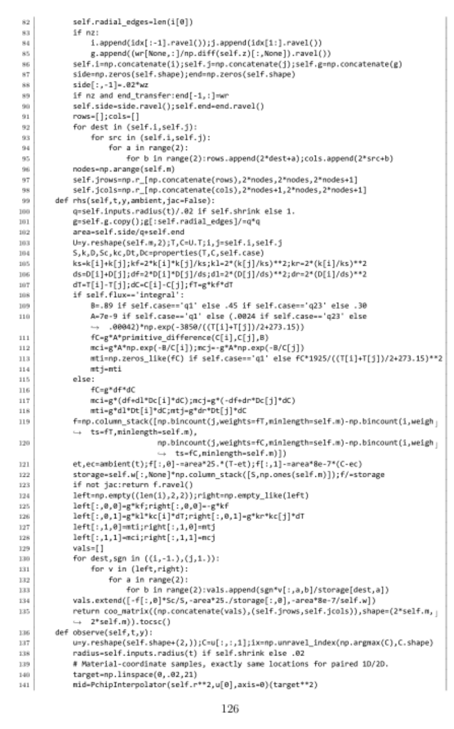
125



```
    wb=load_workbook(path,read_only=True,data_only=True)
    rows=list(wb.active.values); wb.close()
    a=np.asarray([row[:width] for row in rows[1:] if row[0] is not None],float)
    if not np.isfinite(a).all() or not np.all(np.diff(a[:,0])>0):
        raise ValueError('Invalid input records')
    return a

class Inputs:
    def __init__(self):
        self.env=data_xlsx(ROOT/'A 题/附件/附件 1.xlsx',3)
        self.rad=data_xlsx(ROOT/'A 题/附件/附件 2.xlsx',2)
        self.rad[:,1]*=.01
        self.future=self.env[self.env[:,0]>=10800,1:].mean(axis=0)
    def ambient(self,t,future=False):
        if future:return self.future
        return np.array([np.interp(t,self.env[:,0],self.env[:,j]) for j in (1,2)])
    def radius(self,t):
        if t>self.rad[-1,0]+1e-6:raise ValueError('Radius input ends at 72 h')
        return float(np.interp(t,self.rad[:,0],self.rad[:,1]))

def properties(T,C,case):
    if np.any(C<=0) or np.any(T<=-273.15):raise FloatingPointError('Nonpositive material
    ↪  state')
    if case=='q1':
        S=np.full_like(C,820.*2600.); k=np.full_like(C,.36)
        D=7e-9*np.exp(-.89/C)
        return S,k,D,np.zeros_like(C),np.zeros_like(C),np.zeros_like(C),D*.89/C**2
    if case=='q23':a,b,c,d,e,f,A,B=650.,128.,1450.,2736.,.21,.38,.0024,.45
    elif case=='q4':a,b,c,d,e,f,A,B=760.,90.,1850.,2150.,.12,.20,.00042,.30
    else:raise ValueError(case)
    rho=a+b*C;cp=c+d*C/(1+C);S=rho*cp;k=e+f*C/(1+C)
    D=A*np.exp(-B/C-3850/(T+273.15))
    return S,k,D,b*cp+rho*d/(1+C)**2,f/(1+C)**2,D*3850/(T+273.15)**2,D*B/C**2

def primitive_difference(a,b,B):
    mid=(a+b)/2;delta=a-b;close=np.abs(delta)<.002*mid;value=np.empty_like(a)
    if np.any(~close):
        x=a[~close];y=b[~close]
        value[~close]=x*np.exp(-B/x)-B*exp1(B/x)-y*np.exp(-B/y)+B*exp1(B/y)
    if np.any(close):
        m=mid[close];d=delta[close];v=np.zeros_like(m)
        for x,w in
        ↪  ((-.8611363115940526,.3478548451374539),(-.3399810435848563,.6521451548625461)
        ↪  ,(.3399810435848563,.6521451548625461),(.8611363115940526,.3478548451374539)):
            v+=w*np.exp(-B/(m+x*d/2))
        value[close]=v*d/2
    return value

class GeometryFV:
    def __init__(self,nr,nz,case,inputs,grading=2.,zgrading=2.,end_transfer=True,shrink=T
    ↪  rue,flux='integral'):
        self.nr,self.nz,self.case,self.inputs=nr,nz,case,inputs
        self.shrink=shrink and case=='q4'
        self.flux=flux
        self.r=.02*(1-(1-np.linspace(0,1,nr+1))**grading)
        rf=np.r_[0,(self.r[:-1]+self.r[1:])/2,.02];wr=np.diff(rf**2)/2
        if nz:
            self.z=.125*(1-(1-np.linspace(0,1,nz+1))**zgrading)
            zf=np.r_[0,(self.z[:-1]+self.z[1:])/2,.125];wz=np.diff(zf)
        else:self.z=np.array([0.]);wz=np.ones(1)
        self.shape=(nz+1,nr+1);self.m=(nr+1)*(nz+1)
        self.w=(wz[:,None]*wr).ravel();idx=np.arange(self.m).reshape(self.shape)
        i=[idx[:,:-1].ravel()];j=[idx[:,1:].ravel()]
        g=[(wz[:,None]*rf[None,1:-1]/np.diff(self.r)).ravel()]
```

125

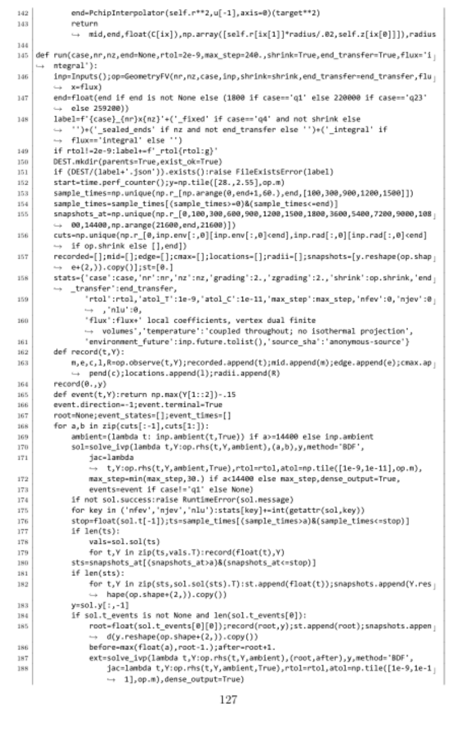
126



```
        self.radial_edges=len(i[0])
        if nz:
            i.append(idx[:-1].ravel());j.append(idx[1:].ravel())
            g.append((wr[None,:]/np.diff(self.z)[:,None]).ravel())
        self.i=np.concatenate(i);self.j=np.concatenate(j);self.g=np.concatenate(g)
        side=np.zeros(self.shape);end=np.zeros(self.shape)
        side[:,-1]=.02*wz
        if nz and end_transfer:end[-1,:]=wr
        self.side=side.ravel();self.end=end.ravel()
        rows=[];cols=[]
        for dest in (self.i,self.j):
            for src in (self.i,self.j):
                for a in range(2):
                    for b in range(2):rows.append(2*dest+a);cols.append(2*src+b)
        nodes=np.arange(self.m)
        self.jrows=np.r_[np.concatenate(rows),2*nodes,2*nodes,2*nodes+1]
        self.jcols=np.r_[np.concatenate(cols),2*nodes+1,2*nodes,2*nodes+1]
    def rhs(self,t,y,ambient,jac=False):
        q=self.inputs.radius(t)/.02 if self.shrink else 1.
        g=self.g.copy();g[:self.radial_edges]/=q*q
        area=self.side/q+self.end
        U=y.reshape(self.m,2);T,C=U.T;i,j=self.i,self.j
        S,k,D,Sc,kc,Dt,Dc=properties(T,C,self.case)
        ks=k[i]+k[j];kf=2*k[i]*k[j]/ks;kl=2*(k[j]/ks)**2;kr=2*(k[i]/ks)**2
        ds=D[i]+D[j];df=2*D[i]*D[j]/ds;dl=2*(D[j]/ds)**2;dr=2*(D[i]/ds)**2
        dT=T[i]-T[j];dC=C[i]-C[j];fT=g*kf*dT
        if self.flux=='integral':
            B=.89 if self.case=='q1' else .45 if self.case=='q23' else .30
            A=7e-9 if self.case=='q1' else (.0024 if self.case=='q23' else
            ↪  .00042)*np.exp(-3850/((T[i]+T[j])/2+273.15))
            fC=g*A*primitive_difference(C[i],C[j],B)
            mci=g*A*np.exp(-B/C[i]);mcj=-g*A*np.exp(-B/C[j])
            mti=np.zeros_like(fC) if self.case=='q1' else fC*1925/((T[i]+T[j])/2+273.15)**2
            mtj=mti
        else:
            fC=g*df*dC
            mci=g*(df+dl*Dc[i]*dC);mcj=g*(-df+dr*Dc[j]*dC)
            mti=g*dl*Dt[i]*dC;mtj=g*dr*Dt[j]*dC
        f=np.column_stack([np.bincount(j,weights=fT,minlength=self.m)-np.bincount(i,weigh
        ↪  ts=fT,minlength=self.m),
                           np.bincount(j,weights=fC,minlength=self.m)-np.bincount(i,weigh
                           ↪  ts=fC,minlength=self.m)])
        et,ec=ambient(t);f[:,0]-=area*25.*(T-et);f[:,1]-=area*8e-7*(C-ec)
        storage=self.w[:,None]*np.column_stack([S,np.ones(self.m)]);f/=storage
        if not jac:return f.ravel()
        left=np.empty((len(i),2,2));right=np.empty_like(left)
        left[:,0,0]=g*kf;right[:,0,0]=-g*kf
        left[:,0,1]=g*kl*kc[i]*dT;right[:,0,1]=g*kr*kc[j]*dT
        left[:,1,0]=mti;right[:,1,0]=mtj
        left[:,1,1]=mci;right[:,1,1]=mcj
        vals=[]
        for dest,sgn in ((i,-1.),(j,1.)):
            for v in (left,right):
                for a in range(2):
                    for b in range(2):vals.append(sgn*v[:,a,b]/storage[dest,a])
        vals.extend([-f[:,0]*Sc/S,-area*25./storage[:,0],-area*8e-7/self.w])
        return coo_matrix((np.concatenate(vals),(self.jrows,self.jcols)),shape=(2*self.m,
        ↪  2*self.m)).tocsc()
    def observe(self,t,y):
        u=y.reshape(self.shape+(2,));C=u[:,:,1];ix=np.unravel_index(np.argmax(C),C.shape)
        radius=self.inputs.radius(t) if self.shrink else .02
        # Material-coordinate samples, exactly same locations for paired 1D/2D.
        target=np.linspace(0,.02,21)
        mid=PchipInterpolator(self.r**2,u[0],axis=0)(target**2)
```

126

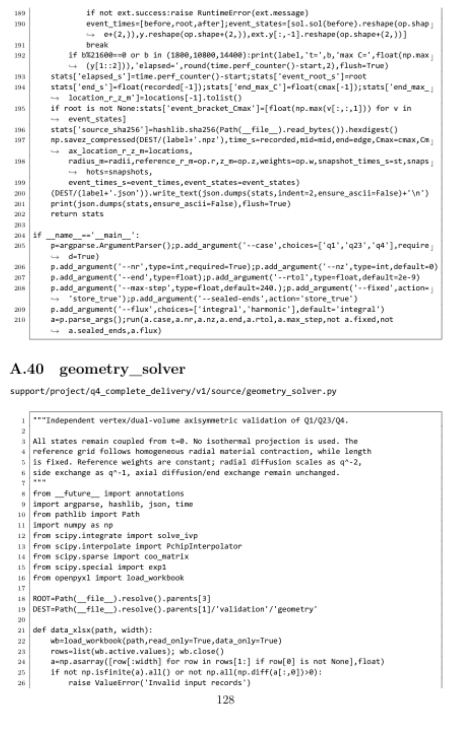
127



```
        end=PchipInterpolator(self.r**2,u[-1],axis=0)(target**2)
        return
        ↪  mid,end,float(C[ix]),np.array([self.r[ix[1]]*radius/.02,self.z[ix[0]]]),radius

def run(case,nr,nz,end=None,rtol=2e-9,max_step=240.,shrink=True,end_transfer=True,flux='i
↪  ntegral'):
    inp=Inputs();op=GeometryFV(nr,nz,case,inp,shrink=shrink,end_transfer=end_transfer,flu
    ↪  x=flux)
    end=float(end if end is not None else (1800 if case=='q1' else 220000 if case=='q23'
    ↪  else 259200))
    label=f'{case}_{nr}x{nz}'+('_fixed' if case=='q4' and not shrink else
    ↪  '')+('_sealed_ends' if nz and not end_transfer else '')+('_integral' if
    ↪  flux=='integral' else '')
    if rtol!=2e-9:label+=f'_rtol{rtol:g}'
    DEST.mkdir(parents=True,exist_ok=True)
    if (DEST/(label+'.json')).exists():raise FileExistsError(label)
    start=time.perf_counter();y=np.tile([28.,2.55],op.m)
    sample_times=np.unique(np.r_[np.arange(0,end+1,60.),end,[100,300,900,1200,1500]])
    sample_times=sample_times[(sample_times>=0)&(sample_times<=end)]
    snapshots_at=np.unique(np.r_[0,100,300,600,900,1200,1500,1800,3600,5400,7200,9000,108
    ↪  00,14400,np.arange(21600,end,21600)])
    cuts=np.unique(np.r_[0,inp.env[:,0][inp.env[:,0]<end],inp.rad[:,0][inp.rad[:,0]<end]
    ↪  if op.shrink else [],end])
    recorded=[];mid=[];edge=[];cmax=[];locations=[];radii=[];snapshots=[y.reshape(op.shap
    ↪  e+(2,)).copy()];st=[0.]
    stats={'case':case,'nr':nr,'nz':nz,'grading':2.,'zgrading':2.,'shrink':op.shrink,'end
    ↪  _transfer':end_transfer,
           'rtol':rtol,'atol_T':1e-9,'atol_C':1e-11,'max_step':max_step,'nfev':0,'njev':0
           ↪  ,'nlu':0,
           'flux':flux+' local coefficients, vertex dual finite
           ↪  volumes','temperature':'coupled throughout; no isothermal projection',
           'environment_future':inp.future.tolist(),'source_sha':'anonymous-source'}
    def record(t,Y):
        m,e,c,l,R=op.observe(t,Y);recorded.append(t);mid.append(m);edge.append(e);cmax.ap
        ↪  pend(c);locations.append(l);radii.append(R)
    record(0.,y)
    def event(t,Y):return np.max(Y[1::2])-.15
    event.direction=-1;event.terminal=True
    root=None;event_states=[];event_times=[]
    for a,b in zip(cuts[:-1],cuts[1:]):
        ambient=(lambda t: inp.ambient(t,True)) if a>=14400 else inp.ambient
        sol=solve_ivp(lambda t,Y:op.rhs(t,Y,ambient),(a,b),y,method='BDF',
            jac=lambda
            ↪  t,Y:op.rhs(t,Y,ambient,True),rtol=rtol,atol=np.tile([1e-9,1e-11],op.m),
            max_step=min(max_step,30.) if a<14400 else max_step,dense_output=True,
            events=event if case!='q1' else None)
        if not sol.success:raise RuntimeError(sol.message)
        for key in ('nfev','njev','nlu'):stats[key]+=int(getattr(sol,key))
        stop=float(sol.t[-1]);ts=sample_times[(sample_times>a)&(sample_times<=stop)]
        if len(ts):
            vals=sol.sol(ts)
            for t,Y in zip(ts,vals.T):record(float(t),Y)
        sts=snapshots_at[(snapshots_at>a)&(snapshots_at<=stop)]
        if len(sts):
            for t,Y in zip(sts,sol.sol(sts).T):st.append(float(t));snapshots.append(Y.res
            ↪  hape(op.shape+(2,)).copy())
        y=sol.y[:,-1]
        if sol.t_events is not None and len(sol.t_events[0]):
            root=float(sol.t_events[0][0]);record(root,y);st.append(root);snapshots.appen
            ↪  d(y.reshape(op.shape+(2,)).copy())
            before=max(float(a),root-1.);after=root+1.
            ext=solve_ivp(lambda t,Y:op.rhs(t,Y,ambient),(root,after),y,method='BDF',
                jac=lambda t,Y:op.rhs(t,Y,ambient,True),rtol=rtol,atol=np.tile([1e-9,1e-1
                ↪  1],op.m),dense_output=True)
```

127

128



```
            if not ext.success:raise RuntimeError(ext.message)
            event_times=[before,root,after];event_states=[sol.sol(before).reshape(op.shap
            ↪  e+(2,)),y.reshape(op.shape+(2,)),ext.y[:,-1].reshape(op.shape+(2,))]
            break
        if b%21600==0 or b in (1800,10800,14400):print(label,'t=',b,'max C=',float(np.max
        ↪  (y[1::2])),'elapsed=',round(time.perf_counter()-start,2),flush=True)
    stats['elapsed_s']=time.perf_counter()-start;stats['event_root_s']=root
    stats['end_s']=float(recorded[-1]);stats['end_max_C']=float(cmax[-1]);stats['end_max_
    ↪  location_r_z_m']=locations[-1].tolist()
    if root is not None:stats['event_bracket_Cmax']=[float(np.max(v[:,:,1])) for v in
    ↪  event_states]
    stats['source_sha256']=hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
    np.savez_compressed(DEST/(label+'.npz'),time_s=recorded,mid=mid,end=edge,Cmax=cmax,Cm
    ↪  ax_location_r_z_m=locations,
        radius_m=radii,reference_r_m=op.r,z_m=op.z,weights=op.w,snapshot_times_s=st,snaps
        ↪  hots=snapshots,
        event_times_s=event_times,event_states=event_states)
    (DEST/(label+'.json')).write_text(json.dumps(stats,indent=2,ensure_ascii=False)+'\n')
    print(json.dumps(stats,ensure_ascii=False),flush=True)
    return stats

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--case',choices=['q1','q23','q4'],require
    ↪  d=True)
    p.add_argument('--nr',type=int,required=True);p.add_argument('--nz',type=int,default=0)
    p.add_argument('--end',type=float);p.add_argument('--rtol',type=float,default=2e-9)
    p.add_argument('--max-step',type=float,default=240.);p.add_argument('--fixed',action=
    ↪  'store_true');p.add_argument('--sealed-ends',action='store_true')
    p.add_argument('--flux',choices=['integral','harmonic'],default='integral')
    a=p.parse_args();run(a.case,a.nr,a.nz,a.end,a.rtol,a.max_step,not a.fixed,not
    ↪  a.sealed_ends,a.flux)
```

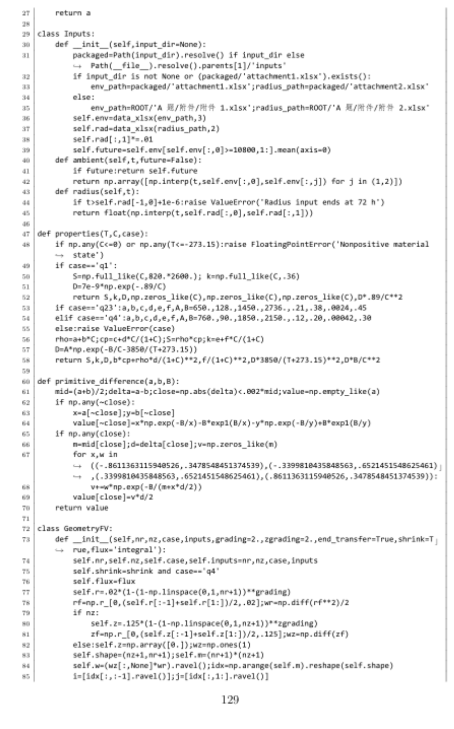
A.40 geometry_solver

support/project/q4_complete_delivery/v1/source/geometry_solver.py

```
"""Independent vertex/dual-volume axisymmetric validation of Q1/Q23/Q4.

All states remain coupled from t=0. No isothermal projection is used. The
reference grid follows homogeneous radial material contraction, while length
is fixed. Reference weights are constant; radial diffusion scales as q^-2,
side exchange as q^-1, axial diffusion/end exchange remain unchanged.
"""
from __future__ import annotations
import argparse, hashlib, json, time
from pathlib import Path
import numpy as np
from scipy.integrate import solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.sparse import coo_matrix
from scipy.special import exp1
from openpyxl import load_workbook

ROOT=Path(__file__).resolve().parents[3]
DEST=Path(__file__).resolve().parents[1]/'validation'/'geometry'

def data_xlsx(path, width):
    wb=load_workbook(path,read_only=True,data_only=True)
    rows=list(wb.active.values); wb.close()
    a=np.asarray([row[:width] for row in rows[1:] if row[0] is not None],float)
    if not np.isfinite(a).all() or not np.all(np.diff(a[:,0])>0):
        raise ValueError('Invalid input records')
```

128

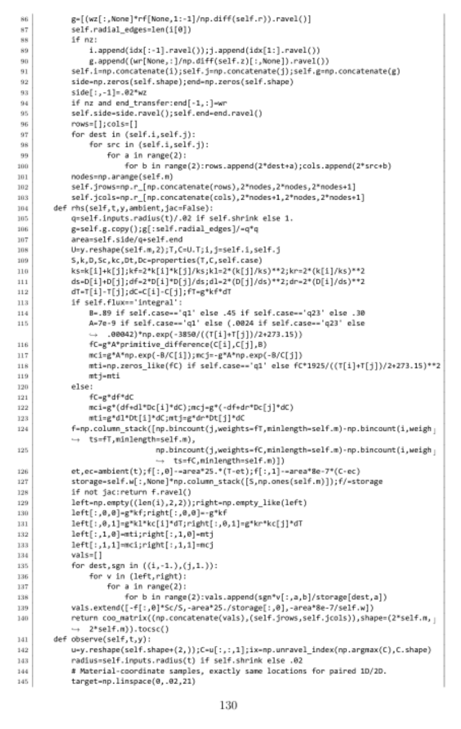
129



```
    return a

class Inputs:
    def __init__(self,input_dir=None):
        packaged=Path(input_dir).resolve() if input_dir else
        ↪  Path(__file__).resolve().parents[1]/'inputs'
        if input_dir is not None or (packaged/'attachment1.xlsx').exists():
            env_path=packaged/'attachment1.xlsx';radius_path=packaged/'attachment2.xlsx'
        else:
            env_path=ROOT/'A 题/附件/附件 1.xlsx';radius_path=ROOT/'A 题/附件/附件 2.xlsx'
        self.env=data_xlsx(env_path,3)
        self.rad=data_xlsx(radius_path,2)
        self.rad[:,1]*=.01
        self.future=self.env[self.env[:,0]>=10800,1:].mean(axis=0)
    def ambient(self,t,future=False):
        if future:return self.future
        return np.array([np.interp(t,self.env[:,0],self.env[:,j]) for j in (1,2)])
    def radius(self,t):
        if t>self.rad[-1,0]+1e-6:raise ValueError('Radius input ends at 72 h')
        return float(np.interp(t,self.rad[:,0],self.rad[:,1]))

def properties(T,C,case):
    if np.any(C<=0) or np.any(T<=-273.15):raise FloatingPointError('Nonpositive material
    ↪  state')
    if case=='q1':
        S=np.full_like(C,820.*2600.); k=np.full_like(C,.36)
        D=7e-9*np.exp(-.89/C)
        return S,k,D,np.zeros_like(C),np.zeros_like(C),np.zeros_like(C),D*.89/C**2
    if case=='q23':a,b,c,d,e,f,A,B=650.,128.,1450.,2736.,.21,.38,.0024,.45
    elif case=='q4':a,b,c,d,e,f,A,B=760.,90.,1850.,2150.,.12,.20,.00042,.30
    else:raise ValueError(case)
    rho=a+b*C;cp=c+d*C/(1+C);S=rho*cp;k=e+f*C/(1+C)
    D=A*np.exp(-B/C-3850/(T+273.15))
    return S,k,D,b*cp+rho*d/(1+C)**2,f/(1+C)**2,D*3850/(T+273.15)**2,D*B/C**2

def primitive_difference(a,b,B):
    mid=(a+b)/2;delta=a-b;close=np.abs(delta)<.002*mid;value=np.empty_like(a)
    if np.any(~close):
        x=a[~close];y=b[~close]
        value[~close]=x*np.exp(-B/x)-B*exp1(B/x)-y*np.exp(-B/y)+B*exp1(B/y)
    if np.any(close):
        m=mid[close];d=delta[close];v=np.zeros_like(m)
        for x,w in
        ↪  ((-.8611363115940526,.3478548451374539),(-.3399810435848563,.6521451548625461)
        ↪  ,(.3399810435848563,.6521451548625461),(.8611363115940526,.3478548451374539)):
            v+=w*np.exp(-B/(m+x*d/2))
        value[close]=v*d/2
    return value

class GeometryFV:
    def __init__(self,nr,nz,case,inputs,grading=2.,zgrading=2.,end_transfer=True,shrink=T
    ↪  rue,flux='integral'):
        self.nr,self.nz,self.case,self.inputs=nr,nz,case,inputs
        self.shrink=shrink and case=='q4'
        self.flux=flux
        self.r=.02*(1-(1-np.linspace(0,1,nr+1))**grading)
        rf=np.r_[0,(self.r[:-1]+self.r[1:])/2,.02];wr=np.diff(rf**2)/2
        if nz:
            self.z=.125*(1-(1-np.linspace(0,1,nz+1))**zgrading)
            zf=np.r_[0,(self.z[:-1]+self.z[1:])/2,.125];wz=np.diff(zf)
        else:self.z=np.array([0.]);wz=np.ones(1)
        self.shape=(nz+1,nr+1);self.m=(nr+1)*(nz+1)
        self.w=(wz[:,None]*wr).ravel();idx=np.arange(self.m).reshape(self.shape)
        i=[idx[:,:-1].ravel()];j=[idx[:,1:].ravel()]
```

129

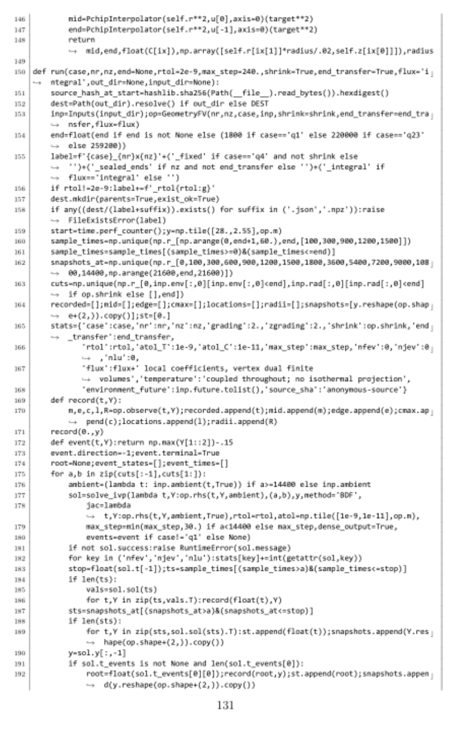
130



```
        g=[(wz[:,None]*rf[None,1:-1]/np.diff(self.r)).ravel()]
        self.radial_edges=len(i[0])
        if nz:
            i.append(idx[:-1].ravel());j.append(idx[1:].ravel())
            g.append((wr[None,:]/np.diff(self.z)[:,None]).ravel())
        self.i=np.concatenate(i);self.j=np.concatenate(j);self.g=np.concatenate(g)
        side=np.zeros(self.shape);end=np.zeros(self.shape)
        side[:,-1]=.02*wz
        if nz and end_transfer:end[-1,:]=wr
        self.side=side.ravel();self.end=end.ravel()
        rows=[];cols=[]
        for dest in (self.i,self.j):
            for src in (self.i,self.j):
                for a in range(2):
                    for b in range(2):rows.append(2*dest+a);cols.append(2*src+b)
        nodes=np.arange(self.m)
        self.jrows=np.r_[np.concatenate(rows),2*nodes,2*nodes,2*nodes+1]
        self.jcols=np.r_[np.concatenate(cols),2*nodes+1,2*nodes,2*nodes+1]
    def rhs(self,t,y,ambient,jac=False):
        q=self.inputs.radius(t)/.02 if self.shrink else 1.
        g=self.g.copy();g[:self.radial_edges]/=q*q
        area=self.side/q+self.end
        U=y.reshape(self.m,2);T,C=U.T;i,j=self.i,self.j
        S,k,D,Sc,kc,Dt,Dc=properties(T,C,self.case)
        ks=k[i]+k[j];kf=2*k[i]*k[j]/ks;kl=2*(k[j]/ks)**2;kr=2*(k[i]/ks)**2
        ds=D[i]+D[j];df=2*D[i]*D[j]/ds;dl=2*(D[j]/ds)**2;dr=2*(D[i]/ds)**2
        dT=T[i]-T[j];dC=C[i]-C[j];fT=g*kf*dT
        if self.flux=='integral':
            B=.89 if self.case=='q1' else .45 if self.case=='q23' else .30
            A=7e-9 if self.case=='q1' else (.0024 if self.case=='q23' else
            ↪  .00042)*np.exp(-3850/((T[i]+T[j])/2+273.15))
            fC=g*A*primitive_difference(C[i],C[j],B)
            mci=g*A*np.exp(-B/C[i]);mcj=-g*A*np.exp(-B/C[j])
            mti=np.zeros_like(fC) if self.case=='q1' else fC*1925/((T[i]+T[j])/2+273.15)**2
            mtj=mti
        else:
            fC=g*df*dC
            mci=g*(df+dl*Dc[i]*dC);mcj=g*(-df+dr*Dc[j]*dC)
            mti=g*dl*Dt[i]*dC;mtj=g*dr*Dt[j]*dC
        f=np.column_stack([np.bincount(j,weights=fT,minlength=self.m)-np.bincount(i,weigh
        ↪  ts=fT,minlength=self.m),
                           np.bincount(j,weights=fC,minlength=self.m)-np.bincount(i,weigh
                           ↪  ts=fC,minlength=self.m)])
        et,ec=ambient(t);f[:,0]-=area*25.*(T-et);f[:,1]-=area*8e-7*(C-ec)
        storage=self.w[:,None]*np.column_stack([S,np.ones(self.m)]);f/=storage
        if not jac:return f.ravel()
        left=np.empty((len(i),2,2));right=np.empty_like(left)
        left[:,0,0]=g*kf;right[:,0,0]=-g*kf
        left[:,0,1]=g*kl*kc[i]*dT;right[:,0,1]=g*kr*kc[j]*dT
        left[:,1,0]=mti;right[:,1,0]=mtj
        left[:,1,1]=mci;right[:,1,1]=mcj
        vals=[]
        for dest,sgn in ((i,-1.),(j,1.)):
            for v in (left,right):
                for a in range(2):
                    for b in range(2):vals.append(sgn*v[:,a,b]/storage[dest,a])
        vals.extend([-f[:,0]*Sc/S,-area*25./storage[:,0],-area*8e-7/self.w])
        return coo_matrix((np.concatenate(vals),(self.jrows,self.jcols)),shape=(2*self.m,
        ↪  2*self.m)).tocsc()
    def observe(self,t,y):
        u=y.reshape(self.shape+(2,));C=u[:,:,1];ix=np.unravel_index(np.argmax(C),C.shape)
        radius=self.inputs.radius(t) if self.shrink else .02
        # Material-coordinate samples, exactly same locations for paired 1D/2D.
        target=np.linspace(0,.02,21)
```

130

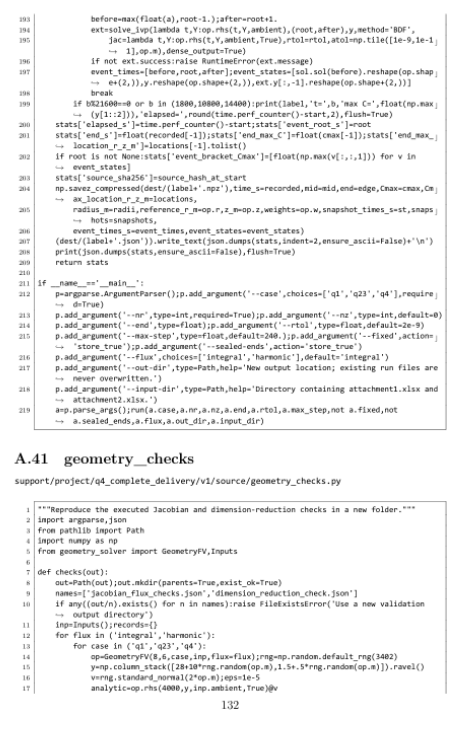
131



```
        mid=PchipInterpolator(self.r**2,u[0],axis=0)(target**2)
        end=PchipInterpolator(self.r**2,u[-1],axis=0)(target**2)
        return
        ↪  mid,end,float(C[ix]),np.array([self.r[ix[1]]*radius/.02,self.z[ix[0]]]),radius

def run(case,nr,nz,end=None,rtol=2e-9,max_step=240.,shrink=True,end_transfer=True,flux='i
↪  ntegral',out_dir=None,input_dir=None):
    source_hash_at_start=hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
    dest=Path(out_dir).resolve() if out_dir else DEST
    inp=Inputs(input_dir);op=GeometryFV(nr,nz,case,inp,shrink=shrink,end_transfer=end_tra
    ↪  nsfer,flux=flux)
    end=float(end if end is not None else (1800 if case=='q1' else 220000 if case=='q23'
    ↪  else 259200))
    label=f'{case}_{nr}x{nz}'+('_fixed' if case=='q4' and not shrink else
    ↪  '')+('_sealed_ends' if nz and not end_transfer else '')+('_integral' if
    ↪  flux=='integral' else '')
    if rtol!=2e-9:label+=f'_rtol{rtol:g}'
    dest.mkdir(parents=True,exist_ok=True)
    if any((dest/(label+suffix)).exists() for suffix in ('.json','.npz')):raise
    ↪  FileExistsError(label)
    start=time.perf_counter();y=np.tile([28.,2.55],op.m)
    sample_times=np.unique(np.r_[np.arange(0,end+1,60.),end,[100,300,900,1200,1500]])
    sample_times=sample_times[(sample_times>=0)&(sample_times<=end)]
    snapshots_at=np.unique(np.r_[0,100,300,600,900,1200,1500,1800,3600,5400,7200,9000,108
    ↪  00,14400,np.arange(21600,end,21600)])
    cuts=np.unique(np.r_[0,inp.env[:,0][inp.env[:,0]<end],inp.rad[:,0][inp.rad[:,0]<end]
    ↪  if op.shrink else [],end])
    recorded=[];mid=[];edge=[];cmax=[];locations=[];radii=[];snapshots=[y.reshape(op.shap
    ↪  e+(2,)).copy()];st=[0.]
    stats={'case':case,'nr':nr,'nz':nz,'grading':2.,'zgrading':2.,'shrink':op.shrink,'end
    ↪  _transfer':end_transfer,
           'rtol':rtol,'atol_T':1e-9,'atol_C':1e-11,'max_step':max_step,'nfev':0,'njev':0
           ↪  ,'nlu':0,
           'flux':flux+' local coefficients, vertex dual finite
           ↪  volumes','temperature':'coupled throughout; no isothermal projection',
           'environment_future':inp.future.tolist(),'source_sha':'anonymous-source'}
    def record(t,Y):
        m,e,c,l,R=op.observe(t,Y);recorded.append(t);mid.append(m);edge.append(e);cmax.ap
        ↪  pend(c);locations.append(l);radii.append(R)
    record(0.,y)
    def event(t,Y):return np.max(Y[1::2])-.15
    event.direction=-1;event.terminal=True
    root=None;event_states=[];event_times=[]
    for a,b in zip(cuts[:-1],cuts[1:]):
        ambient=(lambda t: inp.ambient(t,True)) if a>=14400 else inp.ambient
        sol=solve_ivp(lambda t,Y:op.rhs(t,Y,ambient),(a,b),y,method='BDF',
            jac=lambda
            ↪  t,Y:op.rhs(t,Y,ambient,True),rtol=rtol,atol=np.tile([1e-9,1e-11],op.m),
            max_step=min(max_step,30.) if a<14400 else max_step,dense_output=True,
            events=event if case!='q1' else None)
        if not sol.success:raise RuntimeError(sol.message)
        for key in ('nfev','njev','nlu'):stats[key]+=int(getattr(sol,key))
        stop=float(sol.t[-1]);ts=sample_times[(sample_times>a)&(sample_times<=stop)]
        if len(ts):
            vals=sol.sol(ts)
            for t,Y in zip(ts,vals.T):record(float(t),Y)
        sts=snapshots_at[(snapshots_at>a)&(snapshots_at<=stop)]
        if len(sts):
            for t,Y in zip(sts,sol.sol(sts).T):st.append(float(t));snapshots.append(Y.res
            ↪  hape(op.shape+(2,)).copy())
        y=sol.y[:,-1]
        if sol.t_events is not None and len(sol.t_events[0]):
            root=float(sol.t_events[0][0]);record(root,y);st.append(root);snapshots.appen
            ↪  d(y.reshape(op.shape+(2,)).copy())
```

131

132



```
            before=max(float(a),root-1.);after=root+1.
            ext=solve_ivp(lambda t,Y:op.rhs(t,Y,ambient),(root,after),y,method='BDF',
                jac=lambda t,Y:op.rhs(t,Y,ambient,True),rtol=rtol,atol=np.tile([1e-9,1e-1
                ↪  1],op.m),dense_output=True)
            if not ext.success:raise RuntimeError(ext.message)
            event_times=[before,root,after];event_states=[sol.sol(before).reshape(op.shap
            ↪  e+(2,)),y.reshape(op.shape+(2,)),ext.y[:,-1].reshape(op.shape+(2,))]
            break
        if b%21600==0 or b in (1800,10800,14400):print(label,'t=',b,'max C=',float(np.max
        ↪  (y[1::2])),'elapsed=',round(time.perf_counter()-start,2),flush=True)
    stats['elapsed_s']=time.perf_counter()-start;stats['event_root_s']=root
    stats['end_s']=float(recorded[-1]);stats['end_max_C']=float(cmax[-1]);stats['end_max_
    ↪  location_r_z_m']=locations[-1].tolist()
    if root is not None:stats['event_bracket_Cmax']=[float(np.max(v[:,:,1])) for v in
    ↪  event_states]
    stats['source_sha256']=source_hash_at_start
    np.savez_compressed(dest/(label+'.npz'),time_s=recorded,mid=mid,end=edge,Cmax=cmax,Cm
    ↪  ax_location_r_z_m=locations,
        radius_m=radii,reference_r_m=op.r,z_m=op.z,weights=op.w,snapshot_times_s=st,snaps
        ↪  hots=snapshots,
        event_times_s=event_times,event_states=event_states)
    (dest/(label+'.json')).write_text(json.dumps(stats,indent=2,ensure_ascii=False)+'\n')
    print(json.dumps(stats,ensure_ascii=False),flush=True)
    return stats

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--case',choices=['q1','q23','q4'],require
    ↪  d=True)
    p.add_argument('--nr',type=int,required=True);p.add_argument('--nz',type=int,default=0)
    p.add_argument('--end',type=float);p.add_argument('--rtol',type=float,default=2e-9)
    p.add_argument('--max-step',type=float,default=240.);p.add_argument('--fixed',action=
    ↪  'store_true');p.add_argument('--sealed-ends',action='store_true')
    p.add_argument('--flux',choices=['integral','harmonic'],default='integral')
    p.add_argument('--out-dir',type=Path,help='New output location; existing run files are
    ↪  never overwritten.')
    p.add_argument('--input-dir',type=Path,help='Directory containing attachment1.xlsx and
    ↪  attachment2.xlsx.')
    a=p.parse_args();run(a.case,a.nr,a.nz,a.end,a.rtol,a.max_step,not a.fixed,not
    ↪  a.sealed_ends,a.flux,a.out_dir,a.input_dir)
```

A.41 geometry_checks

support/project/q4_complete_delivery/v1/source/geometry_checks.py

```
"""Reproduce the executed Jacobian and dimension-reduction checks in a new folder."""
import argparse,json
from pathlib import Path
import numpy as np
from geometry_solver import GeometryFV,Inputs

def checks(out):
    out=Path(out);out.mkdir(parents=True,exist_ok=True)
    names=['jacobian_flux_checks.json','dimension_reduction_check.json']
    if any((out/n).exists() for n in names):raise FileExistsError('Use a new validation
    ↪  output directory')
    inp=Inputs();records={}
    for flux in ('integral','harmonic'):
        for case in ('q1','q23','q4'):
            op=GeometryFV(8,6,case,inp,flux=flux);rng=np.random.default_rng(3402)
            y=np.column_stack([28+10*rng.random(op.m),1.5+.5*rng.random(op.m)]).ravel()
            v=rng.standard_normal(2*op.m);eps=1e-5
            analytic=op.rhs(4000,y,inp.ambient,True)@v
```

132

133

134

135

136

137

138

139

140

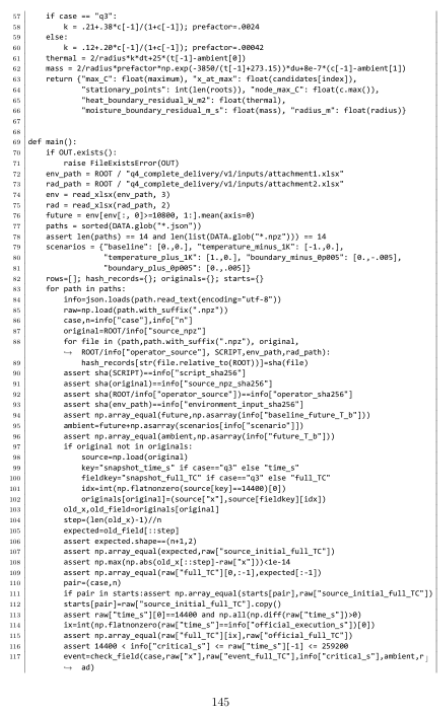
141

142

143

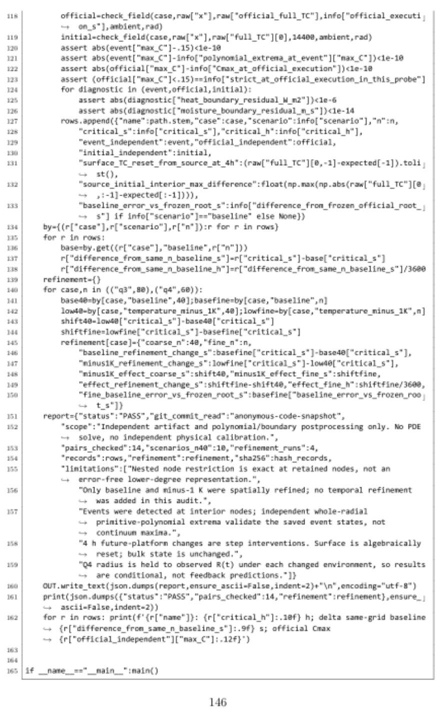
144

145



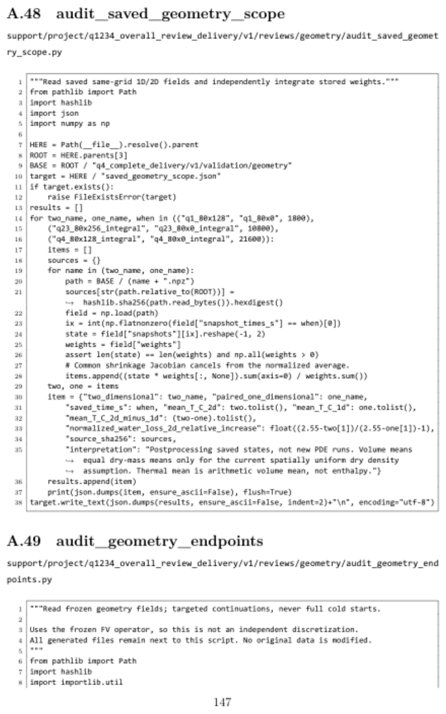
```
    if case == "q3":
        k = .21+.38*c[-1]/(1+c[-1]); prefactor=.0024
    else:
        k = .12+.20*c[-1]/(1+c[-1]); prefactor=.00042
    thermal = 2/radius*k*dt+25*(t[-1]-ambient[0])
    mass = 2/radius*prefactor*np.exp(-3850/(t[-1]+273.15))*du+8e-7*(c[-1]-ambient[1])
    return {"max_C": float(maximum), "x_at_max": float(candidates[index]),
            "stationary_points": int(len(roots)), "node_max_C": float(c.max()),
            "heat_boundary_residual_W_m2": float(thermal),
            "moisture_boundary_residual_m_s": float(mass), "radius_m": float(radius)}


def main():
    if OUT.exists():
        raise FileExistsError(OUT)
    env_path = ROOT / "q4_complete_delivery/v1/inputs/attachment1.xlsx"
    rad_path = ROOT / "q4_complete_delivery/v1/inputs/attachment2.xlsx"
    env = read_xlsx(env_path, 3)
    rad = read_xlsx(rad_path, 2)
    future = env[env[:, 0]>=10800, 1:].mean(axis=0)
    paths = sorted(DATA.glob("*.json"))
    assert len(paths) == 14 and len(list(DATA.glob("*.npz"))) == 14
    scenarios = {"baseline": [0.,0.], "temperature_minus_1K": [-1.,0.],
                 "temperature_plus_1K": [1.,0.], "boundary_minus_0p005": [0.,-.005],
                 "boundary_plus_0p005": [0.,.005]}
    rows=[]; hash_records={}; originals={}; starts={}
    for path in paths:
        info=json.loads(path.read_text(encoding="utf-8"))
        raw=np.load(path.with_suffix(".npz"))
        case,n=info["case"],info["n"]
        original=ROOT/info["source_npz"]
        for file in (path,path.with_suffix(".npz"), original,
        ↪  ROOT/info["operator_source"], SCRIPT,env_path,rad_path):
            hash_records[str(file.relative_to(ROOT))]=sha(file)
        assert sha(SCRIPT)==info["script_sha256"]
        assert sha(original)==info["source_npz_sha256"]
        assert sha(ROOT/info["operator_source"])==info["operator_sha256"]
        assert sha(env_path)==info["environment_input_sha256"]
        assert np.array_equal(future,np.asarray(info["baseline_future_T_b"]))
        ambient=future+np.asarray(scenarios[info["scenario"]])
        assert np.array_equal(ambient,np.asarray(info["future_T_b"]))
        if original not in originals:
            source=np.load(original)
            key="snapshot_time_s" if case=="q3" else "time_s"
            fieldkey="snapshot_full_TC" if case=="q3" else "full_TC"
            idx=int(np.flatnonzero(source[key]==14400)[0])
            originals[original]=(source["x"],source[fieldkey][idx])
        old_x,old_field=originals[original]
        step=(len(old_x)-1)//n
        expected=old_field[::step]
        assert expected.shape==(n+1,2)
        assert np.array_equal(expected,raw["source_initial_full_TC"])
        assert np.max(np.abs(old_x[::step]-raw["x"]))<1e-14
        assert np.array_equal(raw["full_TC"][0,:-1],expected[:-1])
        pair=(case,n)
        if pair in starts:assert np.array_equal(starts[pair],raw["source_initial_full_TC"])
        starts[pair]=raw["source_initial_full_TC"].copy()
        assert raw["time_s"][0]==14400 and np.all(np.diff(raw["time_s"])>0)
        ix=int(np.flatnonzero(raw["time_s"]==info["official_execution_s"])[0])
        assert np.array_equal(raw["full_TC"][ix],raw["official_full_TC"])
        assert 14400 < info["critical_s"] <= raw["time_s"][-1] <= 259200
        event=check_field(case,raw["x"],raw["event_full_TC"],info["critical_s"],ambient,r
        ↪  ad)
```

146



```
        official=check_field(case,raw["x"],raw["official_full_TC"],info["official_executi
        ↪  on_s"],ambient,rad)
        initial=check_field(case,raw["x"],raw["full_TC"][0],14400,ambient,rad)
        assert abs(event["max_C"]-.15)<1e-10
        assert abs(event["max_C"]-info["polynomial_extrema_at_event"]["max_C"])<1e-10
        assert abs(official["max_C"]-info["Cmax_at_official_execution"])<1e-10
        assert (official["max_C"]<.15)==info["strict_at_official_execution_in_this_probe"]
        for diagnostic in (event,official,initial):
            assert abs(diagnostic["heat_boundary_residual_W_m2"])<1e-6
            assert abs(diagnostic["moisture_boundary_residual_m_s"])<1e-14
        rows.append({"name":path.stem,"case":case,"scenario":info["scenario"],"n":n,
            "critical_s":info["critical_s"],"critical_h":info["critical_h"],
            "event_independent":event,"official_independent":official,
            "initial_independent":initial,
            "surface_TC_reset_from_source_at_4h":(raw["full_TC"][0,-1]-expected[-1]).toli
            ↪  st(),
            "source_initial_interior_max_difference":float(np.max(np.abs(raw["full_TC"][0
            ↪  ,:-1]-expected[:-1]))),
            "baseline_error_vs_frozen_root_s":info["difference_from_frozen_official_root_
            ↪  s"] if info["scenario"]=="baseline" else None})
    by={(r["case"],r["scenario"],r["n"]):r for r in rows}
    for r in rows:
        base=by.get((r["case"],"baseline",r["n"]))
        r["difference_from_same_n_baseline_s"]=r["critical_s"]-base["critical_s"]
        r["difference_from_same_n_baseline_h"]=r["difference_from_same_n_baseline_s"]/3600
    refinement={}
    for case,n in (("q3",80),("q4",60)):
        base40=by[case,"baseline",40];basefine=by[case,"baseline",n]
        low40=by[case,"temperature_minus_1K",40];lowfine=by[case,"temperature_minus_1K",n]
        shift40=low40["critical_s"]-base40["critical_s"]
        shiftfine=lowfine["critical_s"]-basefine["critical_s"]
        refinement[case]={"coarse_n":40,"fine_n":n,
            "baseline_refinement_change_s":basefine["critical_s"]-base40["critical_s"],
            "minus1K_refinement_change_s":lowfine["critical_s"]-low40["critical_s"],
            "minus1K_effect_coarse_s":shift40,"minus1K_effect_fine_s":shiftfine,
            "effect_refinement_change_s":shiftfine-shift40,"effect_fine_h":shiftfine/3600,
            "fine_baseline_error_vs_frozen_root_s":basefine["baseline_error_vs_frozen_roo
            ↪  t_s"]}
    report={"status":"PASS","git_commit_read":"anonymous-code-snapshot",
        "scope":"Independent artifact and polynomial/boundary postprocessing only. No PDE
        ↪  solve, no independent physical calibration.",
        "pairs_checked":14,"scenarios_n40":10,"refinement_runs":4,
        "records":rows,"refinement":refinement,"sha256":hash_records,
        "limitations":["Nested node restriction is exact at retained nodes, not an
        ↪  error-free lower-degree representation.",
            "Only baseline and minus-1 K were spatially refined; no temporal refinement
            ↪  was added in this audit.",
            "Events were detected at interior nodes; independent whole-radial
            ↪  primitive-polynomial extrema validate the saved event states, not
            ↪  continuum maxima.",
            "4 h future-platform changes are step interventions. Surface is algebraically
            ↪  reset; bulk state is unchanged.",
            "Q4 radius is held to observed R(t) under each changed environment, so results
            ↪  are conditional, not feedback predictions."]}
    OUT.write_text(json.dumps(report,ensure_ascii=False,indent=2)+"\n",encoding="utf-8")
    print(json.dumps({"status":"PASS","pairs_checked":14,"refinement":refinement},ensure_
    ↪  ascii=False,indent=2))
    for r in rows: print(f'{r["name"]}: {r["critical_h"]:.10f} h; delta same-grid baseline
    ↪  {r["difference_from_same_n_baseline_s"]:.9f} s; official Cmax
    ↪  {r["official_independent"]["max_C"]:.12f}')


if __name__=="__main__":main()
```

147



A.48 audit_saved_geometry_scope

support/project/q1234_overall_review_delivery/v1/reviews/geometry/audit_saved_geometry_scope.py

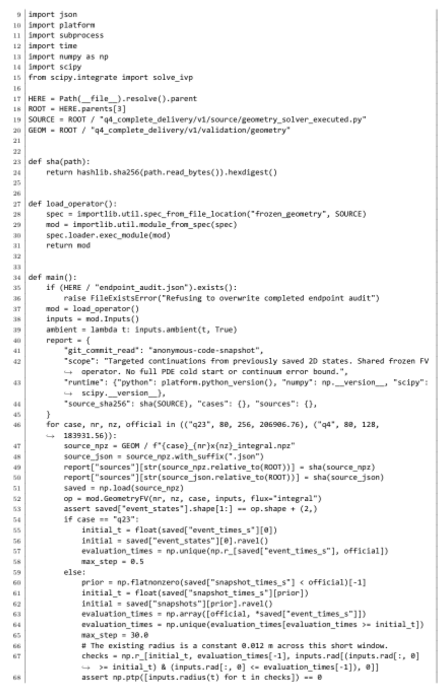
```
"""Read saved same-grid 1D/2D fields and independently integrate stored weights."""
from pathlib import Path
import hashlib
import json
import numpy as np

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[3]
BASE = ROOT / "q4_complete_delivery/v1/validation/geometry"
target = HERE / "saved_geometry_scope.json"
if target.exists():
    raise FileExistsError(target)
results = []
for two_name, one_name, when in (("q1_80x128", "q1_80x0", 1800),
    ("q23_80x256_integral", "q23_80x0_integral", 10800),
    ("q4_80x128_integral", "q4_80x0_integral", 21600)):
    items = []
    sources = {}
    for name in (two_name, one_name):
        path = BASE / (name + ".npz")
        sources[str(path.relative_to(ROOT))] =
        ↪  hashlib.sha256(path.read_bytes()).hexdigest()
        field = np.load(path)
        ix = int(np.flatnonzero(field["snapshot_times_s"] == when)[0])
        state = field["snapshots"][ix].reshape(-1, 2)
        weights = field["weights"]
        assert len(state) == len(weights) and np.all(weights > 0)
        # Common shrinkage Jacobian cancels from the normalized average.
        items.append((state * weights[:, None]).sum(axis=0) / weights.sum())
    two, one = items
    item = {"two_dimensional": two_name, "paired_one_dimensional": one_name,
        "saved_time_s": when, "mean_T_C_2d": two.tolist(), "mean_T_C_1d": one.tolist(),
        "mean_T_C_2d_minus_1d": (two-one).tolist(),
        "normalized_water_loss_2d_relative_increase": float((2.55-two[1])/(2.55-one[1])-1),
        "source_sha256": sources,
        "interpretation": "Postprocessing saved states, not new PDE runs. Volume means
        ↪  equal dry-mass means only for the current spatially uniform dry density
        ↪  assumption. Thermal mean is arithmetic volume mean, not enthalpy."}
    results.append(item)
    print(json.dumps(item, ensure_ascii=False), flush=True)
target.write_text(json.dumps(results, ensure_ascii=False, indent=2)+"\n", encoding="utf-8")
```

A.49 audit_geometry_endpoints

support/project/q1234_overall_review_delivery/v1/reviews/geometry/audit_geometry_endpoints.py

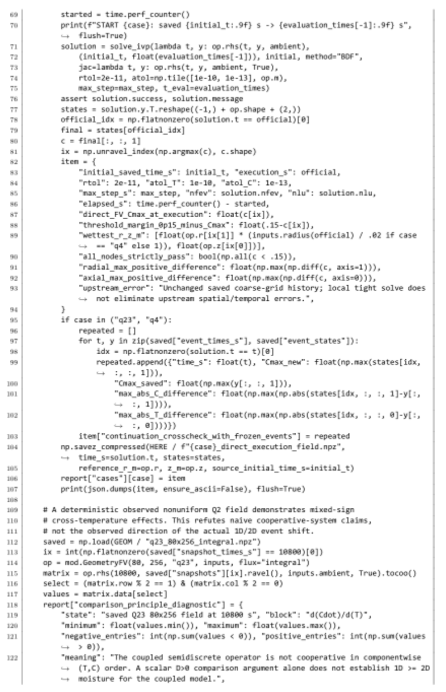
```
"""Read frozen geometry fields; targeted continuations, never full cold starts.

Uses the frozen FV operator, so this is not an independent discretization.
All generated files remain next to this script. No original data is modified.
"""
from pathlib import Path
import hashlib
import importlib.util
```

148



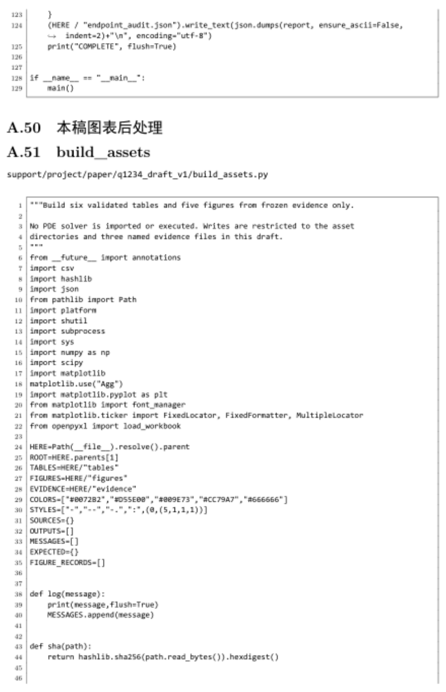
```
import json
import platform
import subprocess
import time
import numpy as np
import scipy
from scipy.integrate import solve_ivp

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[3]
SOURCE = ROOT / "q4_complete_delivery/v1/source/geometry_solver_executed.py"
GEOM = ROOT / "q4_complete_delivery/v1/validation/geometry"


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def load_operator():
    spec = importlib.util.spec_from_file_location("frozen_geometry", SOURCE)
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod)
    return mod


def main():
    if (HERE / "endpoint_audit.json").exists():
        raise FileExistsError("Refusing to overwrite completed endpoint audit")
    mod = load_operator()
    inputs = mod.Inputs()
    ambient = lambda t: inputs.ambient(t, True)
    report = {
        "git_commit_read": "anonymous-code-snapshot",
        "scope": "Targeted continuations from previously saved 2D states. Shared frozen FV
        ↪  operator. No full PDE cold start or continuum error bound.",
        "runtime": {"python": platform.python_version(), "numpy": np.__version__, "scipy":
        ↪  scipy.__version__},
        "source_sha256": sha(SOURCE), "cases": {}, "sources": {},
    }
    for case, nr, nz, official in (("q23", 80, 256, 206906.76), ("q4", 80, 128,
    ↪  183931.56)):
        source_npz = GEOM / f"{case}_{nr}x{nz}_integral.npz"
        source_json = source_npz.with_suffix(".json")
        report["sources"][str(source_npz.relative_to(ROOT))] = sha(source_npz)
        report["sources"][str(source_json.relative_to(ROOT))] = sha(source_json)
        saved = np.load(source_npz)
        op = mod.GeometryFV(nr, nz, case, inputs, flux="integral")
        assert saved["event_states"].shape[1:] == op.shape + (2,)
        if case == "q23":
            initial_t = float(saved["event_times_s"][0])
            initial = saved["event_states"][0].ravel()
            evaluation_times = np.unique(np.r_[saved["event_times_s"], official])
            max_step = 0.5
        else:
            prior = np.flatnonzero(saved["snapshot_times_s"] < official)[-1]
            initial_t = float(saved["snapshot_times_s"][prior])
            initial = saved["snapshots"][prior].ravel()
            evaluation_times = np.array([official, *saved["event_times_s"]])
            evaluation_times = np.unique(evaluation_times[evaluation_times >= initial_t])
            max_step = 30.0
            # The existing radius is a constant 0.012 m across this short window.
            checks = np.r_[initial_t, evaluation_times[-1], inputs.rad[(inputs.rad[:, 0]
            ↪  >= initial_t) & (inputs.rad[:, 0] <= evaluation_times[-1]), 0]]
            assert np.ptp([inputs.radius(t) for t in checks]) == 0
```

149



```
        started = time.perf_counter()
        print(f"START {case}: saved {initial_t:.9f} s -> {evaluation_times[-1]:.9f} s",
        ↪  flush=True)
        solution = solve_ivp(lambda t, y: op.rhs(t, y, ambient),
            (initial_t, float(evaluation_times[-1])), initial, method="BDF",
            jac=lambda t, y: op.rhs(t, y, ambient, True),
            rtol=2e-11, atol=np.tile([1e-10, 1e-13], op.m),
            max_step=max_step, t_eval=evaluation_times)
        assert solution.success, solution.message
        states = solution.y.T.reshape((-1,) + op.shape + (2,))
        official_idx = np.flatnonzero(solution.t == official)[0]
        final = states[official_idx]
        c = final[:, :, 1]
        ix = np.unravel_index(np.argmax(c), c.shape)
        item = {
            "initial_saved_time_s": initial_t, "execution_s": official,
            "rtol": 2e-11, "atol_T": 1e-10, "atol_C": 1e-13,
            "max_step_s": max_step, "nfev": solution.nfev, "nlu": solution.nlu,
            "elapsed_s": time.perf_counter() - started,
            "direct_FV_Cmax_at_execution": float(c[ix]),
            "threshold_margin_0p15_minus_Cmax": float(.15-c[ix]),
            "wettest_r_z_m": [float(op.r[ix[1]] * (inputs.radius(official) / .02 if case
            ↪  == "q4" else 1)), float(op.z[ix[0]])],
            "all_nodes_strictly_pass": bool(np.all(c < .15)),
            "radial_max_positive_difference": float(np.max(np.diff(c, axis=1))),
            "axial_max_positive_difference": float(np.max(np.diff(c, axis=0))),
            "upstream_error": "Unchanged saved coarse-grid history; local tight solve does
            ↪  not eliminate upstream spatial/temporal errors.",
        }
        if case in ("q23", "q4"):
            repeated = []
            for t, y in zip(saved["event_times_s"], saved["event_states"]):
                idx = np.flatnonzero(solution.t == t)[0]
                repeated.append({"time_s": float(t), "Cmax_new": float(np.max(states[idx,
                ↪  :, :, 1])),
                    "Cmax_saved": float(np.max(y[:, :, 1])),
                    "max_abs_C_difference": float(np.max(np.abs(states[idx, :, :, 1]-y[:,
                    ↪  :, 1]))),
                    "max_abs_T_difference": float(np.max(np.abs(states[idx, :, :, 0]-y[:,
                    ↪  :, 0])))})
            item["continuation_crosscheck_with_frozen_events"] = repeated
        np.savez_compressed(HERE / f"{case}_direct_execution_field.npz",
        ↪  time_s=solution.t, states=states,
            reference_r_m=op.r, z_m=op.z, source_initial_time_s=initial_t)
        report["cases"][case] = item
        print(json.dumps(item, ensure_ascii=False), flush=True)

    # A deterministic observed nonuniform Q2 field demonstrates mixed-sign
    # cross-temperature effects. This refutes naive cooperative-system claims,
    # not the observed direction of the actual 1D/2D event shift.
    saved = np.load(GEOM / "q23_80x256_integral.npz")
    ix = int(np.flatnonzero(saved["snapshot_times_s"] == 10800)[0])
    op = mod.GeometryFV(80, 256, "q23", inputs, flux="integral")
    matrix = op.rhs(10800, saved["snapshots"][ix].ravel(), inputs.ambient, True).tocoo()
    select = (matrix.row % 2 == 1) & (matrix.col % 2 == 0)
    values = matrix.data[select]
    report["comparison_principle_diagnostic"] = {
        "state": "saved Q23 80x256 field at 10800 s", "block": "d(Cdot)/d(T)",
        "minimum": float(values.min()), "maximum": float(values.max()),
        "negative_entries": int(np.sum(values < 0)), "positive_entries": int(np.sum(values
        ↪  > 0)),
        "meaning": "The coupled semidiscrete operator is not cooperative in componentwise
        ↪  (T,C) order. A scalar D>0 comparison argument alone does not establish 1D >= 2D
        ↪  moisture for the coupled model.",
```

150



```
    }
    (HERE / "endpoint_audit.json").write_text(json.dumps(report, ensure_ascii=False,
    ↪  indent=2)+"\n", encoding="utf-8")
    print("COMPLETE", flush=True)


if __name__ == "__main__":
    main()
```

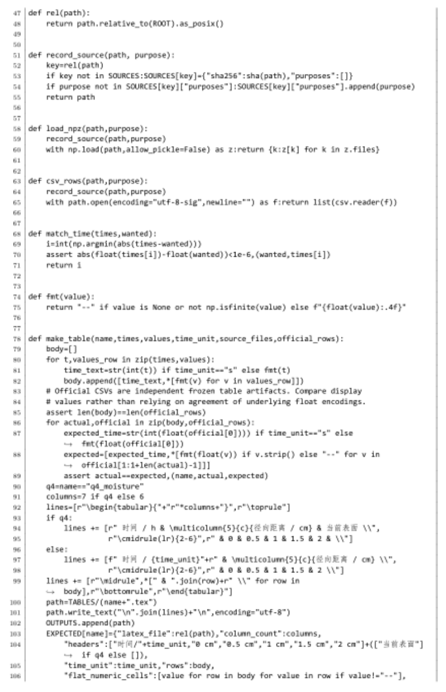
A.50 本稿图表后处理

A.51 build_assets

support/project/paper/q1234_draft_v1/build_assets.py

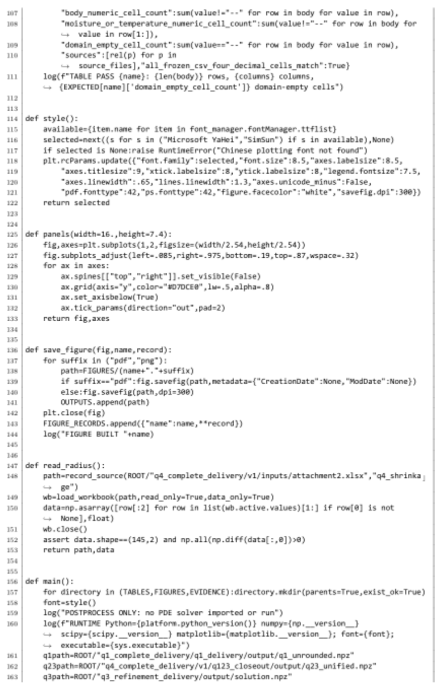
```
"""Build six validated tables and five figures from frozen evidence only.

No PDE solver is imported or executed. Writes are restricted to the asset
directories and three named evidence files in this draft.
"""
from __future__ import annotations
import csv
import hashlib
import json
from pathlib import Path
import platform
import shutil
import subprocess
import sys
import numpy as np
import scipy
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.ticker import FixedLocator, FixedFormatter, MultipleLocator
from openpyxl import load_workbook

HERE=Path(__file__).resolve().parent
ROOT=HERE.parents[1]
TABLES=HERE/"tables"
FIGURES=HERE/"figures"
EVIDENCE=HERE/"evidence"
COLORS=["#0072B2","#D55E00","#009E73","#CC79A7","#666666"]
STYLES=["-","--","-.",":",(0,(5,1,1,1))]
SOURCES={}
OUTPUTS=[]
MESSAGES=[]
EXPECTED={}
FIGURE_RECORDS=[]


def log(message):
    print(message,flush=True)
    MESSAGES.append(message)


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


```

151



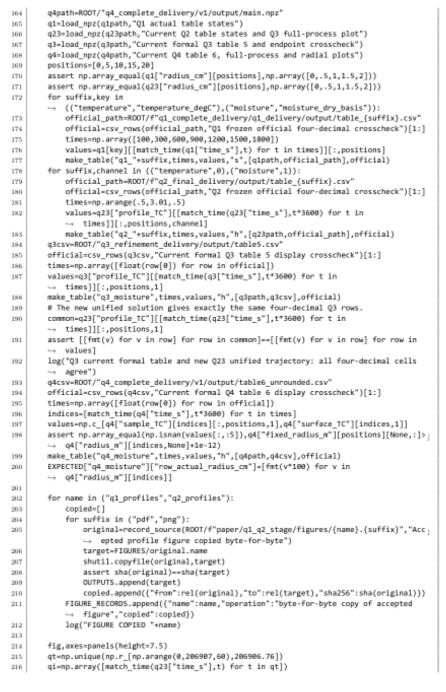
```
def rel(path):
    return path.relative_to(ROOT).as_posix()


def record_source(path, purpose):
    key=rel(path)
    if key not in SOURCES:SOURCES[key]={"sha256":sha(path),"purposes":[]}
    if purpose not in SOURCES[key]["purposes"]:SOURCES[key]["purposes"].append(purpose)
    return path


def load_npz(path,purpose):
    record_source(path,purpose)
    with np.load(path,allow_pickle=False) as z:return {k:z[k] for k in z.files}


def csv_rows(path,purpose):
    record_source(path,purpose)
    with path.open(encoding="utf-8-sig",newline="") as f:return list(csv.reader(f))


def match_time(times,wanted):
    i=int(np.argmin(abs(times-wanted)))
    assert abs(float(times[i])-float(wanted))<1e-6,(wanted,times[i])
    return i


def fmt(value):
    return "--" if value is None or not np.isfinite(value) else f"{float(value):.4f}"


def make_table(name,times,values,time_unit,source_files,official_rows):
    body=[]
    for t,values_row in zip(times,values):
        time_text=str(int(t)) if time_unit=="s" else fmt(t)
        body.append([time_text,*[fmt(v) for v in values_row]])
    # Official CSVs are independent frozen table artifacts. Compare display
    # values rather than relying on agreement of underlying float encodings.
    assert len(body)==len(official_rows)
    for actual,official in zip(body,official_rows):
        expected_time=str(int(float(official[0]))) if time_unit=="s" else
        ↪  fmt(float(official[0]))
        expected=[expected_time,*[fmt(float(v)) if v.strip() else "--" for v in
        ↪  official[1:1+len(actual)-1]]]
        assert actual==expected,(name,actual,expected)
    q4=name=="q4_moisture"
    columns=7 if q4 else 6
    lines=[r"\begin{tabular}{"+"r"*columns+"}",r"\toprule"]
    if q4:
        lines += [r" 时间 / h & \multicolumn{5}{c}{径向距离 / cm} & 当前表面 \\",
                  r"\cmidrule(lr){2-6}",r" & 0 & 0.5 & 1 & 1.5 & 2 & \\"]
    else:
        lines += [f" 时间 / {time_unit}"+r" & \multicolumn{5}{c}{径向距离 / cm} \\",
                  r"\cmidrule(lr){2-6}",r" & 0 & 0.5 & 1 & 1.5 & 2 \\"]
    lines += [r"\midrule",*[" & ".join(row)+r" \\" for row in
    ↪  body],r"\bottomrule",r"\end{tabular}"]
    path=TABLES/(name+".tex")
    path.write_text("\n".join(lines)+"\n",encoding="utf-8")
    OUTPUTS.append(path)
    EXPECTED[name]={"latex_file":rel(path),"column_count":columns,
        "headers":["时间/"+time_unit,"0 cm","0.5 cm","1 cm","1.5 cm","2 cm"]+(["当前表面"]
        ↪  if q4 else []),
        "time_unit":time_unit,"rows":body,
        "flat_numeric_cells":[value for row in body for value in row if value!="--"],
```

152



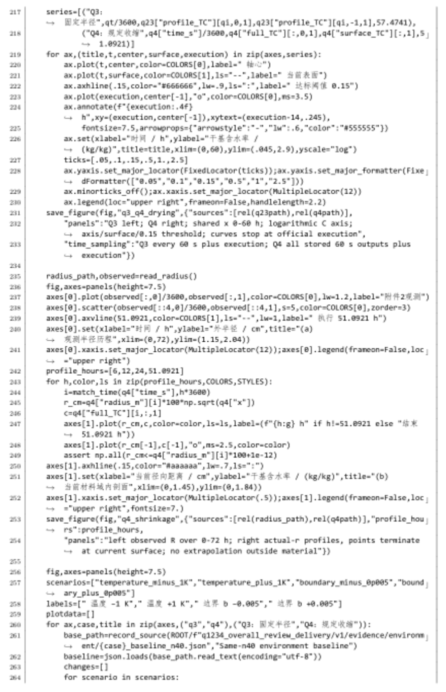
```
        "body_numeric_cell_count":sum(value!="--" for row in body for value in row),
        "moisture_or_temperature_numeric_cell_count":sum(value!="--" for row in body for
        ↪  value in row[1:]),
        "domain_empty_cell_count":sum(value=="--" for row in body for value in row),
        "sources":[rel(p) for p in
        ↪  source_files],"all_frozen_csv_four_decimal_cells_match":True}
    log(f"TABLE PASS {name}: {len(body)} rows, {columns} columns,
    ↪  {EXPECTED[name]['domain_empty_cell_count']} domain-empty cells")


def style():
    available={item.name for item in font_manager.fontManager.ttflist}
    selected=next((s for s in ("Microsoft YaHei","SimSun") if s in available),None)
    if selected is None:raise RuntimeError("Chinese plotting font not found")
    plt.rcParams.update({"font.family":selected,"font.size":8.5,"axes.labelsize":8.5,
        "axes.titlesize":9,"xtick.labelsize":8,"ytick.labelsize":8,"legend.fontsize":7.5,
        "axes.linewidth":.65,"lines.linewidth":1.3,"axes.unicode_minus":False,
        "pdf.fonttype":42,"ps.fonttype":42,"figure.facecolor":"white","savefig.dpi":300})
    return selected


def panels(width=16.,height=7.4):
    fig,axes=plt.subplots(1,2,figsize=(width/2.54,height/2.54))
    fig.subplots_adjust(left=.085,right=.975,bottom=.19,top=.87,wspace=.32)
    for ax in axes:
        ax.spines[["top","right"]].set_visible(False)
        ax.grid(axis="y",color="#D7DCE0",lw=.5,alpha=.8)
        ax.set_axisbelow(True)
        ax.tick_params(direction="out",pad=2)
    return fig,axes


def save_figure(fig,name,record):
    for suffix in ("pdf","png"):
        path=FIGURES/(name+"."+suffix)
        if suffix=="pdf":fig.savefig(path,metadata={"CreationDate":None,"ModDate":None})
        else:fig.savefig(path,dpi=300)
        OUTPUTS.append(path)
    plt.close(fig)
    FIGURE_RECORDS.append({"name":name,**record})
    log("FIGURE BUILT "+name)


def read_radius():
    path=record_source(ROOT/"q4_complete_delivery/v1/inputs/attachment2.xlsx","q4_shrinka
    ↪  ge")
    wb=load_workbook(path,read_only=True,data_only=True)
    data=np.asarray([row[:2] for row in list(wb.active.values)[1:] if row[0] is not
    ↪  None],float)
    wb.close()
    assert data.shape==(145,2) and np.all(np.diff(data[:,0])>0)
    return path,data


def main():
    for directory in (TABLES,FIGURES,EVIDENCE):directory.mkdir(parents=True,exist_ok=True)
    font=style()
    log("POSTPROCESS ONLY: no PDE solver imported or run")
    log(f"RUNTIME Python={platform.python_version()} numpy={np.__version__}
    ↪  scipy={scipy.__version__} matplotlib={matplotlib.__version__}; font={font};
    ↪  executable={sys.executable}")
    q1path=ROOT/"q1_complete_delivery/q1_delivery/output/q1_unrounded.npz"
    q23path=ROOT/"q4_complete_delivery/v1/q123_closeout/output/q23_unified.npz"
    q3path=ROOT/"q3_refinement_delivery/output/solution.npz"
```

153



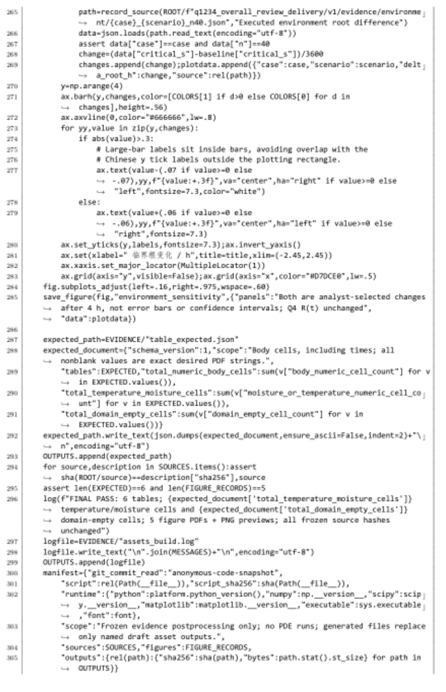
```
    q4path=ROOT/"q4_complete_delivery/v1/output/main.npz"
    q1=load_npz(q1path,"Q1 actual table states")
    q23=load_npz(q23path,"Current Q2 table states and Q3 full-process plot")
    q3=load_npz(q3path,"Current formal Q3 table 5 and endpoint crosscheck")
    q4=load_npz(q4path,"Current Q4 table 6, full-process and radial plots")
    positions=[0,5,10,15,20]
    assert np.array_equal(q1["radius_cm"][positions],np.array([0,.5,1,1.5,2]))
    assert np.array_equal(q23["radius_cm"][positions],np.array([0,.5,1,1.5,2]))
    for suffix,key in
    ↪  (("temperature","temperature_degC"),("moisture","moisture_dry_basis")):
        official_path=ROOT/f"q1_complete_delivery/q1_delivery/output/table_{suffix}.csv"
        official=csv_rows(official_path,"Q1 frozen official four-decimal crosscheck")[1:]
        times=np.array([100,300,600,900,1200,1500,1800])
        values=q1[key][[match_time(q1["time_s"],t) for t in times]][:,positions]
        make_table("q1_"+suffix,times,values,"s",[q1path,official_path],official)
    for suffix,channel in (("temperature",0),("moisture",1)):
        official_path=ROOT/f"q2_final_delivery/output/table_{suffix}.csv"
        official=csv_rows(official_path,"Q2 frozen official four-decimal crosscheck")[1:]
        times=np.arange(.5,3.01,.5)
        values=q23["profile_TC"][[match_time(q23["time_s"],t*3600) for t in
        ↪  times]][:,positions,channel]
        make_table("q2_"+suffix,times,values,"h",[q23path,official_path],official)
    q3csv=ROOT/"q3_refinement_delivery/output/table5.csv"
    official=csv_rows(q3csv,"Current formal Q3 table 5 display crosscheck")[1:]
    times=np.array([float(row[0]) for row in official])
    values=q3["profile_TC"][[match_time(q3["time_s"],t*3600) for t in
    ↪  times]][:,positions,1]
    make_table("q3_moisture",times,values,"h",[q3path,q3csv],official)
    # The new unified solution gives exactly the same four-decimal Q3 rows.
    common=q23["profile_TC"][[match_time(q23["time_s"],t*3600) for t in
    ↪  times]][:,positions,1]
    assert [[fmt(v) for v in row] for row in common]==[[fmt(v) for v in row] for row in
    ↪  values]
    log("Q3 current formal table and new Q23 unified trajectory: all four-decimal cells
    ↪  agree")
    q4csv=ROOT/"q4_complete_delivery/v1/output/table6_unrounded.csv"
    official=csv_rows(q4csv,"Current formal Q4 table 6 display crosscheck")[1:]
    times=np.array([float(row[0]) for row in official])
    indices=[match_time(q4["time_s"],t*3600) for t in times]
    values=np.c_[q4["sample_TC"][indices][:,positions,1],q4["surface_TC"][indices,1]]
    assert np.array_equal(np.isnan(values[:,:5]),q4["fixed_radius_m"][positions][None,:]>
    ↪  q4["radius_m"][indices,None]+1e-12)
    make_table("q4_moisture",times,values,"h",[q4path,q4csv],official)
    EXPECTED["q4_moisture"]["row_actual_radius_cm"]=[fmt(v*100) for v in
    ↪  q4["radius_m"][indices]]

    for name in ("q1_profiles","q2_profiles"):
        copied=[]
        for suffix in ("pdf","png"):
            original=record_source(ROOT/f"paper/q1_q2_stage/figures/{name}.{suffix}","Acc
            ↪  epted profile figure copied byte-for-byte")
            target=FIGURES/original.name
            shutil.copyfile(original,target)
            assert sha(original)==sha(target)
            OUTPUTS.append(target)
            copied.append({"from":rel(original),"to":rel(target),"sha256":sha(original)})
        FIGURE_RECORDS.append({"name":name,"operation":"byte-for-byte copy of accepted
        ↪  figure","copied":copied})
        log("FIGURE COPIED "+name)

    fig,axes=panels(height=7.5)
    qt=np.unique(np.r_[np.arange(0,206907,60),206906.76])
    qi=np.array([match_time(q23["time_s"],t) for t in qt])
```

154



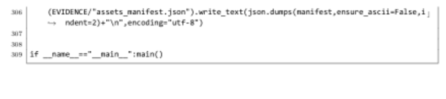
```
    series=[("Q3:
    ↪  固定半径",qt/3600,q23["profile_TC"][qi,0,1],q23["profile_TC"][qi,-1,1],57.4741),
            ("Q4: 规定收缩",q4["time_s"]/3600,q4["full_TC"][:,0,1],q4["surface_TC"][:,1],5
            ↪  1.0921)]
    for ax,(title,t,center,surface,execution) in zip(axes,series):
        ax.plot(t,center,color=COLORS[0],label=" 轴心")
        ax.plot(t,surface,color=COLORS[1],ls="--",label=" 当前表面")
        ax.axhline(.15,color="#666666",lw=.9,ls=":",label=" 达标阈值 0.15")
        ax.plot(execution,center[-1],"o",color=COLORS[0],ms=3.5)
        ax.annotate(f"{execution:.4f}
        ↪  h",xy=(execution,center[-1]),xytext=(execution-14,.245),
            fontsize=7.5,arrowprops={"arrowstyle":"-","lw":.6,"color":"#555555"})
        ax.set(xlabel="时间 / h",ylabel="干基含水率 /
        ↪  (kg/kg)",title=title,xlim=(0,60),ylim=(.045,2.9),yscale="log")
        ticks=[.05,.1,.15,.5,1.,2.5]
        ax.yaxis.set_major_locator(FixedLocator(ticks));ax.yaxis.set_major_formatter(Fixe
        ↪  dFormatter(["0.05","0.1","0.15","0.5","1","2.5"]))
        ax.minorticks_off();ax.xaxis.set_major_locator(MultipleLocator(12))
        ax.legend(loc="upper right",frameon=False,handlelength=2.2)
    save_figure(fig,"q3_q4_drying",{"sources":[rel(q23path),rel(q4path)],
        "panels":"Q3 left; Q4 right; shared x 0-60 h; logarithmic C axis;
        ↪  axis/surface/0.15 threshold; curves stop at official execution",
        "time_sampling":"Q3 every 60 s plus execution; Q4 all stored 60 s outputs plus
        ↪  execution"})

    radius_path,observed=read_radius()
    fig,axes=panels(height=7.5)
    axes[0].plot(observed[:,0]/3600,observed[:,1],color=COLORS[0],lw=1.2,label="附件2观测")
    axes[0].scatter(observed[::4,0]/3600,observed[::4,1],s=5,color=COLORS[0],zorder=3)
    axes[0].axvline(51.0921,color=COLORS[1],ls="--",lw=1,label=" 执行 51.0921 h")
    axes[0].set(xlabel="时间 / h",ylabel="外半径 / cm",title="(a)
    ↪  观测半径历程",xlim=(0,72),ylim=(1.15,2.04))
    axes[0].xaxis.set_major_locator(MultipleLocator(12));axes[0].legend(frameon=False,loc
    ↪  ="upper right")
    profile_hours=[6,12,24,51.0921]
    for h,color,ls in zip(profile_hours,COLORS,STYLES):
        i=match_time(q4["time_s"],h*3600)
        r_cm=q4["radius_m"][i]*100*np.sqrt(q4["x"])
        c=q4["full_TC"][i,:,1]
        axes[1].plot(r_cm,c,color=color,ls=ls,label=(f"{h:g} h" if h!=51.0921 else "结束
        ↪  51.0921 h"))
        axes[1].plot(r_cm[-1],c[-1],"o",ms=2.5,color=color)
        assert np.all(r_cm<=q4["radius_m"][i]*100+1e-12)
    axes[1].axhline(.15,color="#aaaaaa",lw=.7,ls=":")
    axes[1].set(xlabel="当前径向距离 / cm",ylabel="干基含水率 / (kg/kg)",title="(b)
    ↪  当前材料域内剖面",xlim=(0,1.45),ylim=(0,1.84))
    axes[1].xaxis.set_major_locator(MultipleLocator(.5));axes[1].legend(frameon=False,loc
    ↪  ="upper right",fontsize=7.)
    save_figure(fig,"q4_shrinkage",{"sources":[rel(radius_path),rel(q4path)],"profile_hou
    ↪  rs":profile_hours,
        "panels":"left observed R over 0-72 h; right actual-r profiles, points terminate
        ↪  at current surface; no extrapolation outside material"})

    fig,axes=panels(height=7.5)
    scenarios=["temperature_minus_1K","temperature_plus_1K","boundary_minus_0p005","bound
    ↪  ary_plus_0p005"]
    labels=[" 温度 -1 K"," 温度 +1 K"," 边界 b -0.005"," 边界 b +0.005"]
    plotdata=[]
    for ax,case,title in zip(axes,("q3","q4"),("Q3: 固定半径","Q4: 规定收缩")):
        base_path=record_source(ROOT/f"q1234_overall_review_delivery/v1/evidence/environm
        ↪  ent/{case}_baseline_n40.json","Same-n40 environment baseline")
        baseline=json.loads(base_path.read_text(encoding="utf-8"))
        changes=[]
        for scenario in scenarios:
```

155



```
            path=record_source(ROOT/f"q1234_overall_review_delivery/v1/evidence/environme
            ↪  nt/{case}_{scenario}_n40.json","Executed environment root difference")
            data=json.loads(path.read_text(encoding="utf-8"))
            assert data["case"]==case and data["n"]==40
            change=(data["critical_s"]-baseline["critical_s"])/3600
            changes.append(change);plotdata.append({"case":case,"scenario":scenario,"delt
            ↪  a_root_h":change,"source":rel(path)})
        y=np.arange(4)
        ax.barh(y,changes,color=[COLORS[1] if d>0 else COLORS[0] for d in
        ↪  changes],height=.56)
        ax.axvline(0,color="#666666",lw=.8)
        for yy,value in zip(y,changes):
            if abs(value)>.3:
                # Large-bar labels sit inside bars, avoiding overlap with the
                # Chinese y tick labels outside the plotting rectangle.
                ax.text(value-(.07 if value>=0 else
                ↪  -.07),yy,f"{value:+.3f}",va="center",ha="right" if value>=0 else
                ↪  "left",fontsize=7.3,color="white")
            else:
                ax.text(value+(.06 if value>=0 else
                ↪  -.06),yy,f"{value:+.3f}",va="center",ha="left" if value>=0 else
                ↪  "right",fontsize=7.3)
        ax.set_yticks(y,labels,fontsize=7.3);ax.invert_yaxis()
        ax.set(xlabel=" 临界根变化 / h",title=title,xlim=(-2.45,2.45))
        ax.xaxis.set_major_locator(MultipleLocator(1))
        ax.grid(axis="y",visible=False);ax.grid(axis="x",color="#D7DCE0",lw=.5)
    fig.subplots_adjust(left=.16,right=.975,wspace=.60)
    save_figure(fig,"environment_sensitivity",{"panels":"Both are analyst-selected changes
    ↪  after 4 h, not error bars or confidence intervals; Q4 R(t) unchanged",
    ↪  "data":plotdata})

    expected_path=EVIDENCE/"table_expected.json"
    expected_document={"schema_version":1,"scope":"Body cells, including times; all
    ↪  nonblank values are exact desired PDF strings.",
        "tables":EXPECTED,"total_numeric_body_cells":sum(v["body_numeric_cell_count"] for v
        ↪  in EXPECTED.values()),
        "total_temperature_moisture_cells":sum(v["moisture_or_temperature_numeric_cell_co
        ↪  unt"] for v in EXPECTED.values()),
        "total_domain_empty_cells":sum(v["domain_empty_cell_count"] for v in
        ↪  EXPECTED.values())}
    expected_path.write_text(json.dumps(expected_document,ensure_ascii=False,indent=2)+"\
    ↪  n",encoding="utf-8")
    OUTPUTS.append(expected_path)
    for source,description in SOURCES.items():assert
    ↪  sha(ROOT/source)==description["sha256"],source
    assert len(EXPECTED)==6 and len(FIGURE_RECORDS)==5
    log(f"FINAL PASS: 6 tables; {expected_document['total_temperature_moisture_cells']}
    ↪  temperature/moisture cells and {expected_document['total_domain_empty_cells']}
    ↪  domain-empty cells; 5 figure PDFs + PNG previews; all frozen source hashes
    ↪  unchanged")
    logfile=EVIDENCE/"assets_build.log"
    logfile.write_text("\n".join(MESSAGES)+"\n",encoding="utf-8")
    OUTPUTS.append(logfile)
    manifest={"git_commit_read":"anonymous-code-snapshot",
        "script":rel(Path(__file__)),"script_sha256":sha(Path(__file__)),
        "runtime":{"python":platform.python_version(),"numpy":np.__version__,"scipy":scip
        ↪  y.__version__,"matplotlib":matplotlib.__version__,"executable":sys.executable
        ↪  ,"font":font},
        "scope":"Frozen evidence postprocessing only; no PDE runs; generated files replace
        ↪  only named draft asset outputs.",
        "sources":SOURCES,"figures":FIGURE_RECORDS,
        "outputs":{rel(path):{"sha256":sha(path),"bytes":path.stat().st_size} for path in
        ↪  OUTPUTS}}
```

156



```
    (EVIDENCE/"assets_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,i
    ↪  ndent=2)+"\n",encoding="utf-8")


if __name__=="__main__":main()
```